Make Your First GAN With PyTorch

A gentle introduction to Generative Adversarial Networks,
and a practical step-by-step tutorial on making your own with PyTorch.

Contents

Introduction

AI is Exploding!

Machine learning and artificial intelligence has exploded in the last few years, with amazing achievements being reported in the news every few months.

Not only does your smartphone understand you when you talk to it, it is pretty good at translating between many human languages. Self-driving cars are now able to drive as safely as humans, and machines can now diagnose some diseases more accurately and sooner than experienced doctors.

The ancient Chinese game of Go has been played for 3,000 years, and although the rules are simpler than chess, the game itself is much more complex requiring longer-term strategies. Only very recently researchers were able to guide a machine learning system to play and beat a world champion for the first time. That machine learning system also discovered new game strategies that we think had not been discovered by humans in those 3,000 years!

Beyond learning to perform a task, the discovery of new strategies is a profound success for the field of machine learning.

Creative AI

In October 2018, the prestigious auction house Christies sold a portrait for $432,500. That painting was not painted by a person, but was generated by a neural network. A portrait painted using AI and sold for almost half a million dollars is a milestone in the history of art.

That neural network was trained using a new and exciting technique called adversarial training. The architecture is called a generative adversarial network, or GAN.

GANs are attracting a lot of interest, particularly from the creative technology sector, because they can create images that look very plausible. Those images are not made by simply copying and pasting parts of the training examples, and they're not mushy averages of the training data either. This is what sets GANs apart from most other forms of machine learning. GANs are learning to create images at a level above merely replicating or averaging training data.

Yann LeCun, one of the world's leading researchers on neural networks called GANs "The coolest idea in deep learning in the last 20 years."

GANs Are New

Compared to the decades of research and refinement behind traditional neural networks, GANs only came to prominence in 2014 with Ian Goodfellow's now seminal paper.

That means GANs are very new, and the creative possibilities are only starting to be explored.

It also means we don't yet fully understand how to train them as effectively as we can with traditional networks. When they work, they work spectacularly well. But too often they fail. Research into how GANs work, and why they can fail, is currently very active.

Who This Book Is For

This guide is for anyone who wants to take their first steps to understanding what GANS are and how they work. It's also for anyone who wants to learn how to actually build them with industry standard tools.

This guide will try to use friendly simple language and make use of lots of pictures to visually explain ideas. It will avoid unnecessary jargon and minimise the use of mathematical equations.

My aim is for as many readers as possible, from as many backgrounds as possible, to understand GANs and get excited about making their own.

This book won't try to exhaustively cover every topic or be an encyclopedia of GANs. It intentionally tries to cover the minimum to give you a genuine foundation on which you can go on to explore further.

Students of machine learning courses will find this book a good primer for further study.

How To Use This Book

The best way to learn and understand a concept is by doing it. That's why this book develops ideas and theory around a practical step-by-step journey.

This guide will accompany you on that journey where we sometimes fail before finding a solution. Experiencing failure, and working through the remedies, is actual experience, and much more valuable than simply reading a theoretical guide to GANs.

Neural Networks

GANs are made of neural networks. Although this guide will refresh your understanding of them, my previous book *Make Your Own Neural Network* is dedicated to providing the gentlest introduction to neural networks and how they work. It also provides an introduction to calculus and gradient descent, which is useful for this journey into GANs.

- https://www.amazon.com/Make-Your-Own-Neural-Network/dp/1530826608

It also provides an introduction to programming in python, covering just enough to be productive in building simple neural networks.

Free and Open Source

All the tools and services presented here to build your own GANs are either free, or free and open source. This is important to ensure as few people as possible are excluded from learning about and building neural networks and GANs.

Python is one of the most popular and easy to learn programming languages, and has become the standard in machine learning and AI, with a vibrant global community and healthy ecosystem of libraries.

Google currently provides a free web-based python environment called Google Colab which means you don't need to install python, or any software at all. You can develop and run powerful neural networks entirely in Google's infrastructure using only a modern web browser, from a very modest computer or laptop.

PyTorch is an extension of python that makes it easy to design, build and run machine learning models. Alongside TensorFlow it is amongst the most popular machine learning frameworks.

All these tools are also the industry standard, so you're learning valuable reusable skills.

Author's Note

I will have failed in my mission if any reader has trouble understanding what GANs are, and they are trained. I will also have failed if you haven't been able to build your own simple GAN.

The content of this guide has been tested with a range of students and researchers, so if something doesn't make sense please do get in touch on twitter @myoneuralnet, by email makeyourownneuralnetwork@gmail.com or on github:

- https://github.com/makeyourownneuralnetwork/gan

Additional discussion and responses to interesting reader questions are written up on the blog that accompanies this and the previous book:

- https://makeyourownneuralnetwork.blogspot.com

Finally, I'd really recommend joining your local machine learning or algorithmic art community. Learning in a supportive group is much more effective, and it's fun sharing your work and being inspired by what others come up with.

Have fun!

Part 1

We begin learning about PyTorch, and practice using it by building a simple image classifier, refreshing our understanding of neural networks.

PyTorch Basics

In *Make Your Own Neural Network*, we created simple but effective neural networks using only python and the **numpy** library for working with arrays of data.

We didn't use popular frameworks for building neural networks like PyTorch and TensorFlow because it was important to really understand how they work by building them from scratch.

Having done that hard work, we can see that building larger networks could be a laborious task. One of the most laborious parts is doing the calculus to work out the relationship between the back-propagated error and weights in our network. If we decide to change our network we potentially have to do all this work again.

Here we'll be using **PyTorch** because it takes away a lot of that leg work, and lets us focus on thinking about the design of our networks.

One of the most powerful and convenient features of PyTorch is that it does all the calculus for us, whatever shape or size of network we dream up. And if we change our minds about the design of the network, PyTorch automatically works out the new calculus without us having to pull out a pencil and paper to work out the gradients again.

PyTorch also tries really hard to look and feel just like normal python. This means it's easy to learn if you know python already, and there are fewer surprises when using it.

Google Colab

In *Make Your Own Neural Network* we wrote code using web-based python notebooks that ran on our own computer. Here we'll be using python notebooks provided by Google's free **Colab** service which runs our code on Google's own computers.

Google's Colab service is accessed entirely through a web browser. There is no need to install any software on our own computer or laptop.

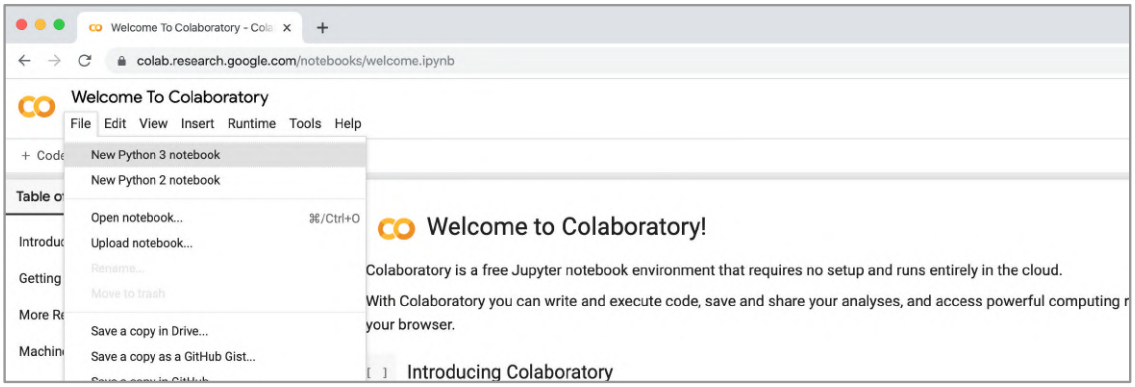
Before we get started, ensure you are logged in with a Google account. If you have a gmail or youtube account, that is your google account. If you don't have a google account, you can create one through this link:

- https://accounts.google.com/signup

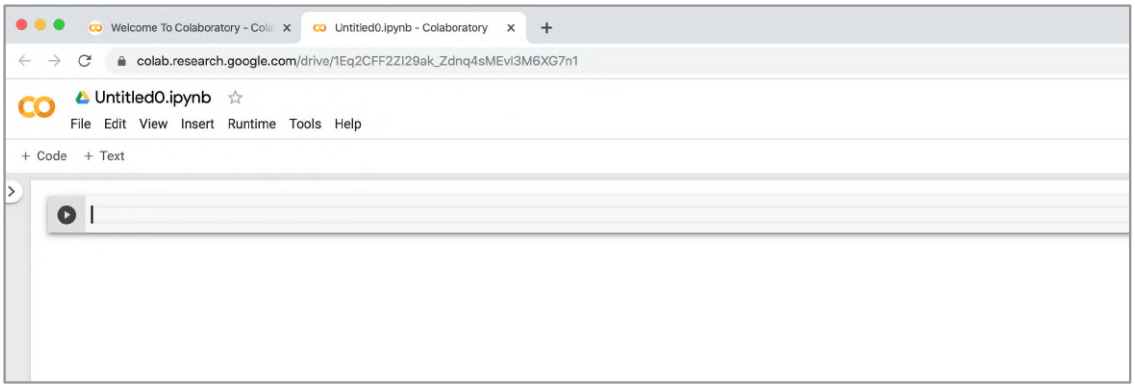
Once you're signed in, activate the Google Colab service by visiting the following link:

- https://colab.research.google.com

You'll be taken to a sample python notebook. From the **File** menu, create a fresh new notebook by choosing **New Python 3 notebook** like this:

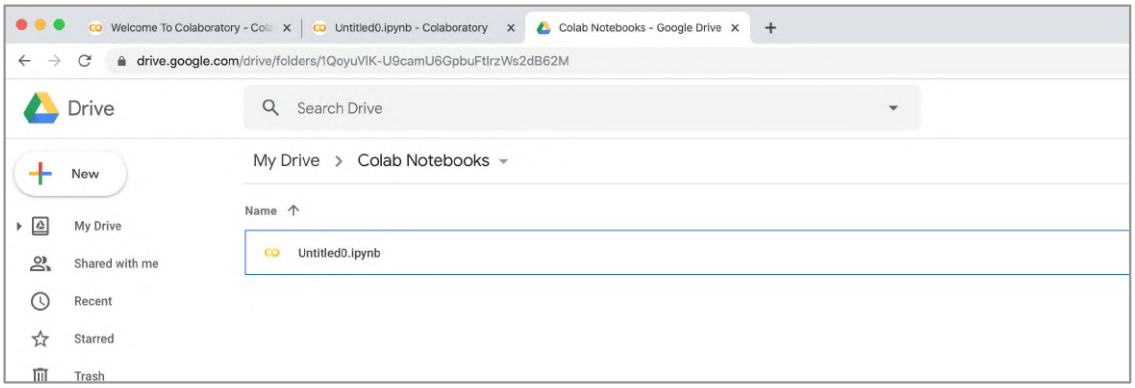


You should see an empty python notebook, ready for us to use:



In a separate browser tab, if you look at the google file store **Drive**, you'll notice a new folder called **Colab Notebooks**. This is the folder in which new python notebooks are stored by default.

Below you can see our new notebook called **Untitled0.ipynb**.

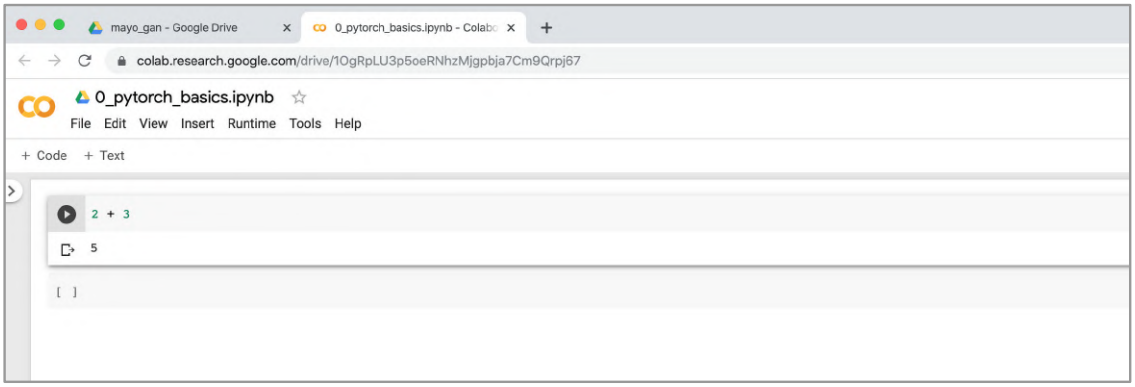


Let's check that we can run python code. In the first cell type the following very simple code:

```
2 + 3
```

Click the play button at the left of the cell to run the code. If you've not recently used the Colab service, it can take some time to run this first python instruction because Google takes a few moments to start a virtual machine and connect your notebook to it.

Eventually you should see the answer **5** appear, like this:



Great! Everything is working and we're ready to learn about PyTorch.

PyTorch Tensors

Before we can use PyTorch we need to import the python **torch** module. Luckily Google's Colab service has many of the popular machine learning libraries, including PyTorch, ready for us to use simply by importing them. There's no need to go through a complicated installation process.

Enter the following into the first cell and run it.

```
import torch
```

A good way to start understanding PyTorch is to compare its basic units of information with plain python. In plain python, we use **variables** to store numbers. We can use these variables like mathematical symbols to calculate new values, and store these in new variables if we want to.

Have a look at the following plain python code:

```
# normal python variables
x = 3.5
y = x*x + 2
print(x, y)
```

We create a variable called **x** and give it a value **3.5**. We then create a new variable called **y** and give it a value that is calculated from the expression **x*x + 2**, which is **(3.5*3.5) + 2**, or **14.25**. Finally, we print the values of **x** and **y**.

Enter the code into a new cell and run it. You should get the expected answer like this:

PyTorch has its own kind of variable for storing numbers. They're called PyTorch **tensors**. Let's create a very simple one.

```
# simple pytorch tensor
x = torch.tensor(3.5)
```

```
print(x)
```

We can see that we're creating something called **x**. That thing appears to be a PyTorch **tensor**, initialised with a value of **3.5**.

Enter and run the code to see what printing **x** does.

The output tells us that the numerical value is **3.5000** but also that it is contained in a PyTorch **tensor**. Being able to see what kind of container that number is in is useful.

Let's do some simple arithmetic with this tensor. In the next cell type the following code.

```
# simple arithmetic with tensors
y = x + 3
print(y)
```

Here we're creating a new variable **y** from the expression **x + 3**. We just created **x** as a PyTorch tensor with value **3.5**. So what will **y** be?

Try it.

We can see **y** has a value of **6.5**, which makes sense because **3.5 + 3 = 6.5**. We see that **y** is also a PyTorch **tensor**.

You'll remember that **numpy** arrays work in the same way too. This familiarity is nice, and the consistency with numpy helps us learn PyTorch more easily.

Automatic Gradients With PyTorch

Let's now see how PyTorch is different to plain python and numpy, and makes it so special.

The following code creates a tensor called **x**, just like before, but this time we give PyTorch an additional option **requires_grad=True**. We'll see very soon what this option does.

```
# pytorch tensor
x = torch.tensor(3.5, requires_grad=True)
print(x)
```

Run the code to see what printing **x** does.

We can see that **x** has a value of **3.5000** and is of type **tensor**. The output also shows that the tensor **x** has the **requires_grad** option set to **True**.

Let's create a new variable **y** from **x**, just like before but this time using a different expression.

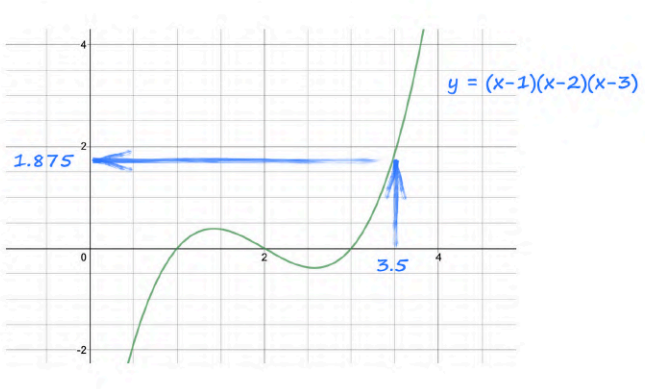
```
# y is defined as a function of x
y = (x-1) * (x-2) * (x-3)
print(y)
```

We can see that **y** is calculated from the expression **(x-1) * (x-2) * (x-3)**. Let's run the code.

```
[34] # pytorch tensor with gradient enabled

     x = torch.tensor(3.5, requires_grad=True)

     print(x)

     tensor(3.5000, requires_grad=True)

     # y is defined as a function of x

     y = (x-1) * (x-2) * (x-3)

     print(y)

     tensor(1.8750, grad_fn=<MulBackward0>)

[ ]
```

As expected, the value of **y** is **1.8750**. This is because **x** is **3.5**, and so **(3.5-1) * (3.5-2) * (3.5-3)** is **1.8750**.

The following graph of **y = (x-1) * (x-2) * (x-3)** illustrates what we've just calculated.



So far, everything seems normal and familiar.

Actually, PyTorch has done extra work that we didn't see. PyTorch has not just calculated the value **1.8750** and placed it inside a tensor called **y**.

PyTorch has actually remembered that **y** is defined mathematically in terms of **x**.

If **x** and **y** were normal python variables, or even numpy arrays, python would not remember that **y** came from **x**. There's no need for it to. Once the value of **y** is calculated, from **x**, the only important thing is that value, and it is placed inside **y**. Job done.

PyTorch tensors work differently. They remember which other tensors they are calculated from, and how. Here, PyTorch remembers that **y** came from **x**.

Why is this useful? Let's see.

You'll remember that the calculations for training a neural network require the error gradient to be worked out using calculus. That is, the rate at which the output error changes as a result of changing network link weights.

The output of a neural network is worked out from the link weights. That output depends on the weights just like **y** depends on **x**. So let's see how PyTorch can work out the rate of change of **y** as **x** varies.

Let's work out the gradient of **y** at **x = 3.5**. That is, **dy/dx** at **x = 3.5**.

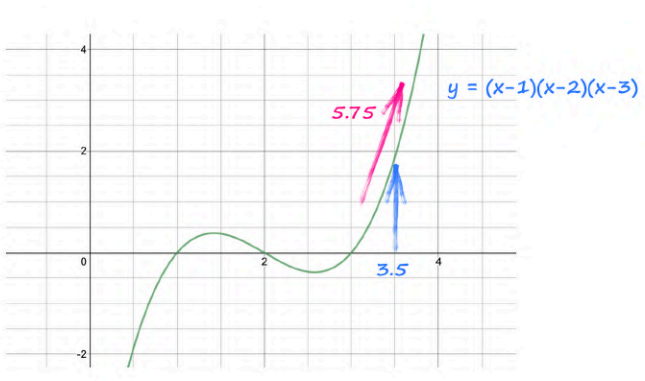
To do this PyTorch needs to look at **y** and see which tensors it depends on, and the mathematical form of that dependency. It can then do the calculus to work out **dy/dx**.

```
# work out gradients
y.backward()
```

This single instruction does all that. It looks at **y** and sees that it came from **(x-1) * (x-2) * (x-3)** and automatically works out the gradient **dy/dx** which, if you did the working out, would be **$3x^2$ - 12x + 11**.

That instruction also works out the numerical value of that gradient and places it inside the tensor **x** alongside the actual value of **x**. Because **x** is **3.5**, the gradient is **3*(3.5*3.5) - 12*(3.5) + 11 = 5.75**.

The following graph illustrates where we've calculated this gradient.



That's an impressive amount of work from **y.backward()**, just one single instruction!

We can take a peek at the numerical value of the gradient that was placed inside the tensor **x**.

```
# what is gradient at x = 3.5
x.grad
```

Run the code to check it works correctly.

It worked!

That **requires_grad=True** option we set for the **x** tells PyTorch that we will be interested in working out a gradient with respect to **x**.

We can see now that PyTorch tensors are richer than normal python variables and numpy arrays. A PyTorch tensor can contain

- additional information beyond the primary numerical value, such as a gradient value.
- information about which others tensors it depends on, and the mathematical form of that dependency.

What we've seen is a simple example of a very powerful capability. This ability to link tensors and do automatic differentiation is the single most important feature of PyTorch. Almost all of the other features need it to be useful themselves.

You can find a notebook of the code we've just been working with online:

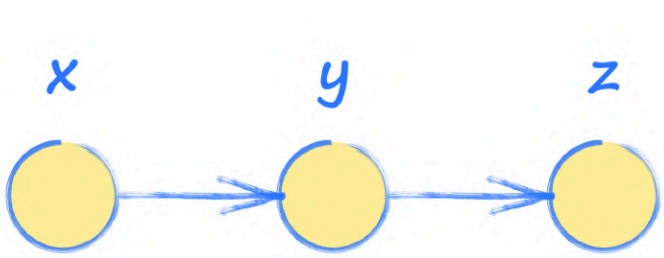
- https://github.com/makeyourownneuralnetwork/gan/blob/master/00_pytorch_basics.ipynb

Computation Graphs

That automatic gradient calculation we just saw seems magical, but of course it isn't.

It is worth understanding just a little bit about how it is done, because this knowledge will help us later when we build larger networks.

Take a look at the following very simple network. It's not a neural network, just a sequence of calculations.



In this picture we can see an input **x**, which is used to calculate **y**, which is then used to calculate **z**, the output.

Imagine that **y** and **z** are calculated as follows:

$$y = x^2$$

$$z = 2y + 3$$

If we want to know how the output **z** varies as **x** varies, we need to do some calculus to work out the gradient **dy/dx**. Let's do this step by step.

$$dz/dx = dz/dy \cdot dy/dx$$

$$dz/dx = 2 \cdot 2x$$

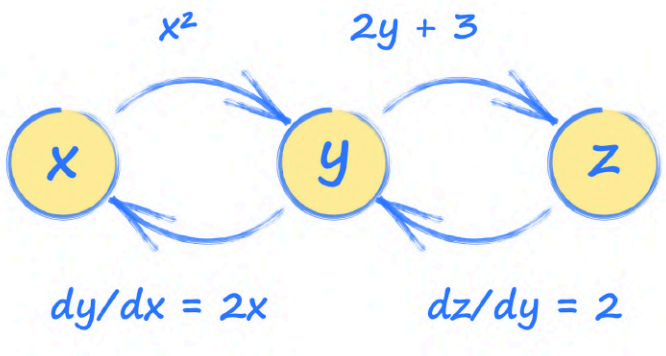
$$dz/dx = 4x$$

The first line is the **chain rule of calculus** and it really helps us get started here. One of the appendices of *Make Your Own Neural Network* is an introduction to calculus and the chain rule if you need a refresher.

So we've just worked out that the output **z** varies with **x** as **4x**. If **x = 3.5**, then the **dz/dx** is **4*3.5 = 14**.

When **y** is defined in terms of **x**, and **z** in terms of **y**, PyTorch builds a picture of how these tensors are connected. That picture is called a **computation graph**.

For our example, it might look like this:



We can see how **y** is calculated from **x**, and how **z** is calculated from **y**. In addition, PyTorch adds backwards arrows which show how **y** changes as **x** changes, and how **z** changes as **y** changes. These are the gradients used to update a neural network during training. The calculus is done by PyTorch, we don't need to do the working out ourselves.

To work out how **z** changes as **x** changes, we can follow the path back from **z** back to **x** via **y**, combining the gradients. This is effectively the chain rule of calculus.

Let's check it works in code. In a new notebook, after importing **torch**, enter the following code to set up the relationships between **x**, **y** and **z**.

```
# set up simple graph relating x, y and z
x = torch.tensor(3.5, requires_grad=True)
y = x*x
z = 2*y + 3
```

PyTorch will build the computation graph but only in the forward direction. We need to use the **backward()** function to get PyTorch to work out the gradients in the backwards direction.

```
# work out gradients
z.backward()
```

The gradient **dz/dx** will be inside the **x** tensor as **x.grad**.

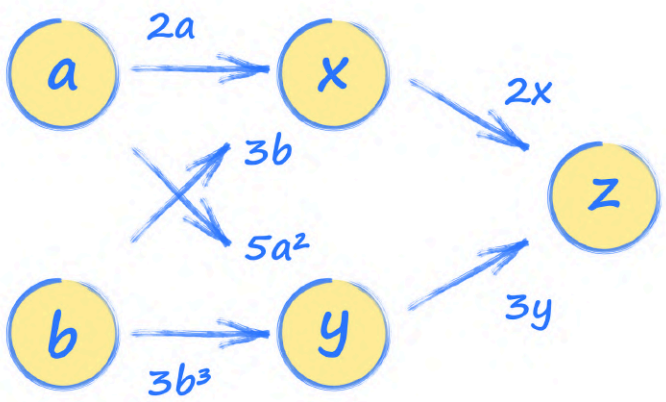
```
# what is gradient at x = 3.5
x.grad
```

Run the code to check it works.

The answer is **14**, exactly what we worked out by hand above. Great!

It is worth noting that the gradient value inside the **x** tensor relates to how **z** changes. That's because we asked PyTorch to work backwards from **z** using **z.backward()**. So **x.grad** is **dz/dx** not **dy/dx**.

Most useful neural networks have nodes with multiple connections into them and emerging from them. Let's look at another simple example that has more than one connection into a node.



Here we can see that the inputs **a** and **b** both contribute to **x** and **y**, and the output **z** is calculated from **x** and **y**.

Let's write out the relationships between these nodes.

$$x = 2a + 3b$$

$$y = 5a^2 + 3b^3$$

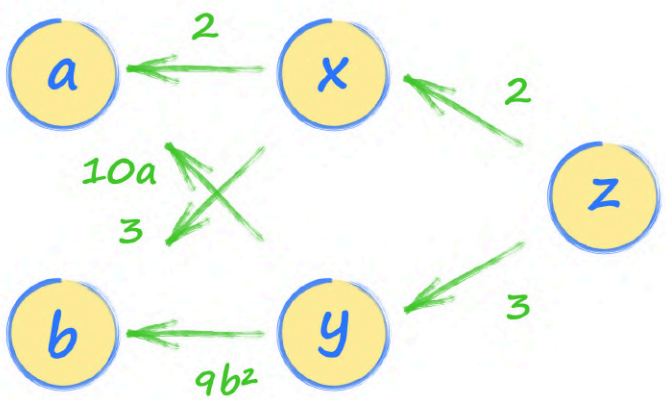
$$z = 2x + 3y$$

We can work out the gradients just as we did before.

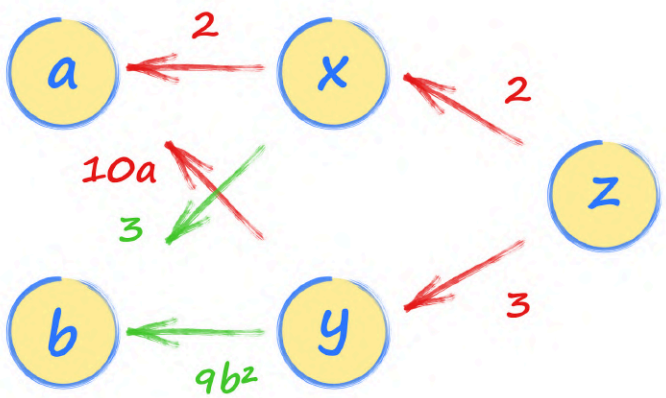
$$dz/dx = 2 \qquad dx/da = 2 \qquad dy/da = 10a$$

$$dz/dy = 3 \qquad dx/db = 3 \qquad dy/db = 9b^2$$

Let's add this information to the computation graph.



We can now easily work out the gradient **dz/da** by following the path back from **z** to **a**. There are actually two paths, one going through **x** and the other through **y**. That's ok, we follow them both and simply add the expressions from both paths together. This makes sense because both paths from **a** to **z** contribute to the value of **z**. This is also what would have happened if we had worked out **dz/da** using the chain rule of calculus.



The first path through **x** gives us **2 * 2** and the second path through **y** gives us **3 * 10a**. So the rate at which **z** changes with **a** is **4 + 30a**.

If **a** is **2**, then **dz/da** is **4 + 30*2 = 64**.

Let's check to see if this is what PyTorch calculates too. First we set up the relationships so that PyTorch can build its computation graph.

```
# set up simple graph relating x, y and z
a = torch.tensor(2.0, requires_grad=True)
```

```
b = torch.tensor(1.0, requires_grad=True)

x = 2*a + 3*b
y = 5*a*a + 3*b*b*b
z = 2*x + 3*y
```

We then trigger the gradient calculations and ask for the value inside the tensor **a**.

```
# work out gradients
z.backward()

# what is gradient at a = 2.0
a.grad
```

Let's see if PyTorch calculates the answer just as we did.

It does!

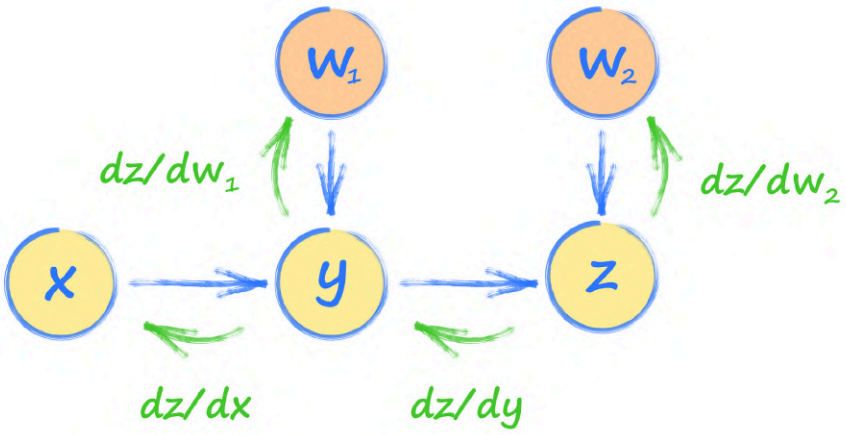
Useful neural networks are usually larger than this small network, but PyTorch builds a computation graph in exactly the same way, and uses the same method of following paths backwards to compute the gradients.

It might not be clear how what we've just seen relates to the **error** from a neural network and updating the internal weights. Let's talk about this.

If the output of a network is **z**, like our simple network above, and the correct output should be **t**, the error **E** is **(z-t)**, or more commonly $\mathbf{(z-t)^2}$. That error **E** is just another node at the end of the network with $\mathbf{(z-t)^2}$ as the calculation from **z** to **E**. It is now effectively the output node, not **z**. PyTorch can then work out the gradient of the new output **E** with respect to the inputs to the network.

You'll remember from *Make Your Own Neural Network* that to train a neural network we use **dE/dw** where **w** is a weight in the network. We've been working with **dz/da** where **a** is an input, not a weight. Is this a problem? It's not because we can imagine weights as being nodes too.

The following picture shows how **z** depends on both the signal from **y** as well as the weight $\mathbf{w_2}$. That relationship could be $\mathbf{z = w_2 * y}$, familiar from neural networks.



We can see how $\mathbf{dz/dw_2}$ can be calculated in the same way as **dz/dy**. And just like before, we can follow the path back from **z** to $\mathbf{w_1}$ to find $\mathbf{dz/dw_1}$, for example.

What we've seen here is a slightly simplified version of what actually happens inside PyTorch, but the core ideas are the same. By understanding the models we've just talked about, we'll be able to build more sophisticated PyTorch networks and get them right more often.

The code we've just written to explore computation graphs is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/01_pytorch_computation_graph.ipynb

Enough theory, let's make a useful neural network with PyTorch.

Learning Points

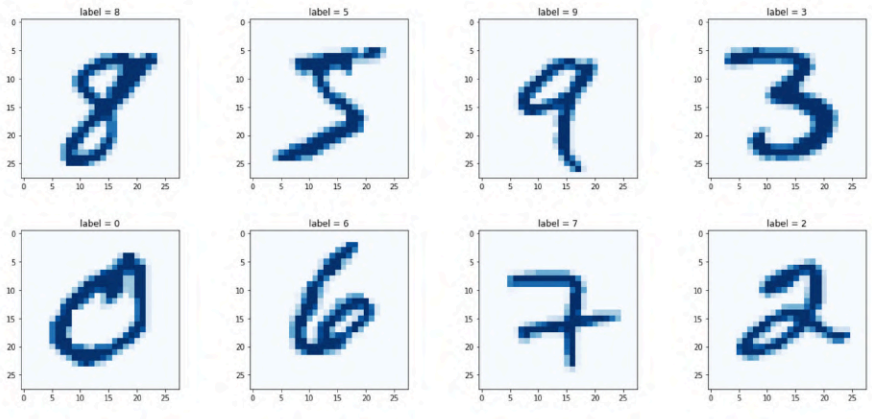
- Google's **Colab** lets us run python code on their computers. Colab uses python notebooks which means we only need a web browser.
- **PyTorch** is a leading machine learning framework for python. Similar to **numpy**, it allows us to work with arrays of numbers. It also provides a rich library of convenient tools and functions to make machine learning easier.
- In PyTorch, the basic unit of data is called a **tensor**. These can be multi-dimensional arrays, simple 2-dimensional grids, 1-dimensional lists, or even a single value.
- The key feature of PyTorch is the ability to automatically work out **gradients** for functions we've defined. Calculating gradients is essential for training neural networks. To do this, PyTorch builds a **computation graph** of tensors and how they're related to other tensors. It does this transparently when we define how one tensor is calculated from another in our code.

First PyTorch Neural Network

Let's continue to learn about PyTorch by building an actual neural network that performs a simple but familiar task.

MNIST Image Dataset

In *Make Your Own Neural Network* we developed a neural network that learned to classify images of hand-written digits.



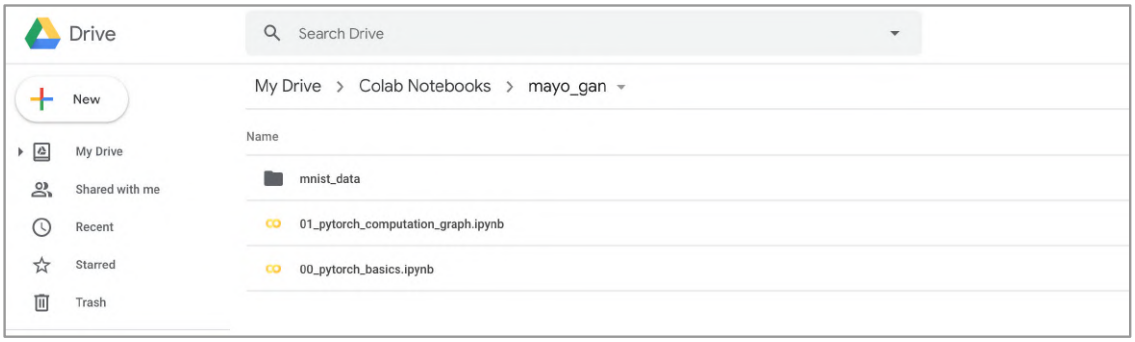
The **MNIST** dataset is a well-known set of images often used to measure and compare the performance of machine learning algorithms. It contains **60,000** images to be used for training a machine learning model, and **10,000** images for testing its performance.

The images are **28** by **28** pixels in size, and are monochrome, not colour. The pixels have a numerical value for how light or dark that pixel is and, depending on where you get the data, the values will be from **0** to **255**.

Getting MNIST Data

Go to your **Colab Notebooks** folder in Drive where our python notebooks are stored. Use the **New** button to create a new folder called **mnist_data**.

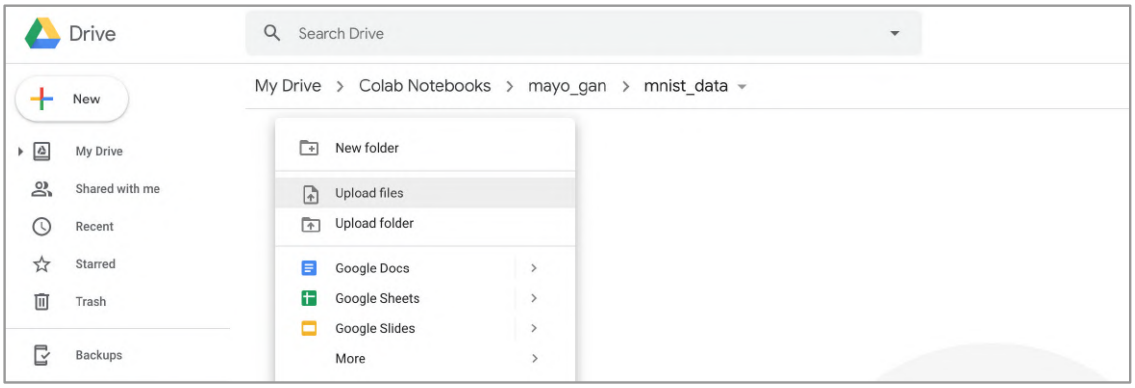
This **mnist_data** folder should be in the same folder as our notebooks, like this.



Download the MNIST data to your computer using the following links:

- Training data https://pjreddie.com/media/files/mnist_train.csv
- Test data https://pjreddie.com/media/files/mnist_test.csv

After you've downloaded these two files, upload them to the **mnist_data** folder. You can do this by navigating to the **mnist_data** folder and using the **New** button to select **Upload Files**.



After a few moments, you should see the two files listed in the **mnist_data** folder.

A Look At The Data

It is a good habit to explore any new data to get a feel for it, before we attack it with tools and algorithms!

We need to make our uploaded data accessible to our python code. We do this by mounting our Drive so it appears as a folder.

Start a new notebook and run the following code.

```
# mount Drive to access data files
from google.colab import drive
drive.mount('./mount')
```

You'll be prompted to click a link which takes you to a new tab. That tab asks you to confirm the account and to grant permissions for the drive to be mounted and made accessible. You'll then be given a code to copy and paste into the notebook cell.

Once that's done, we get a confirmation that Drive has been mounted and is visible to our python code from the folder **./mount**.

Our MNIST data files are in **CSV** format, that is, rows of **comma separated values**. There are many ways to load the data and view it. We'll use the **pandas** library because it makes the task really easy.

In a new cell, import the **pandas** library.

```
# import pandas to read csv files
import pandas
```

In the next cell let's use pandas to load the training data into a dataframe. The following is a single line of code. It's only long because the file path to the **mnist_train.csv** file is long.

```
df = pandas.read_csv('mount/My Drive/Colab
Notebooks/myo_gan/mnist_data/mnist_train.csv', header=None)
```

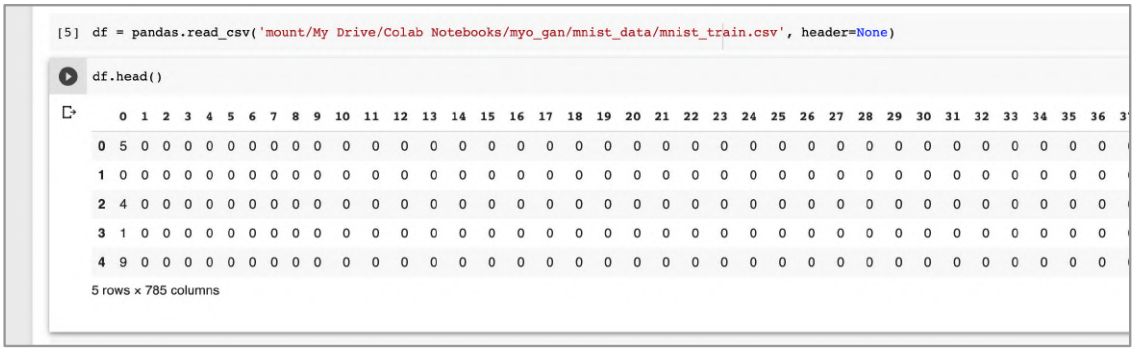
Check that the file path you use matches where you've placed your files. My own code and data all sit within a **myo_gan** folder inside **Colab Notebooks**.

A pandas **dataframe** is like a **numpy** array, but with additional features like named columns and rows, and many convenience functions like summing and filtering data.

We can take a peek at the top of a large dataframe just by using the **head()** function.

```
df.head()
```

This shows the first five rows of the dataset.



Each row of the MNIST data consists of **785** values. The first number is the digit that the image is supposed to be, and the remaining **784** are the pixel values for the **28** by **28** image.

We can get an overview of the dataframe using the **info()** function.

The summary tells us the dataframe has **60,000** rows. That's the **60,000** training images. It also confirms that we have **785** values per row.

Let's visualise the data by turning a row of pixel values into an actual image.

We'll use the versatile **matplotlib** library. Update the cell which imports libraries to include the **pyplot** package from the **matplotlib** library.

```
# import pandas to read csv files
import pandas
```

```
# import matplotlib to show images
import matplotlib.pyplot as plt
```

Run the updated cell again to make **pyplot** available.

Let's look at the following code.

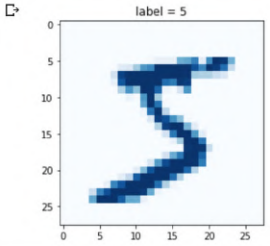
```
# get data from dataframe
row = 0
data = df.iloc[row]

# label is the first value
label = data[0]

# image data is the remaining 784 values
img = data[1:].values.reshape(28,28)
plt.title("label = " + str(label))
plt.imshow(img, interpolation='none', cmap='Blues')
plt.show()
```

The first part chooses which image from the MNIST data we're interested in. The first image, which is the first row, is chosen by setting **row = 0**. The **df.iloc[row]** picks out the first row of the dataset and assigns it to **data**.

In the next part we pick the first number from that row and call it **label**, which is what it is.

The third part takes the remaining **784** values from that row and reshapes them to a **28** by **28** square array and assigns it to a variable named **img**, because it is the image. It then draws the array as a bitmap, with a title showing the **label** we extracted from the data. The **imshow()** function, which draws the bitmap, can have lots of options, and the two we've used set the colour palette and tell pyplot not to smooth the pixels.

```
# get data from dataframe
row = 0
data = df.iloc[row]

# label is the first value
label = data[0]

# image data is the remaining 784 values
img = data[1:].values.reshape(28,28)
plt.title("label = " + str(label))
plt.imshow(img, interpolation='none', cmap='Blues')
plt.show()
```



We can see the first image in the MNIST training dataset. It looks like a five, and the label confirms it is supposed to be five.

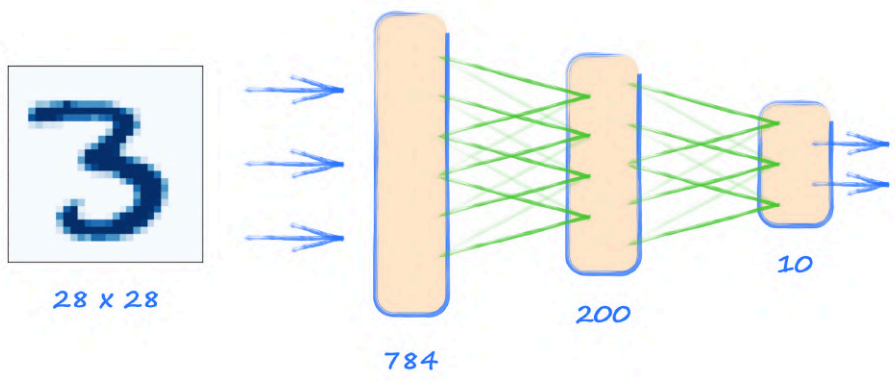
Experiment with other images from the dataset by changing the **row** value and running the cell again. For example, if you try **row = 13**, you should see an image that looks like a six.

The code we've developed so far is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/02_mnist_data.ipynb

Simple Neural Network

Before we dive into writing code for a neural network, let's step back and draw a picture of what we're trying to achieve. The following picture shows our starting point and where we want to end up.



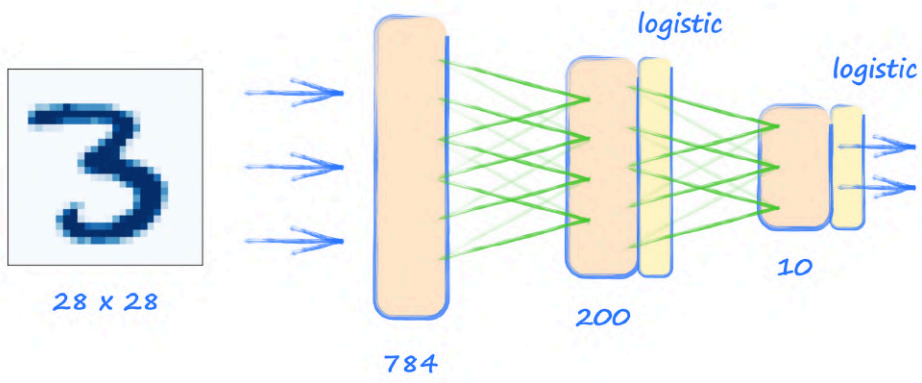
The start is an MNIST image which is **28** by **28**, or **784**, pixel values. That means the first layer of our neural network must have **784** nodes. We don't really have much of a choice about the size of the input layer.

We do have some choice about the last output layer. It needs to be able to answer the question "which digit is this?". The answer could be any one of the digits from **0** to **9**, which is **10** different possibilities. The simplest approach is to have one node for each of these **10** possible classes.

We do have more choice about the hidden middle layer. Because we're focusing on learning about PyTorch and not trying to find the best size, we'll use what we learned in *Make Your Own Neural Network*, and pick a size of **200** to get us going.

All the nodes in one layer are connected to every node in the next layer. These are sometimes called **fully connected layers**.

That picture is missing one key thing. We need to choose an **activation function** that is applied to the outputs from hidden and output layers. In *Make Your Own Neural Network* we used the s-shaped **logistic** function. Let's go with that for simplicity.



We're ready to turn that neural network **architecture** into PyTorch code. PyTorch simplifies the task of building and running neural networks by doing a lot of the work behind the scenes. For this to work we need to follow PyTorch's coding patterns.

Whenever we create a neural network class we need to inherit from PyTorch's own **torch.nn**. This brings with it a lot of PyTorch machinery, for example, automatically building the computation graph, looking after the weights, and updating them during training.

In a new notebook, start by importing both **torch** and **torch.nn**.

```
# import libraries

import torch
import torch.nn as nn
```

The **torch.nn** module is imported as **nn**, which has become the naming convention.

Let's start building our neural network class. The following shows a class which we've called **Classifier**. We can see it inherits from **nn.Module**.

```
class Classifier(nn.Module):

    def __init__(self):
        # initialise parent pytorch class
        super().__init__()
```

The **__init__(self)** function is a special function called when an object is created from a class. It is often used to set up an object and get it ready for use. You may have heard it called a **constructor**, which is a fairly descriptive name. Here that set up is **super().__init()** which looks

mysterious but is simply calling the parent class' constructor. So PyTorch's **nn.Module** will be setting up our **Classifier** for us. Neat!

Let's now define the architecture of the neural network. There are different approaches for doing this. For simple networks we can use **nn.Sequential()** to provide a list of network elements. The elements must be provided in the order we want information to be passed through them.

```
class Classifier(nn.Module):

    def __init__(self):
        # initialise parent pytorch class
        super().__init__()

        # define neural network layers
        self.model = nn.Sequential(
            nn.Linear(784, 200),
            nn.Sigmoid(),
            nn.Linear(200, 10),
            nn.Sigmoid()
        )
```

Here we can see that following elements defined in **nn.Sequential()**:

- **nn.Linear(784, 200)** a fully-connected mapping from **784** nodes to **200** nodes. This is the element which contains the weights for the links between the nodes that are updated during training.

- **nn.Sigmoid()** applies the s-shaped logistic activation function to the outputs of the previous element, in this case the **200** nodes.

- **nn.Linear(200, 10)** maps **200** nodes to **10** nodes. This contains the weights for the links between the middle hidden layer and the final layer of **10** output nodes.

- **nn.Sigmoid()** applies the s-shaped logistic activation function to the output of the **10** nodes. The result is the final output of the network.

You might be wondering why **nn.Linear** is called that. It's because it applies a linear function of the form **Ax + B** to the values as they pass from the input to the output. The **A** is the link weight and the **B** can be considered a bias. Both of these parameters are updated during training. You'll hear some people call them **learnable parameters**.

We've defined the elements of the neural network that information flows through in the forward direction, but we haven't yet defined how it works out the error and then uses it to update the network's learnable parameters.

There are lots of ways in which we can define an error for the network, and PyTorch provides convenient functions for popular options. One of the simplest is the **mean squared error**, which calculates the square of the differences between the actual and desired outputs at each of the output nodes, before calculating the mean average. PyTorch provides this as **torch.nn.MSELoss()**.

We can choose this error function and give it a name inside the constructor.

```
# create loss function
self.loss_function = nn.MSELoss()
```

You'll sometimes see the words **error** function and **loss** function used interchangeably, and often this is fine. If we want to be precise, the **error** is the simple difference between a desired and actual output, and the **loss** is calculated from the error in ways that make sense given the problem we're trying to solve.

We need to use that **error**, or more correctly the **loss**, to update the network's link weights. Again, there are several ways to do this, and PyTorch provides functions for popular options. Let's start with the simple one we developed in *Make Your Own Neural Network*, called **stochastic gradient descent**, or **SGD**, with a learning rate of **0.01**.

```
# create optimiser, using simple stochastic gradient descent
self.optimiser = torch.optim.SGD(self.parameters(), lr=0.01)
```

It is interesting to see that we're passing all the learnable parameters to the SGD optimiser. These are accessible through **self.parameters()** which the machinery of PyTorch creates for us.

To pass information through the network, PyTorch assumes a **forward()** method. So we need to create one, but it can be very minimal.

```
def forward(self, inputs):
    # simply run model
    return self.model(inputs)
```

Here we simply take the given inputs and pass them to **self.model()** which we defined above using **nn.Sequential()**. The output of the model is simply returned to whoever calls **forward()**.

Let's pause and think about what we've created so far:

- We've created a neural network class that inherits from **nn.Module**. Inheriting from **nn.Module** provides much of the machinery needed to train a neural network.

- We've defined the elements of the neural network through which information flows. We've used the **nn.Sequential** method as it is concise for simple networks.

- We've defined the **loss** function and the **optimiser** method for updating the network's **learnable parameters**.

- Finally, we added a **forward()** function which PyTorch expects to pass information through the network.

Our neural network class should look like this:

```
# classifier class

class Classifier(nn.Module):

    def __init__(self):
        # initialise parent pytorch class
        super().__init__()

        # define neural network layers
        self.model = nn.Sequential(
            nn.Linear(784, 200),
            nn.Sigmoid(),
            nn.Linear(200, 10),
            nn.Sigmoid()
        )

        # create loss function
        self.loss_function = nn.MSELoss()

        # create optimiser, using simple stochastic gradient descent
        self.optimiser = torch.optim.SGD(self.parameters(), lr=0.01)

        pass

    def forward(self, inputs):
        # simply run model
        return self.model(inputs)
```

How do we train the network? Do we need a **train()** function like we have a **forward()** function? Actually, PyTorch doesn't require a **train()** method. It is left up to us to structure our code to do the training.

Let's keep our own code simple and consistent by creating a **train()** method to sit alongside the **forward()** method.

A **train()** method needs both the **inputs** to the network, as well as the desired **target** outputs so it can compare them with the actual output and calculate the **loss**.

```
def train(self, inputs, targets):
    # calculate the output of the network
    outputs = self.forward(inputs)

    # calculate loss
    loss = self.loss_function(outputs, targets)
```

The first thing the **train()** function does is pass the **inputs** through the network using the **forward()** method to obtain the **outputs**.

The loss function we defined earlier is used to calculate the **loss**. We can see how PyTorch makes this very easy. All we do is supply the function with the network's outputs and the desired targets.

The next step is to use the **loss** to update the network's link weights. You'll remember from *Make Your Own Neural Network* that we need to work out the error gradients at each node and use them to update the link weights connected to those nodes.

PyTorch makes this really easy.

```
# zero gradients, perform a backward pass, and update the weights
self.optimiser.zero_grad()
loss.backward()
self.optimiser.step()
```

Those three steps are the key pattern for almost all neural networks built with PyTorch. Let's talk about them step by step.

- First the gradients in the computation graph are all set to zero with **optimiser.zero_grad()**.

- The gradients inside the network are calculated working back from the loss function using **loss.backward()**.

- These gradients are used to update the network's learnable parameters using **optimiser.step()**.

The gradients need to be set to zero every time we train the network. If they aren't, the values accumulate with every calculation of the gradients with **loss.backward()**.

We've used the **backward()** function before to calculate gradients for a very simple network. Its use here is no different. We can think of the computation graph's final node as the loss function. The gradient calculated at each node that feeds into this loss is the change in the loss as each learnable parameter varies.

The optimiser takes the gradients and updates the learnable parameters by taking a **step** down the gradient.

We finally have a network that can be trained. Before we train it, let's add a way of visualising how well, or badly, training went.

Visualising Training

When we trained networks in *Make Your Own Neural Network*, we didn't really have a way of seeing the progress of that training. We did measure how well the network performed after training, but we didn't get a sense of how smoothly the training itself had gone, or a sense of whether further training would be helpful.

One way to keep track of training is to monitor the loss. We could do this by keeping a copy of the **loss** value in a list every time it is calculated inside **train()**. This would mean the list would get very big because neural network training is often run for thousands, if not millions, of examples. The MNIST dataset has **60,000** training examples, and we might run several epochs of these. A better way is to keep a copy of the **loss** value after every **10** training examples. This means we need to keep a count of how often **train()** has been run.

The following code creates a **counter**, initially set to **0**, and an empty list called **progress** in the constructor of the neural network class.

```
# counter and accumulator for progress
self.counter = 0
self.progress = []
```

Inside the **train()** function we can increment the **counter** and add the loss value to the end of the list after every **10** training examples.

```
# increase counter and accumulate error every 10
self.counter += 1
if (self.counter % 10 == 0):
    self.progress.append(loss.item())
    pass
```

The **% 10** means remainder after dividing by ten, and is only **0** when the counter is **10, 20, 30** and so on. The **item()** function used here is just a convenience function for unwrapping a single-value tensor to get at the number inside.

We can print out the **counter** after every **10000** so we can see how slow or fast the training is progressing through the dataset.

```
if (self.counter % 10000 == 0):
    print("counter = ", self.counter)
    pass
```

To picture the loss as a chart, we can add a new function **plot_progress()** to the neural network class.

```
def plot_progress(self):
    df = pandas.DataFrame(self.progress, columns=['loss'])
    df.plot(ylim=(0, 1.0), figsize=(16,8), alpha=0.1, marker='.',
grid=True, yticks=(0, 0.25, 0.5))
    pass
```

The code looks complicated but there are only two lines. The first one converts the list of loss values **progress** into a pandas dataframe, so that we can easily plot a chart of these values. The options given to the **plot()** function set the design and style of the plot.

We're almost ready to train our network!

MNIST Dataset Class

We've previously loaded the MNIST data from a CSV file into a pandas dataframe. We could use the data directly from the dataframe and that would be fine. However, since we're learning about PyTorch we should try to load and use data in the PyTorch way.

PyTorch can do useful things like automatically shuffle data, load it with multiple processes in parallel, and provide it in batches too. PyTorch does this using **torch.utils.data.DataLoader** which expects the data itself to come through a **torch.utils.data.Dataset** object.

To keep things simple, we won't be using shuffling or batching, but we will work with the **torch.utils.data.Dataset** class to gain some experience of working with PyTorch's machinery.

We import the PyTorch **torch.utils.data.Dataset** class as follows.

```
from torch.utils.data import Dataset
```

Just like we inherit a neural network class from **nn.Module** and provide a **forward()** function, for a dataset we inherit from **Dataset** and provide the following two special functions:

- **__len__()** which returns the number of items in the dataset.
- **__getitem__()** which returns the nth item in the dataset.

Creating a MnistDataset class and providing the **__len__()** method allows PyTorch to get the size of the dataset using **len(mnist_dataset)**. Providing a **__getitem__()** allows us to get items by index, using **mnist_dataset[3]** for example.

Let's have a look at the following MnistDataset class.

```
class MnistDataset(Dataset):

    def __init__(self, csv_file):
        self.data_df = pandas.read_csv(csv_file, header=None)
        pass

    def __len__(self):
        return len(self.data_df)

    def __getitem__(self, index):
        # image target (label)
        label = self.data_df.iloc[index,0]
        target = torch.zeros((10))
        target[label] = 1.0

        # image data, normalised from 0-255 to 0-1
        image_values =
torch.FloatTensor(self.data_df.iloc[index,1:].values) / 255.0

        # return label, image data tensor and target tensor
        return label, image_values, target

    pass
```

When an object is created from this class, the **csv_file** is read into a pandas dataframe named **data_df**.

The **__len__()** function simply returns the length of the dataframe. Easy!

The **__getitem__()** function is a little more interesting. Just like our earlier experiments with the MNIST data, we extract a **label** from the **index**-th item in the dataset.

We also create a tensor that represents the desired output of the neural network. A tensor of length **10** is filled with zeros but with one entry set to **1.0** which matches the label. So a **label** of **0** results in a tensor that looks like **[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]**, and a **label** of **4** results in a tensor that looks like **[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]**. This is called **one-hot encoding**.

We then create a tensor **image_values** out of the image pixel values. The numbers are divided by **255** so they range from **0** to **1**.

Finally, **__getitem__()** returns all three, the **label**, the **image_values** and the **target**.

Even though it isn't required by PyTorch, we can add a method to the MnistDataset class to plot the image for a selected record in the dataset. It is good practice to have a method to see the data we're working with. We'll need to import the **matplotlib.pyplot** library like we did before.

```
def plot_image(self, index):
    arr = self.data_df.iloc[index,1:].values.reshape(28,28)
    plt.title("label = " + str(self.data_df.iloc[index,0]))
    plt.imshow(arr, interpolation='none', cmap='Blues')
    pass
```

Let's check things are working so far. First we create a dataset object from the class, passing it the CSV file location

```
mnist_dataset = MnistDataset('mount/My Drive/Colab
Notebooks/myo_gan/mnist_data/mnist_train.csv')
```

We know the class constructor loads the data from the CSV file into a pandas dataframe. Let's use our **plot_image()** function to draw the tenth image in the dataset. The tenth image has an index of **9**, because the first image has an index of **0**.

```
mnist_dataset.plot_image(9)
```

You should see an image of a hand-written four. The label also tells us it is meant to be a four.

This confirms our dataset class did load the data correctly.

Check that our **mnist_dataset** allows access by index, using examples like **mnist_dataset[100]**. You should see it return the **label**, the pixel values and the **target** tensor.

Training Our Classifier

Training a classifier neural network is now really simple. That's because we've done all the hard work setting up the dataset class and the neural network class.

We first create a neural network from our Classifier class.

```
# create neural network
C = Classifier()
```

To train the network, the code is again very simple:

```
# train network on MNIST data set
for label, image_data_tensor, target_tensor in mnist_dataset:
    C.train(image_data_tensor, target_tensor)
    pass
```

The **mnist_dataset**, because it inherits from PyTorch's Dataset, allows us to work through all the training data with an elegant **for** loop. For each training example, we simply pass the image data and target tensor to the classifier's **train()** method.

We know from *Make Your Own Neural Network* that working through the entire dataset more than once to train our network can be helpful. We can easily add an outer **epoch** loop around the training loop.

Finally, python notebooks make it really easy to time how long a cell takes. We simply add a **%%time** at the top of the cell we want to time. This can be useful when experimenting with neural networks, and estimating how long we've got to make a coffee as the network trains!

Our cell should look like this:

```
%%time
# create neural network
C = Classifier()

# train network on MNIST data set
epochs = 3

for i in range(epochs):
    print('training epoch', i+1, "of", epochs)
    for label, image_data_tensor, target_tensor in mnist_dataset:
        C.train(image_data_tensor, target_tensor)
        pass
    pass
```

Running the cell takes some time. After every **10000** calls, the **train()** method will print an update on how many examples it has worked through.

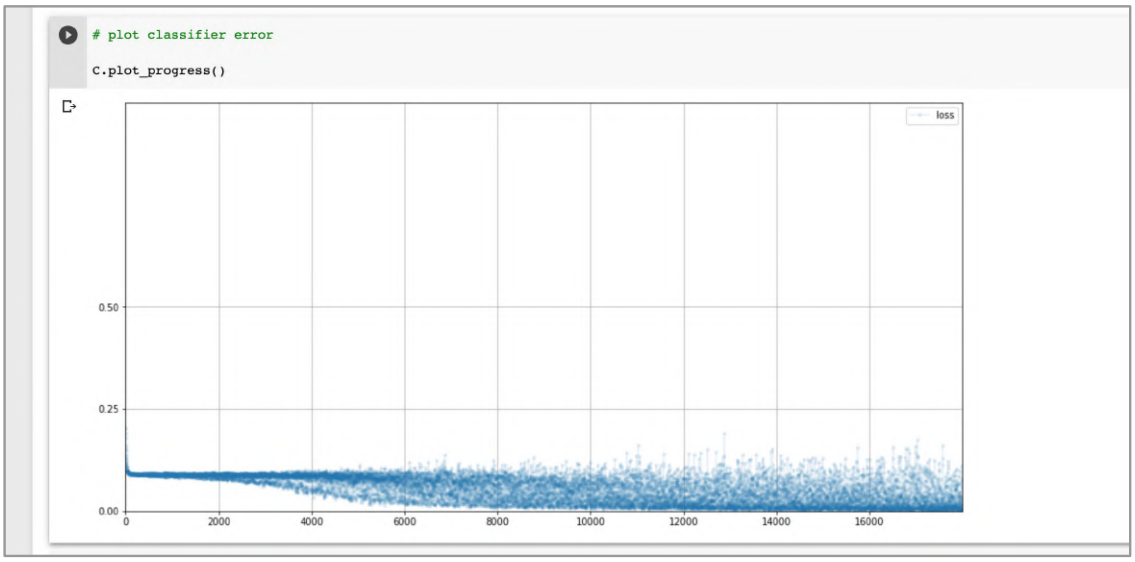
```
training epoch 1 of 3
counter =  10000
counter =  20000
counter =  30000
counter =  40000
counter =  50000
counter =  60000
training epoch 2 of 3
counter =  70000
counter =  80000
counter =  90000
counter =  100000
counter =  110000
counter =  120000
training epoch 3 of 3
counter =  130000
counter =  140000
counter =  150000
counter =  160000
counter =  170000
counter =  180000
CPU times: user 3min 42s, sys: 9.66 s, total: 3min 51s
Wall time: 3min 54s
```

We can see that **3** training epochs took just over **4** minutes. That's not bad at all if we remember that each epoch means **60,000** training examples.

Let's plot the collected loss values to get a view of how training progressed.

```
# plot classifier error
C.plot_progress()
```

You should see a chart similar to the following. It won't be exactly the same because training a neural network is inherently a random process.



We can see the **loss** values fall very quickly to about **0.1** and then over the course of training fall more slowly, and fairly noisily, towards zero.

A falling loss means the network is getting better at classifying the images correctly.

These loss charts are really useful. They allow us to see whether the network training is working at all. They also give us a sense of whether the training is smooth and steady, or unstable and chaotic.

Querying Our Neural Network

Now that we have a fairly well-trained network, let's ask it to classify images. We'll switch to the MNIST test dataset of **10,000** images. These are images our neural network has not yet seen.

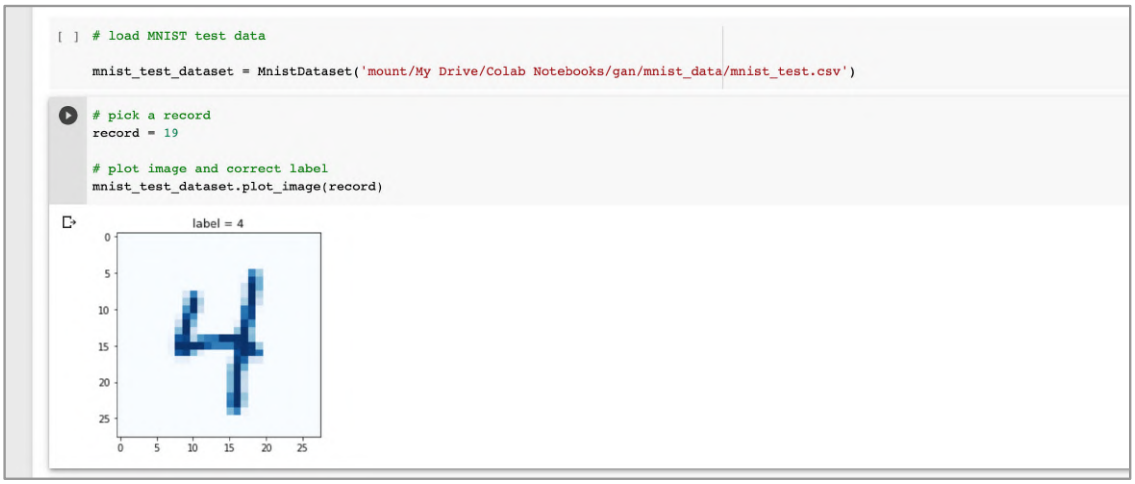
Let's load the dataset with a new Dataset object.

```
# load MNIST test data
mnist_test_dataset = MnistDataset('mount/My Drive/Colab
Notebooks/gan/mnist_data/mnist_test.csv')
```

We can pick a record from the test dataset and see what the image looks like. The following code picks the **20th** record, which has index **19**.

```
# pick a record
record = 19

# plot image and correct label
mnist_test_dataset.plot_image(record)
```

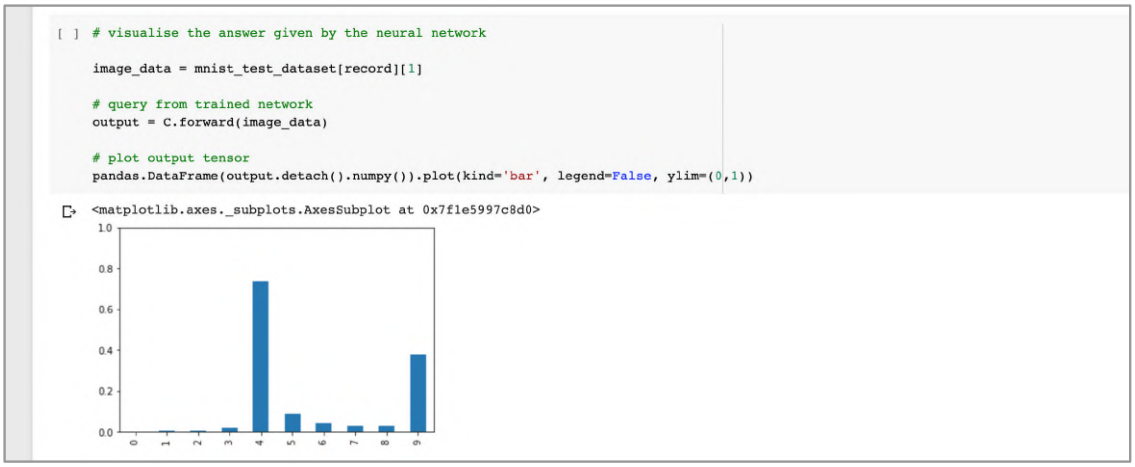
We can see the image looks like a **4**. The label, extracted from the record, also confirms it is meant to be a **4**.



Let's see what our trained neural network thinks of this image. The following code uses **record** set to **19** above to extract the image pixel values as **image_data**. The image is passed through the neural network using the **forward()** function.

```
image_data = mnist_test_dataset[record][1]

# query from trained network
output = C.forward(image_data)

# plot output tensor
pandas.DataFrame(output.detach().numpy()).plot(kind='bar',
legend=False, ylim=(0,1))
```

The **output** is converted to a simpler numpy array before being wrapped into a dataframe for easy plotting as a bar chart.



Those **10** bars are the values of the **10** neural network output nodes. The largest value is for node **4**, which tells us that the network thinks the image is a four.

Great! The network correctly classified the test image.

If you look more closely you'll see that all the other nodes don't have a zero output. We don't expect classifying neural networks to produce such clear-cut answers. In fact, this one seems to think the image could also be a **9**, but not as strongly as a **4**. Looking back at the actual image we can see how the difference between a **9** and a **4** isn't always clear.

Pick out other records to try yourself. Record **42** is a good example of an ambiguous image. Also, see if you can find an image that the network gets wrong. My trained network gets record **33** wrong, and looking at the image we can see it is a particularly badly written digit.

Simple Classifier Performance

A simple way to see how well our neural network correctly classifies images is to work through all the **10,000** images in the MNIST test dataset and keep a count of how many it got right. We do this by comparing the output of the network with the label that came with the image.

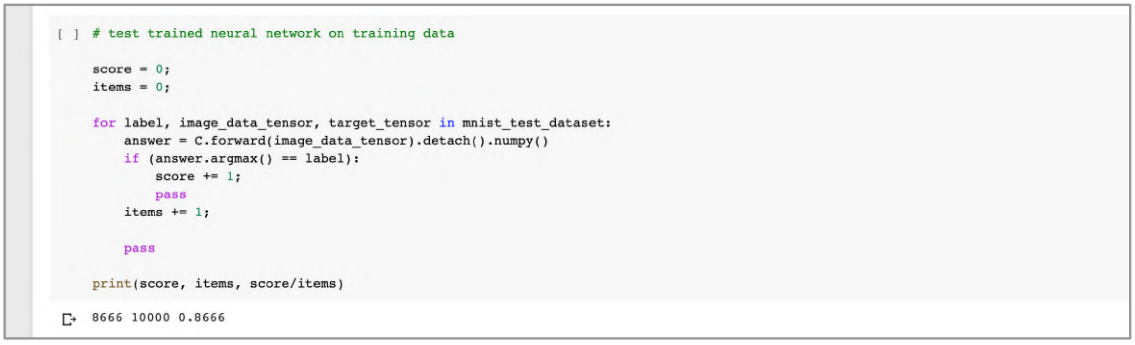
The following code sets a **score** count to **0** and then works through the test data, incrementing the **score** every time the label matches the network's output.

```
# test trained neural network on training data

score = 0
items = 0

for label, image_data_tensor, target_tensor in mnist_test_dataset:
```

```
    answer = C.forward(image_data_tensor).detach().numpy()
    if (answer.argmax() == label):
        score += 1
        pass
    items += 1

    pass

print(score, items, score/items)
```

The **answer.argmax()** code finds the index of the tensor **answer** which has the largest value. If the first value was the largest the argmax would be **0**. It's a neat way of asking "which node has the largest value?".

We finally print the score, a fraction which tells us how many answers the neural network got right.



```
[ ] # test trained neural network on training data

    score = 0;
    items = 0;

    for label, image_data_tensor, target_tensor in mnist_test_dataset:
        answer = C.forward(image_data_tensor).detach().numpy()
        if (answer.argmax() == label):
            score += 1;
            pass
        items += 1;

        pass

    print(score, items, score/items)

    8666 10000 0.8666
```

I got a score of **87%** which is not bad given the simplicity of our neural network.

See if you can improve the score by training the network for more than **3** epochs. What happens to the score if you train for less than **3** epochs?

You can explore the code we've developed for this simple MNIST classifier online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/03_mnist_classifier.ipynb

Refinements

In the last section we developed a neural network to classify images of hand-written digits. Even though we intentionally designed the network to be simple, it worked remarkably well, getting an accuracy score of about **87%** with the MNIST test dataset.

Here we'll explore some refinements which can help us improve a network's performance.

Loss Function

Some neural networks are designed to produce a continuous range of output values. For example, we might want a network predicting temperatures to be able to output any value in the range **0** to **100** degrees centigrade.

Some networks are designed to produce true/false or **1/0** outputs. For example, we would want a network which decides whether an image is a cat or not, to output values close to **0.0** or **1.0**, and not all of the values in between.

If we did the maths to work out loss functions for each of these different scenarios, we'd find that the Mean Squared Error loss makes sense for the first kind of task. You may know that this is a **regression** task, but don't worry if you don't.

For the second task, a **classification** task, a different kind of loss function makes more sense. A popular one is called the Binary Cross Entropy loss, and it tries to penalise wrong but confident outputs as well as correct but not so confident outputs. PyTorch provides this as **nn.BCELoss()**.

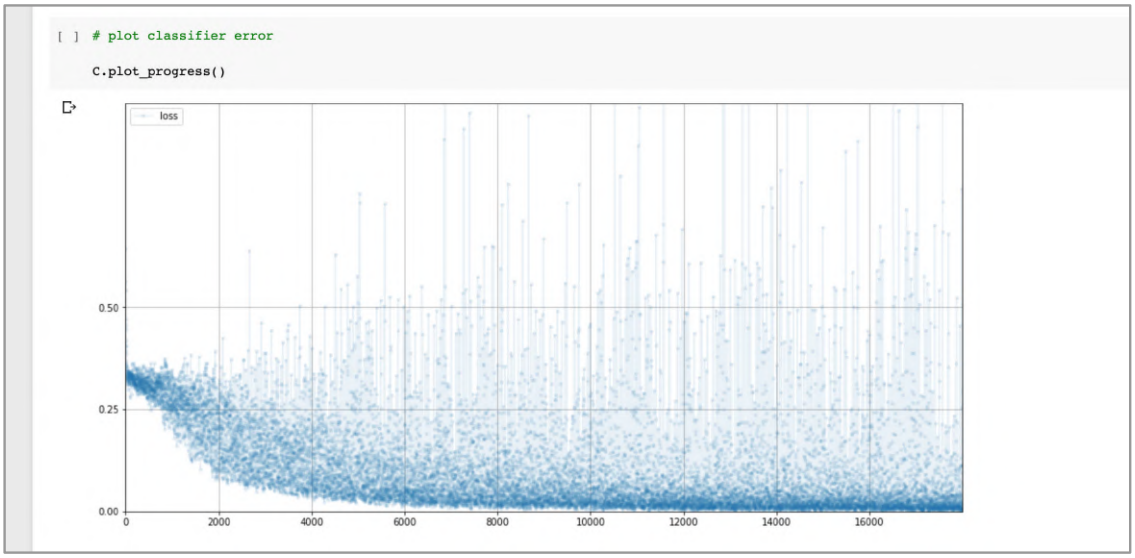
Our network, which classifies MNIST images, is of the second type. The output node values should ideally be confidently close to **0.0**, with just one being confidently close to **1.0**.

Make a copy of the previous notebook and change the loss function from **MSELoss()** to **BCELoss()**.

```
self.loss_function = nn.BCELoss()
```

Let's train the network again with **3** epochs, just like before. The performance score against test data is now **91%**. That's a good leap from **87%**.

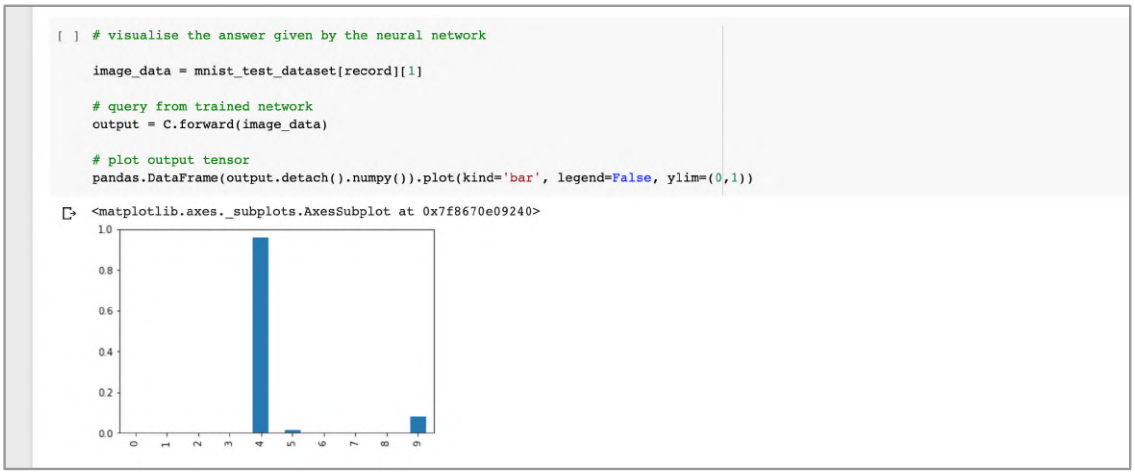
It's interesting to look at the loss chart.



We can see the loss does fall, but starts to fall more slowly than with **MSELoss()**. The loss is also much noisier, even late in training there are occasionally large values.

Even though the loss chart might look worse than the previous one, towards the end of training the bulk of the loss values are lower, and the better performance score reflects this.

The following shows the output values from record **19** of the test dataset again.

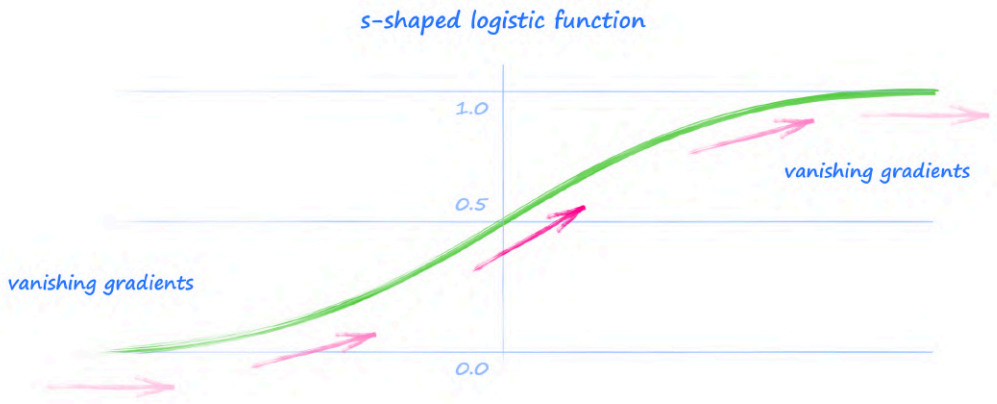


We can see the network is much more confident in believing the image is a **4**. Its confidence in the image being a **9** is much lower now.

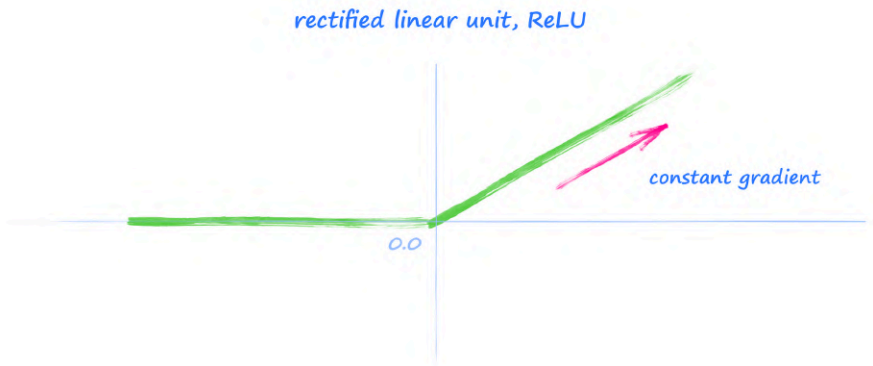
Activation Function

The s-shaped logistic function was used in the early days of neural networks because its shape seemed to match what we thought was happening in animals as a threshold for signals passing between neurons, and also because it is mathematically convenient for calculating gradients.

However, it does have some weaknesses. The main one is that for large values, the gradients get very small, and can effectively disappear. This means that training neural networks, which depend on gradients to correct the link weights, can start to fail when this kind of **saturation** happens.

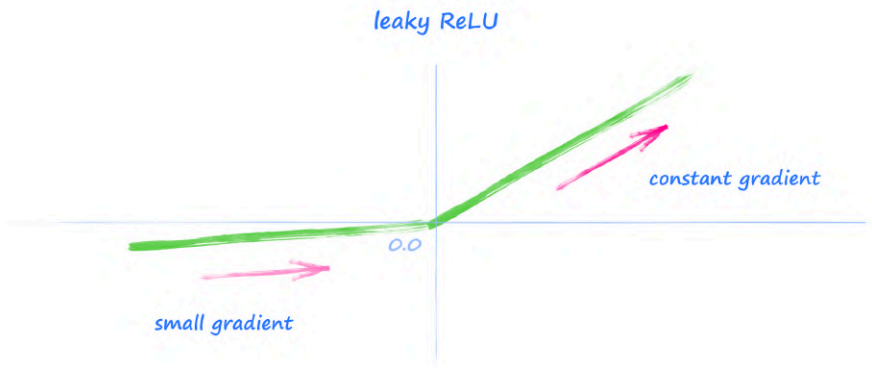


There are many alternative activation functions, and a simple one that solves this problem is just a straight line. The fixed gradient of a straight line doesn't disappear.



This activation function is called a **rectified linear unit**, and PyTorch provides a **ReLU()** function.

Actually, the slope is zero for values less than zero, and so this does suffer the disappearing gradients problem for signals less than zero. An easy change is to have a small gradient for the left hand half of the function. This is called a **Leaky ReLU**.

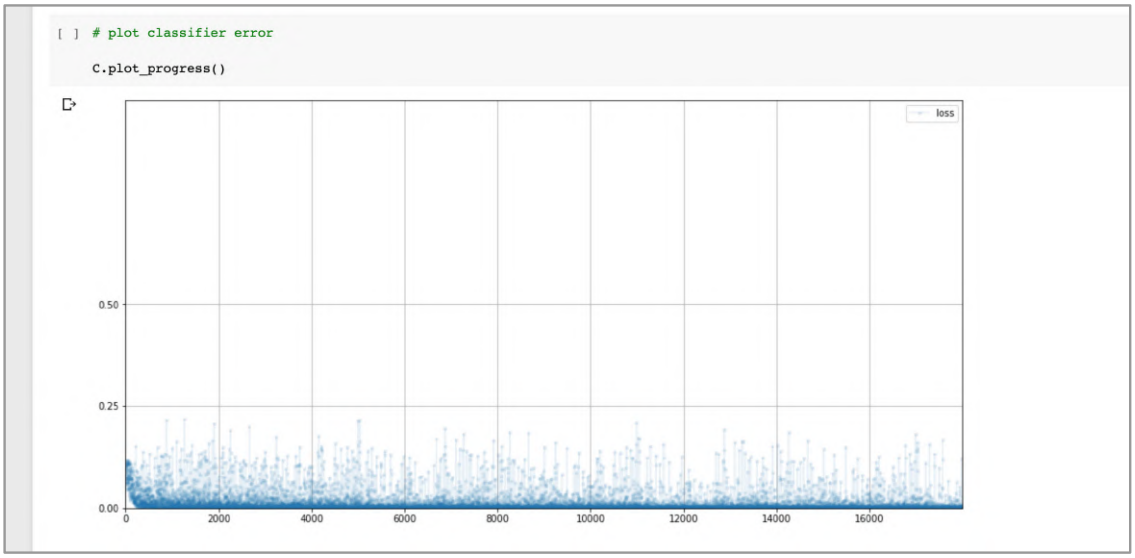


Let's reset the loss back to **MSELoss()** and change the activation functions to **LeakyReLU(0.02)** where **0.02** is the gradient of the left half of the function.

```
# define neural network layers
self.model = nn.Sequential(
    nn.Linear(784, 200),
    nn.LeakyReLU(0.02),
    nn.Linear(200, 10),
    nn.LeakyReLU(0.02)
)
```

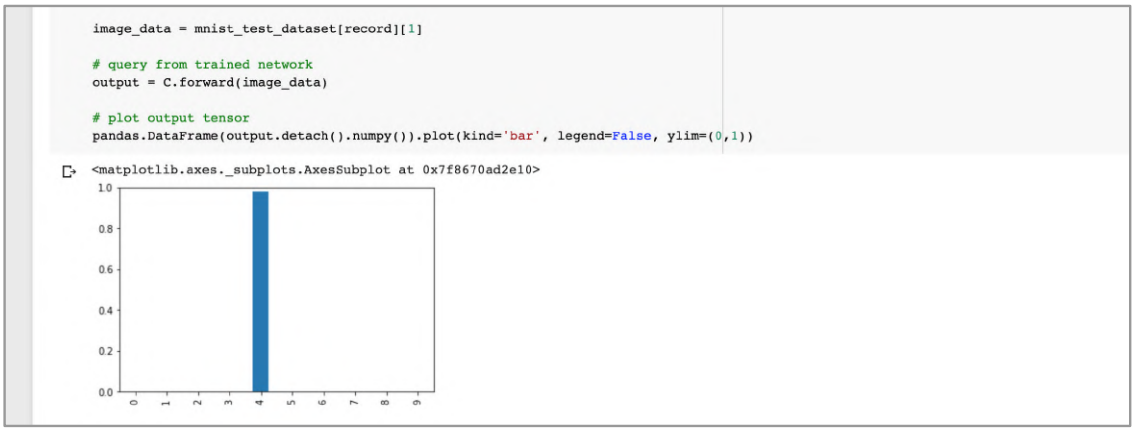
Train the network again with **3** epochs. For me, the performance score against test data is now **97%**. That's a huge leap from **87%** and approaching industry leading records which use much more complicated networks.

Let's look at the loss chart.



We can see the loss falls very rapidly towards zero, and even early in training the average loss is very low, with much lower noise spread.

The output values for record **19** of the test dataset show the network is very confident in the answer **4**. The values for all other nodes are very close to zero.



Changing the activation function has a significant effect, and this is simply due to the better gradients.

Optimisation Method

We can also refine the technique that uses the back propagated gradients to update the network weights.

Previously we used a fairly simple method, stochastic gradient descent, which we also developed from first principles in *Make Your Own Neural Network*. The method is popular because it is simple, and fairly lightweight in terms of computer resources.

A weakness with the simple stochastic gradient descent is that it can get stuck in local minima in the loss function. Another weakness is that a single learning rate applies to all learnable parameters.

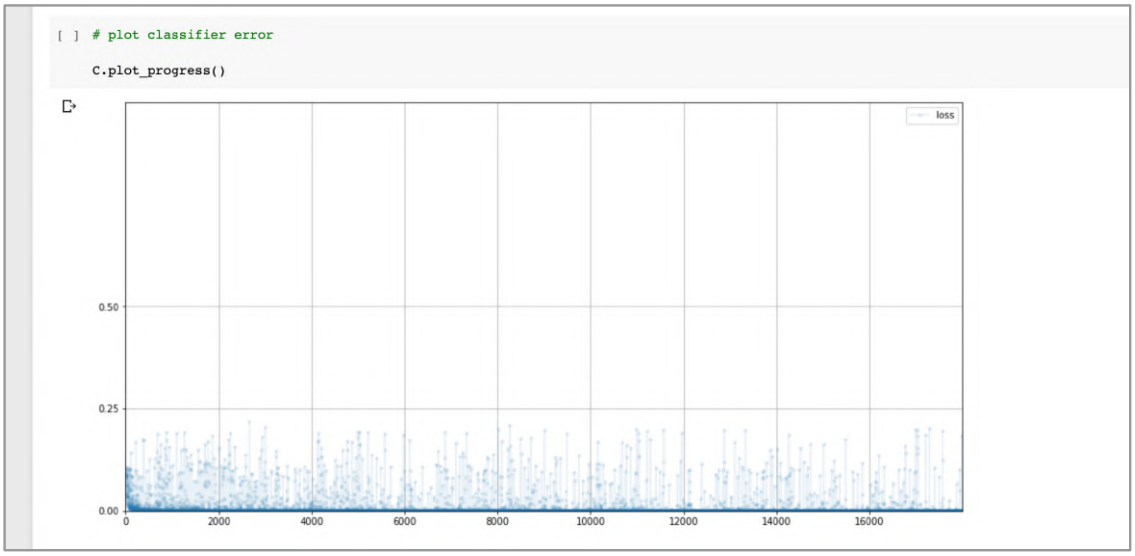
There are several popular alternatives, and a notable one is called **Adam**. It addresses the two challenges directly. It uses the idea of a momentum to reduce the chances of getting stuck in a local minimum, a bit like how a heavy ball can roll through a small pothole because its momentum carries it through. It also uses a separate learning rate for each learnable parameter, which can adapt to how each parameter changes during training.

Let's reset the code back to using **MSELoss()** and sigmoid activation functions, and change the optimiser from SGD to Adam.

```
self.optimiser = torch.optim.Adam(self.parameters())
```

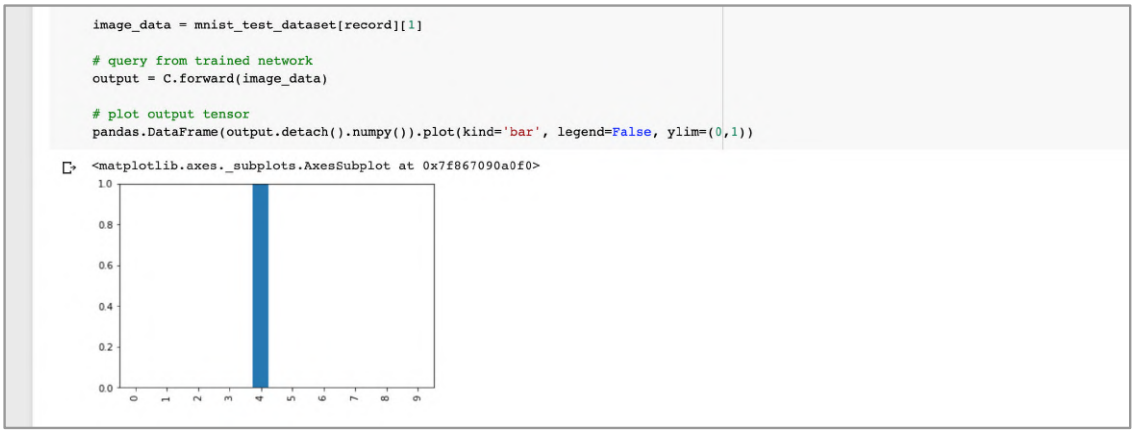
Train the network again with **3** epochs. For me, the performance score against test data is about **97%** again. That's still a big improvement from **87%** and very similar to the improvement given by the LeakyRELU activation function.

Let's examine the loss chart.



The loss falls towards zero really quickly and the average remains very low too. It looks like the Adam optimiser is pretty effective.

The output for record **19** of the training data shows the network thinks the image is a **4** to a very high degree of confidence.



Although not perfect, Adam is considered a good choice for many tasks.

Normalisation

The weights in a neural network, and the signals that pass through a network, can have potentially large values. We've already seen how this can lead to saturation, which can make learning harder.

A lot of research has been done on the benefits of reducing the range of parameter and signal values in a neural network, and also shifting the values so the mean is zero. This is called **normalisation**.

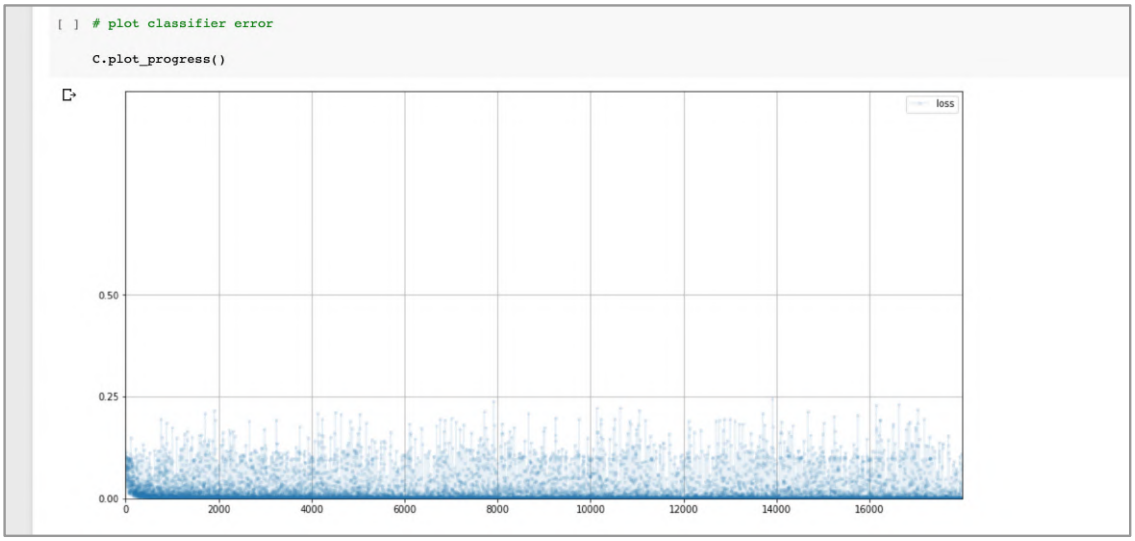
One simple application of this idea is to ensure that signals that pass into a neural network layer are already normalised.

Let's revert the code back to using MSELoss, sigmoid activation functions, and the SGD optimiser, and then use **LayerNorm(200)** to normalise the network signals just before they enter the final layer.

```
# define neural network layers
self.model = nn.Sequential(
    nn.Linear(784, 200),
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 10),
    nn.LeakyReLU(0.02)
)
```

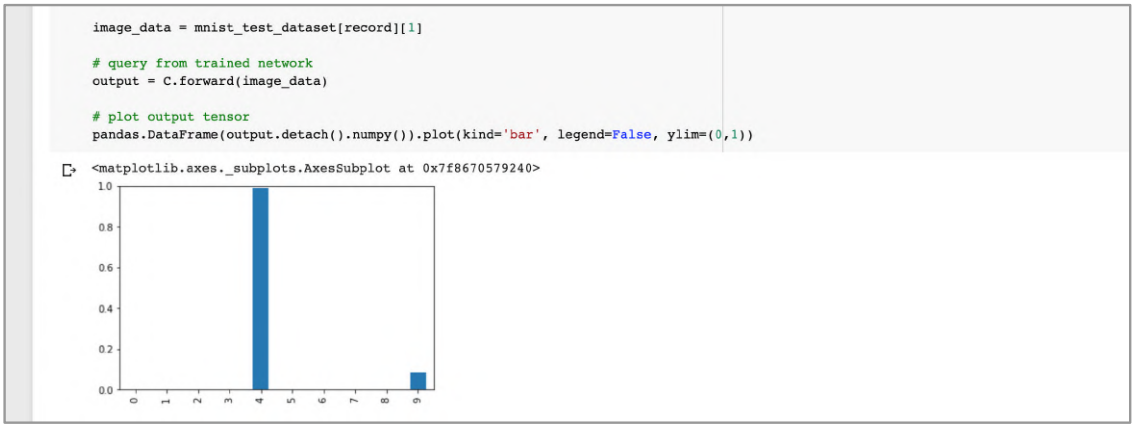
After training the network again with **3** epochs, the performance score against test data is **91%**. This is a good improvement over the vanilla network's **87%**.

Let's look at the loss chart.



We can see the loss falls more rapidly than the vanilla network. The loss is also less noisy if you consider its density on the chart rather than just the heights.

We can also see that the network's outputs are much cleaner, with a very confident value for **4** but also a smaller and understandable **9** too. Unlike the vanilla network, the other values are effectively zero.



It's worth noting that even in 2020, the exact reasons for how and why normalisation helps neural networks to train are not fully settled, with research papers still appearing offering a new angle. We're working with new techniques - and it's exciting!

Combined Refinements!

Let's combine all these refinements, BCE loss, leaky ReLU activation, Adam optimiser, and layer normalisation.

Because the BCE loss can't accept values outside the range **0** to **1**, which can emerge from a leaky ReLU, we can have a sigmoid after the last layer, but keep a LeakyRELU after the hidden layer.

```
# define neural network layers
self.model = nn.Sequential(
    nn.Linear(784, 200),
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 10),
    nn.Sigmoid()
)
```

Training the network again with **3** epochs gives us a performance score of about **97%**.

It seems that combining our refinements doesn't get us above about **97%** accuracy. To get world class scores we'll need more complex network architectures, which we won't do here.

You can explore the code we've developed online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/04_mnist_classifier_refinements.ipynb

Learning Points

- When working with new data, or building a new workflow, it is good practice to **preview** the data to ensure it was loaded and transformed properly.
- PyTorch can do a lot of the work in a machine learning task for us. To take advantage of this, we need to work with PyTorch's machinery. For example, neural networks should be derived from PyTorch's **nn.Module** class.
- It is good practice to **visualise loss** values so we can see how well training went.
- The **mean squared error** loss is suited to regression tasks where the output can be in a continuous range. The **binary cross entropy** loss is better suited to classification tasks where the output should be **1** or **0,** for true and false.
- The traditional sigmoid activation function suffers from **vanishing gradients** for large input values. This leads to poor feedback signals for training a network. The **ReLU** activation function solves this by having good gradients for positive inputs. The **LeakyReLU** improves this by also having a small non-vanishing gradient for negative values.
- The **Adam** optimiser uses momentum to overcome local minima, and maintains individual learning rates for each learnable parameter. For many tasks it performs better than the simple SGD optimiser.
- **Normalisation** can stabilise the training of neural networks. It is common to normalise a network's initial weights. Normalising signal values as they pass through a neural network with **LayerNorm** can improve performance.

CUDA Basics

In *Make Your Own Neural Network* we worked out the calculations that needed to be done for passing the signal forward through a neural network, and also for the backwards pass that updates the network link weights.

We found that both of these calculations could be written using matrix multiplication. Instead of writing out hundreds, or even thousands, of separate calculations, we could just write down a simple matrix multiplication. We were excited by this because libraries like numpy are designed to do matrix multiplication really quickly and efficiently. Using numpy to multiply two matrices is far more efficient than us writing python code to work through all the different combinations of calculations required.

Numpy Vs Python

Using numpy to multiply matrices avoids having to juggle matrix values up and down the layers of software underneath python. Numpy can work more directly on the memory that stores the matrix values. Numpy will also try to take advantage of special features of the CPU, for example parallelising calculations, instead of doing them strictly one after another.

Let's see if we can get a sense of how much more efficient matrix multiplication is with numpy than python.

Start a new notebook and import **torch** and **numpy**. In a new cell enter the following code.

```
# size of square matrix
size = 600

a = numpy.random.rand(size, size)
b = numpy.random.rand(size, size)
```

The code simply creates two numpy arrays filled with random values between **0** and **1**. The width and height of the arrays is **size**, here set to **600**.

In a separate new cell enter the following code.

```
%%timeit

x = numpy.dot(a,b)
```

The **%%timeit** is a special instruction that times how long code in a cell runs. The code in this cell simply uses numpy to matrix multiply the two arrays.

Run the cell to see how long the code takes.

The **%%timeit** instruction runs the code many times and picks the best in an attempt to minimise the effect of the computer also doing other things aside from running our code. The best time is reported as **12.7** milliseconds. That's pretty fast given how big the arrays are!

Let's now write code that intentionally doesn't use numpy to do the matrix multiplication, but instead uses only python to pick out all the combinations of values from the two matrices to build the answer matrix. We won't talk about the code in detail, the important point is that it avoids using numpy to do the multiplication.

We can see the code takes much longer at **3** minutes and **11** seconds, or **191** seconds.

The numpy version is about **1500** times faster!

It is worth noting that these timing tests are simple and easy, but not very scientific. If we really wanted to compare performance, we'd have to run many more tests in a much more controlled set up. For our purposes, a rough comparison is all we need. And we can see straight away that numpy matrix multiplication is much faster than with pure python.

Many real world neural networks are much larger and process much more data than we've been experimenting with. In those cases, the training times can be very long, taking hours, days or weeks, even when using libraries like numpy to do fast matrix multiplication.

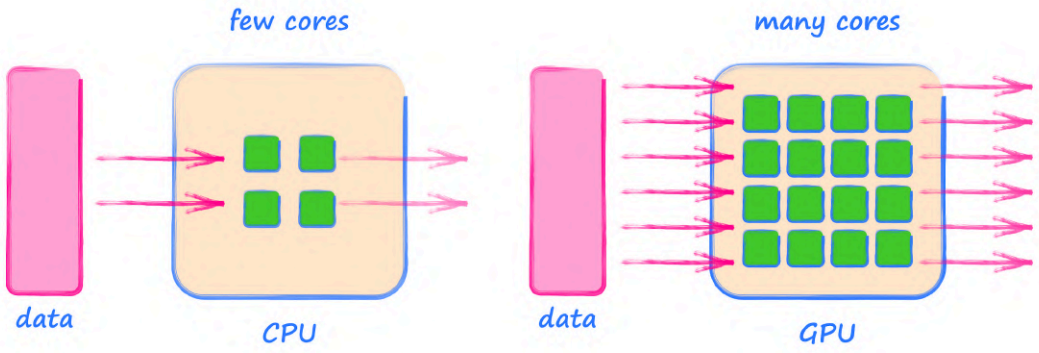
In the search for more speed, machine learning researchers started taking advantage of special hardware found in some computers, originally designed to improve graphics performance. You may have heard these called graphics cards.

NVIDIA CUDA

All computers have a **CPU**, the main component that does most of the work. It runs our python code, for example. We can think of it as the brains of our computer. CPU stands for central processing unit, but nobody really thinks about those words anymore. CPUs are designed to be general purpose, good at many different kinds of tasks.

Those graphics cards contain a **GPU**, or graphics processing unit. Unlike a general purpose CPU, a GPU is designed to perform specific tasks, and do them well. One of those tasks is to carry out arithmetic, including matrix multiplication, in a highly parallel way.

The following picture illustrates this key difference between a CPU and a GPU.



If we have lots of calculations to do, a CPU will work through them one after another. Modern CPUs might be able to use **2** or **4**, or maybe even **8** or **16** cores to work through the calculations. Only very recently have consumer CPUs contained **64** cores.

GPUs have many more arithmetic cores, thousands are fairly common today. This means a huge workload can be split amongst all these cores and the job can be done quickly.

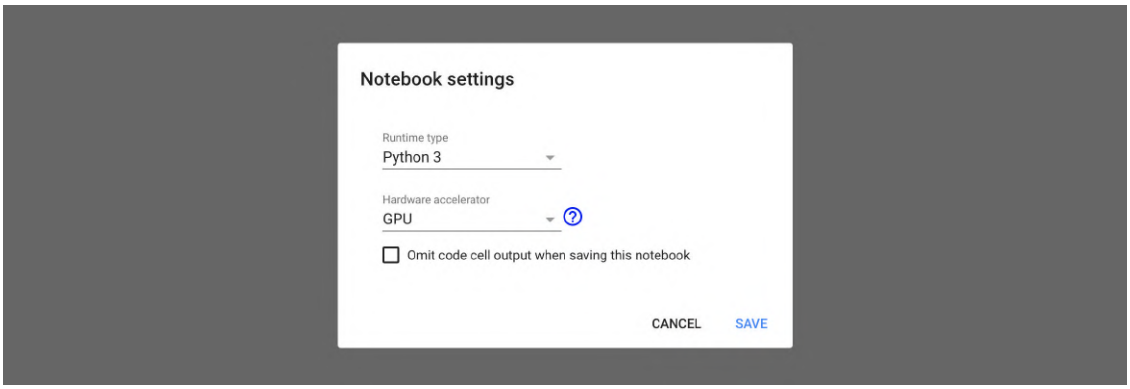
NVIDIA has been a leader in the GPU market, and are pretty much the standard for machine learning research because their software tools for taking advantage of hardware acceleration are fairly mature. That software framework is called **CUDA**.

The downside of CUDA is that it only works with GPUs from NVIDIA. You don't have a choice except to buy NVIDIA GPUs. NVIDIA's competitor AMD is only just starting to develop a comparable framework for its GPUs. In the future, a cross-platform standard might emerge and become well adopted, but for now we'll need to use NVIDIA and CUDA.

Google's Colab service gives us free access to GPUs to accelerate our calculations. Their GPUs are amongst the most powerful available. The catch is that Google will prevent you from using a GPU continuously for more than about **12** hours.

Using CUDA in Python

To use Google's GPUs from our python Notebook's we need to change a setting. From the menu of options at the top of our notebook choose **Runtime** and then **Change runtype type**.



Keep the runtime type as **Python 3**, and change the hardware accelerator from **None** to **GPU**. This will restart the virtual machine inside Google's platform, and attach a GPU.

Probably the simplest thing we can do is create a tensor that lives on the GPU. Up to this point, our tensors were stored in normal computer memory and accessed by the CPU.

At the beginning of this guide we created a tensor like this.

```
x = torch.tensor(3.5)
print(x)
```

That simple code didn't specify which type of number to use for the tensor, so the default type **float32** was used. You may know that numpy has many options for data type, such as **int32** for integer, **float32** for floating point numbers, and even **float64** for higher precision.

If we wanted to be more specific and not rely on default types, we would write the following.

```
x = torch.FloatTensor([3.5])
```

To check the type we simply use **x.type()**. We can see the type is **torch.FloatTensor**.

To create a tensor on the GPU we simply change the type to **torch.cuda.FloatTensor**.

```
x = torch.cuda.FloatTensor([3.5])
```

We can see the type has changed.

Using **x.device** we can double check which device the tensor is on.

The device is a CUDA device. Great!

Working with these tensors that live on the GPU is really simple, and in many cases no different to how we normally use PyTorch.

```
# calculation with tensor on GPU
y = x * x
y
```

This calculation is not done by the CPU. It is done by the GPU. We can also check that the new tensor lives on the GPU.

This might seem like a small achievement, but it is actually very powerful. Taking advantage of GPU acceleration was really difficult in the past, and this trivial simplicity is a huge achievement of modern software.

That calculation we just did won't benefit from the power of a GPU. In fact it is very likely to be slower than doing it on the CPU. To benefit from a GPU, we need lots of data to split into parts to feed the many GPU cores.

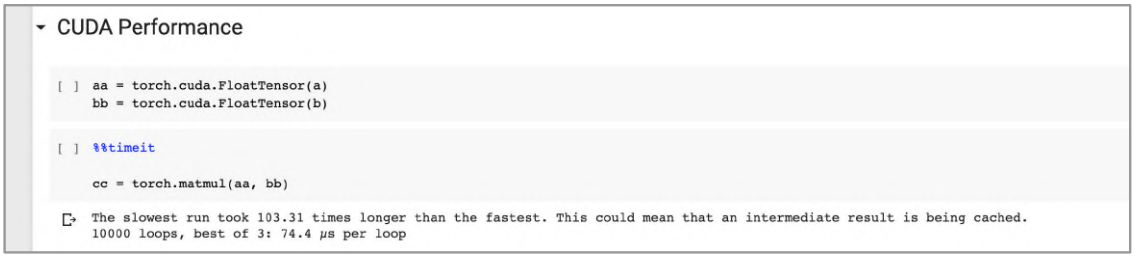
Let's create CUDA tensors from the **600** by **600** matrices we used earlier and do a matrix multiplication on the GPU.

```
aa = torch.cuda.FloatTensor(a)
bb = torch.cuda.FloatTensor(b)
```

The PyTorch function for doing the equivalent matrix multiplication is **matmul()**.

```
cc = torch.matmul(aa, bb)
```

Let's time it in a separate cell.

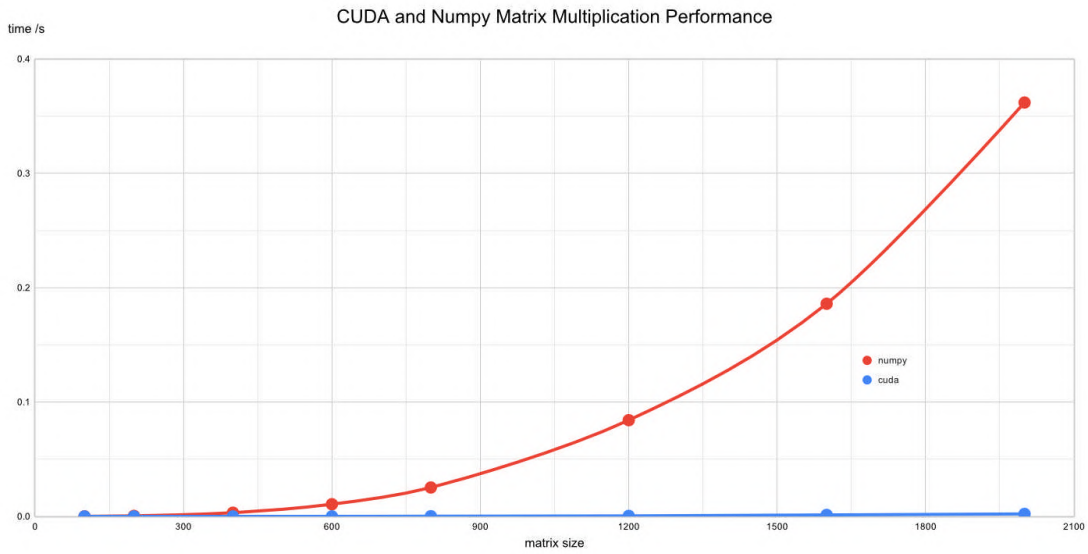


This calculation on the GPU takes **74.4** microseconds. Not milliseconds, but microseconds. That's really fast!

Although not scientifically rigorous, this simple test does show us that matrix multiplication with CUDA is about **150** times faster than with numpy.

If we tried these experiments with different matrix sizes, we would have a better comparison of CPU and GPU performance.

The following shows how long it takes numpy and CUDA to multiply matrices of sizes from **100** to **2000**. We can see that numpy slows down as the matrices get larger. The CUDA calculations do slow down too, but at this scale the GPU hardly breaks a sweat!



This chart really shows the benefit of GPUs to crunch through lots of data where it can be broken down into chunks and the calculations done in parallel.

For small amounts of data, you might be surprised to find a GPU can be slower. This is because GPUs aren't always faster than CPUs at individual calculations, and there is also a cost to moving data between the CPU and GPU. GPUs are only of benefit if we can take advantage of the many compute cores, and that means having enough data that can be chopped up and worked on in parallel.

Before we finish, let's look at some code which has become the standard way to check whether CUDA is available at the beginning of a python notebook.

```
if torch.cuda.is_available():
  torch.set_default_tensor_type(torch.cuda.FloatTensor)
  print("using cuda:", torch.cuda.get_device_name(0))
  pass

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

device
```

The check is simply **torch.cuda.is_available()**. If it is available, we set the default tensor type to be **torch.cuda.FloatTensor.** We also print the device name that CUDA found.

This standard code also sets up a PyTorch **device** which can be used in our code to more easily move data to the GPU if it doesn't by default. If CUDA isn't available, the **device** points to the CPU.

Let's try the code to see which CUDA device it finds.

```
[ ] # check if CUDA is available
    # if yes, set default tensor type to cuda

    if torch.cuda.is_available():
      torch.set_default_tensor_type(torch.cuda.FloatTensor)
      print("using cuda:", torch.cuda.get_device_name(0))
      pass

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    device

    using cuda: Tesla P100-PCIE-16GB
    device(type='cuda')
```

We can see the CUDA device is a **Tesla P100**, a very powerful and expensive bit of hardware. The GPU that's attached to your virtual machine can be different depending on location and availability, but all of them are pretty powerful. I often get a **Tesla T4** too, another powerful GPU.

The code we've been working with is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/05_cuda_basics.ipynb

Learning Points

- **GPU**s contain many compute cores for performing certain calculations in a highly parallel way. Originally designed to accelerate computer graphics, they are increasingly used to accelerate machine learning calculations.
- **CUDA** is NVIDIA's programming framework for GPU accelerated computation. PyTorch makes it very easy to take advantage of CUDA without significantly changing coding style.
- On synthetic benchmarks, such as matrix multiplication, we can see a GPU outperform a CPU by a factor of over **150**.
- GPUs can be slower than CPUs at individual calculations. There is also a cost to transferring data between the CPU and GPU. If the data isn't wide enough to distribute over many cores, there may be little or no benefit to using a GPU.

Part 2

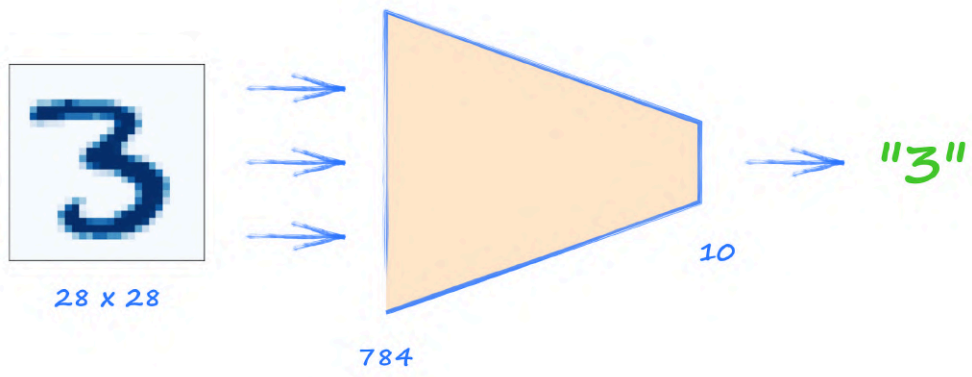
We introduce the idea of adversarial training and build progressively more sophisticated GANs; first to learn a simple 1010 pattern, then monochrome images of handwritten digits, and finally full colour images of faces.

The GAN Idea

Before we jump into exploring GANs, let's set the scene a little.

Generating Images

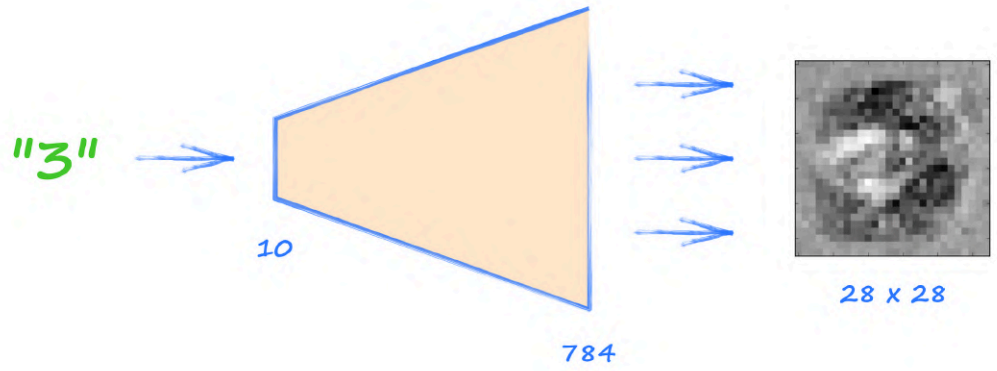
Normally we use neural networks to reduce, distill, summarise information. The MNIST classifier is a good example of this. It takes **784** values as input, and outputs **10** values, a much smaller number.



Let's do a thought experiment. If we flipped these networks back to front, we would be doing the opposite of reducing, we would be expanding smaller data into bigger data. In this case, we would be generating images.

Imagine that - networks that can create data!

That's not such a crazy idea. We did do this in *Make Your Own Neural Network*. We passed a one-hot target vector representing a digit backwards through an already trained network to generate some kind of ideal image of that digit. We called the process **backquery**.



We found the images created by this backquery were

- always the same given the same one-hot vector.
- a kind of average of the images in the training data with that label.

It's great that we can use a network to generate images, but ideally they

- would create different images.
- would look like an example of the training data, not a smoothed average of it.

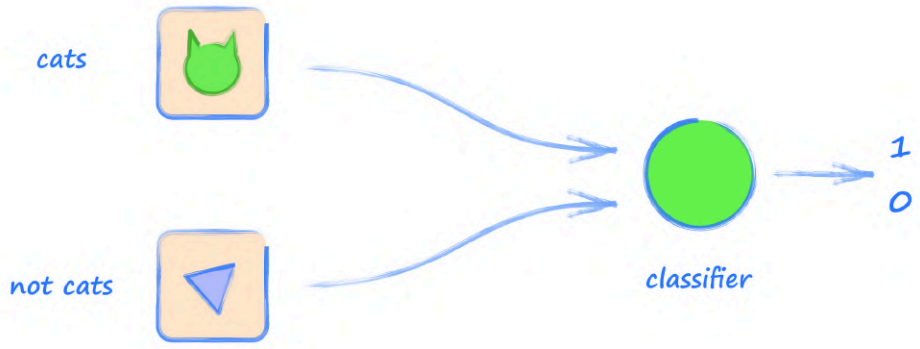
These two challenges are important for realistic and useful image generation. Our simple backquery method doesn't solve these challenges. We need a different kind of network architecture.

Adversarial Training

In 2014 Ian Goodfellow presented a different kind of network architecture. It wasn't a bigger, wider or deeper version of other neural networks. It didn't use fancier activation functions or more advanced optimisation techniques. It was structurally different.

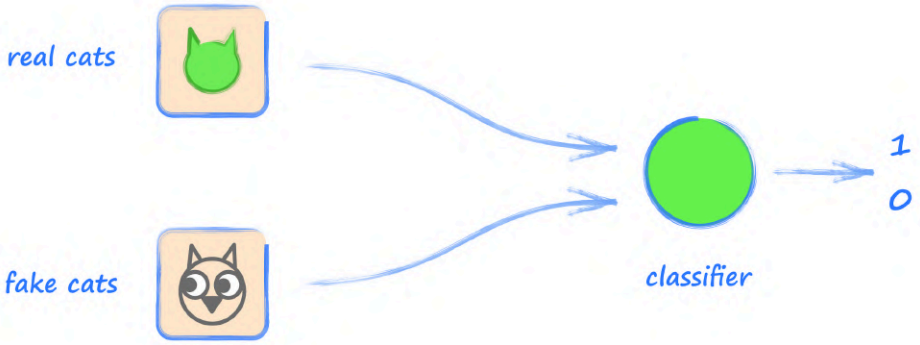
Let's approach this idea small step by small step.

The following picture shows a neural network which learns to classify images as cats or not cats.



If the image fed to the network is a cat, the output value should be **1**, for true. If the image isn't a cat, the output should be **0**, for false. This architecture is not too different from our MNIST example. The only difference is that this classifier outputs a single value, not ten.

Let's make a small change to the task. Instead of the classifier trying to separate cats and not cats, we could have a classifier trying to separate photos of real cats from cartoon images of cats that I drew myself.

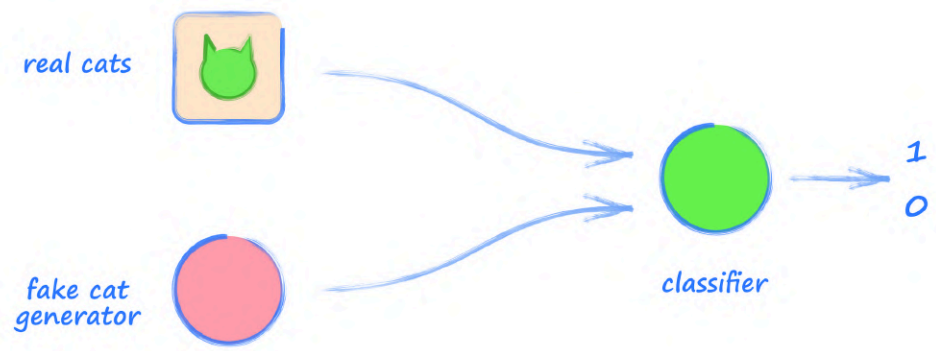


Nothing significant has really changed with the architecture. We still have two kinds of images, and the neural network classifier can be trained to separate the two kinds.

We can think of the classifier as a **detective**. Before any training, the detective will not be very good at telling real cats from fake cats. As training progresses the detective will get better at spotting the fakes and separating them from the real cats.

Simple enough so far. Let's take the next step.

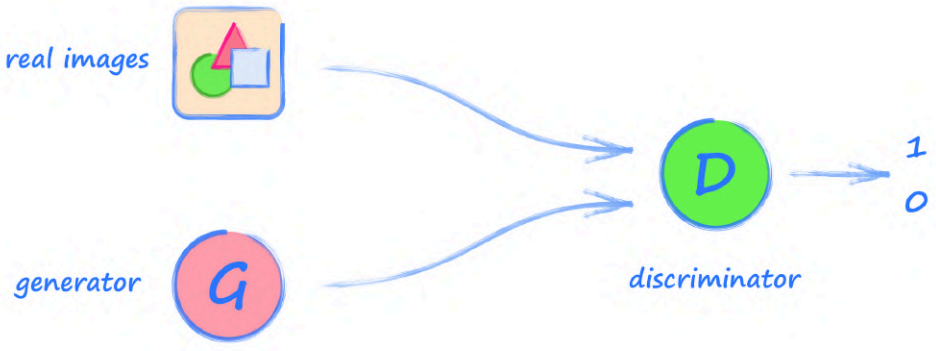
Instead of having a pile of fake images, imagine we had an element which generated fake images.



This saves us having to prepare a dataset of fake images. We just generate them using code. It wouldn't be difficult to generate truly rubbish images that looked nothing like cats. We could simply draw random triangles, for example. The job of the detective classifier would not be difficult at all.

The next step is the important one.

Instead of settling for a poor image generator, imagine we had a neural network that could be trained to generate images. Let's call it a **generator**. Let's also call the classifier a **discriminator**, which has become the accepted name in this kind of architecture.



Let's think about how we might train that **generator**. Training is about which behaviour we reward and which we punish. That's what a loss function does.

- Let's reward the **generator** when it gets an image past the **discriminator**.
- Let's punish the **generator** when it gets caught as an image forger.

Trying not to think too much about the loss function, let's take a step back and look at the whole picture.

The discriminator's job is to separate the real images from the generated ones. If the generator is not very good, this job will be easy. But if we train the generator, then it should get better and better at generating images that look more and more like real images.

That's pretty cool. But there's more.

As the discriminator gets better and better with training, the generator must also get better and better if it is to get past an ever more capable discriminator. Eventually, the generator should get really good at creating images that are indistinguishable from real images.

The discriminator and generator are set to compete against each other as **adversaries**, each trying to outdo the other, and in the process, both getting better and better. This architecture is called a **Generative Adversarial Network**, or **GAN**.

It's a clever architecture, not just because it uses competition to drive improvement, but also because we don't need to define detailed rules that describe a real image to be encoded into the loss function. The history of machine learning has taught us that we're not very good at defining such rules. Instead we let the GAN itself learn what a real image is.

What we've described is genuinely exciting. Yann LeCun, one of the world's leading machine learning researchers, called **adversarial training** "the coolest idea in machine learning in the last twenty years".

Training a GAN

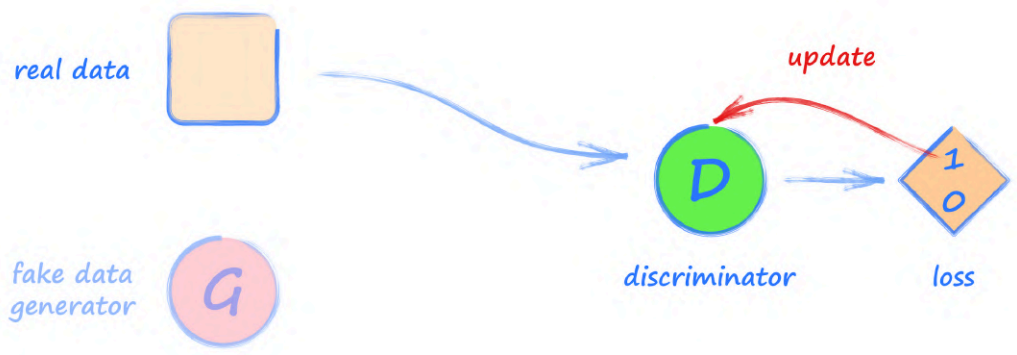
In a GAN, both the generator and discriminator need to be trained. What we don't want to do is train one of the elements, discriminator or generator, first using all the training data, and then train the other afterwards. We want the generator and discriminator to both learn together, without one getting too far ahead of the other.

The following **3** step training loop is one way to do this:

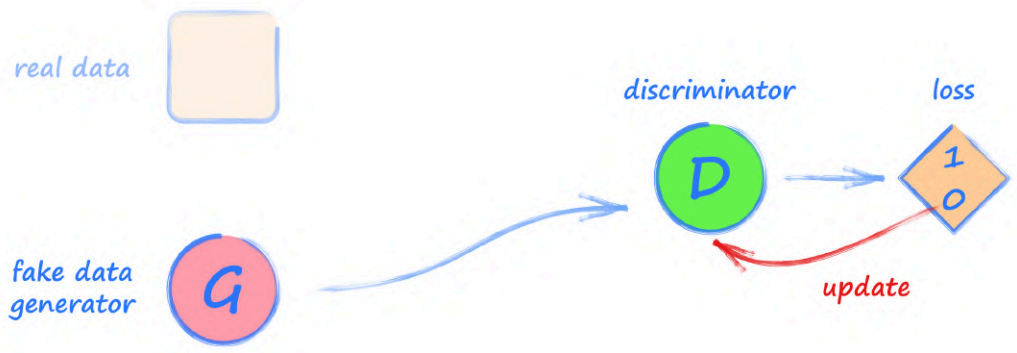
- **Step 1** - show the **discriminator** a real data example, tell it the classification should be **1.0**
- **Step 2** - show the **discriminator** the output of the generator, tell it the classification should be **0.0**
- **Step 3** - show the discriminator the output of the generator, tell the **generator** the result should be **1.0**

This is the heart of most GAN training schemes.

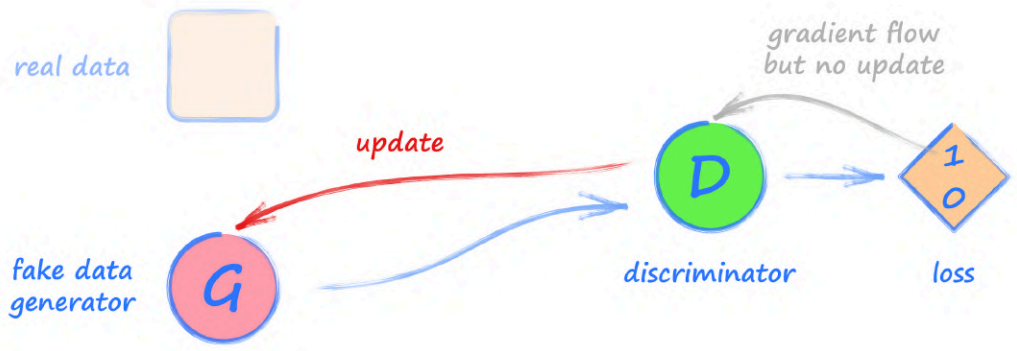
Let's draw some pictures to show what these steps actually mean.



Step 1 is the simplest and most familiar. We show the discriminator an image from the real dataset, and ask it to classify it. The output should be **1.0**, and we use the error to update the discriminator.



Step 2 also trains the discriminator but this time it is shown an image from the generator. The output should be **0.0**. We use the error to update the discriminator only. We must be careful not to update the generator in this step because we don't want to reward it for getting caught by the discriminator. When we code a GAN later, we'll see how to prevent the updates flowing through the computation graph back to the generator.



Step 3 trains the generator. We use it to generate an image which is presented to the discriminator to classify. The discriminator output should be **1.0**. That is, we want the generator to fool the discriminator into classifying the image as a real image, not the fake image we know it is. The error is used to update the generator only. We don't update the discriminator because we don't want to encourage it to get its classifications wrong. When we code a GAN, we'll see this is easy to arrange.

Phew! That sounds like a lot, but in practice we'll see it is very simple to implement.

GANs Are Hard To Train

We've just talked about how a GAN could or should work. In reality, training GANs can be difficult. If we set the generator and discriminator against each other, it's not hard to see that they will only improve each other if they are finely balanced. If the discriminator gets too good too quickly, the generator may never catch up. On the other hand, if the discriminator is too slow to learn, the generator will keep being rewarded for poor quality images.

GANs are such a new idea in machine learning that we're still at the early stages of understanding precisely what makes them work well, and what makes them fail. This is actually exciting, because we're working at the cutting edge of machine learning and any one of us could make a discovery that adds to the body of human knowledge!

For now, let's put aside this theoretical talk of how GANs can fail, and start building them. We can explore any problems with training as they emerge.

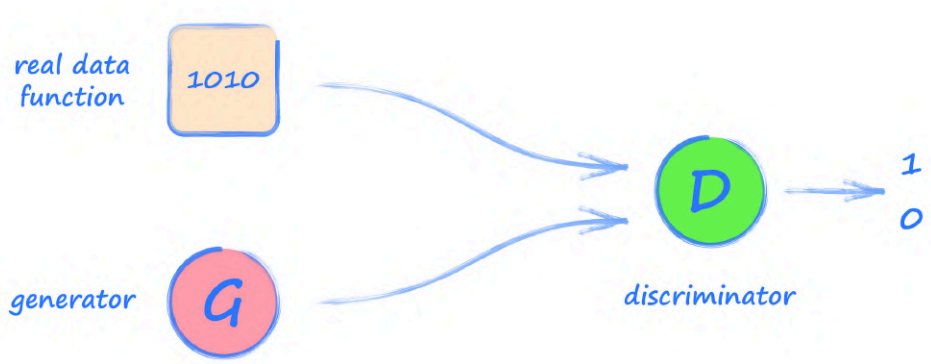
Learning Points

- **Classification** is a reduction of data. Classifying neural networks will reduce many input values to a much smaller number of output values, one for each class.
- **Generation** is normally an expansion of data. A generative neural network will expand a small number of input **seed** values to a much larger number of output values, image pixel values for example.
- A generative adversarial network **(GAN)** has two neural networks, a **generator** and a **discriminator**, set to compete against each other as **adversaries**. The discriminator is trained to classify data from a training set as **real**, and data from the generator as **fake**. The generator is trained to create data that looks real enough to fool the discriminator.
- Designing and training GANs to reliably succeed is difficult. GANs are new and the underlying theory describing how they work, and why training can fail, is not yet mature.
- The standard **GAN training loop** has **3** steps: (1) train the discriminator with a real data example, (2) train the discriminator with a generated data example, (3) train the generator to get a positive classification from the discriminator.

Simple 1010 Pattern

Let's build a GAN where the generator learns to create a **1010** pattern of values, a task that's simpler than generating images. The reason for this simplicity is that we want to focus on what GAN code looks like in general, and also practice how we observe training progress. Having done this simpler task, we'll be better prepared for the task of generating images.

As always, it is good to start with a pen and paper sketch of what we're trying to do.



We can see the overall architecture is that of a GAN. The set of real data has been replaced with a function that always gives us a **1010** pattern. We won't need a PyTorch **torch.utils.data.Dataset** object for such a simple source of data.

The **generator** is a neural network that outputs **4** values, which we hope will be a **1010** pattern after training. The **discriminator** takes a **4** value pattern and tries to determine if it is from the real data source or from the **generator**.

Let's code each of these elements in turn. Start a new notebook and import the standard libraries.

```
import torch
import torch.nn as nn

import pandas
import matplotlib.pyplot as plt
```

Real Data Source

The real data source can be a function that always returns a **1010** pattern.

```
def generate_real():
    real_data = torch.FloatTensor([1, 0, 1, 0])
    return real_data
```

Real life data is very rarely that precise or constant, so let's make this function a little more realistic by adding a bit of randomness to the high and low values. We'll need to import the python **random** module to make use of its **random.uniform()** function.

```
def generate_real():
    real_data = torch.FloatTensor(
        [random.uniform(0.8, 1.0),
         random.uniform(0.0, 0.2),
         random.uniform(0.8, 1.0),
         random.uniform(0.0, 0.2)])
    return real_data
```

Try this function to check it returns a tensor with **4** values, the first and third being a high random number between **0.8** and **1.0**, and the second and fourth being a low random number between **0.0** and **0.2**.

Building The Discriminator

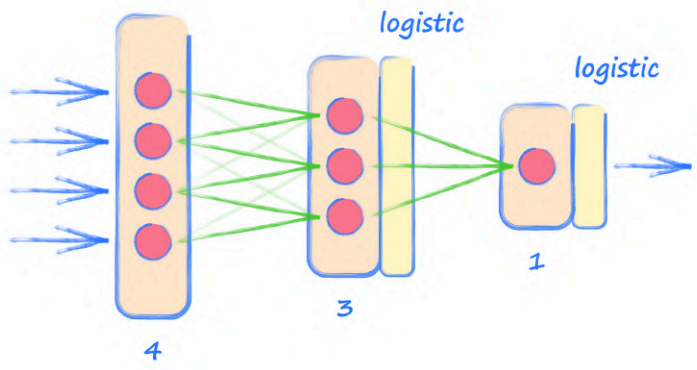
Let's code the discriminator. Just like before, it is a neural network with a class inherited from **nn.Module**. We follow the expected PyTorch patterns for initialising the network and providing a **forward()** function.

Have a look at the following constructor for a **Discriminator** class.

```
class Discriminator(nn.Module):

    def __init__(self):
        # initialise parent pytorch class
        super().__init__()

        # define neural network layers
        self.model = nn.Sequential(
            nn.Linear(4, 3),
            nn.Sigmoid(),
            nn.Linear(3, 1),
            nn.Sigmoid()
        )

        # create loss function
        self.loss_function = nn.MSELoss()

        # create optimiser, using stochastic gradient descent
        self.optimiser = torch.optim.SGD(self.parameters(), lr=0.01)

        # counter and accumulator for progress
        self.counter = 0
        self.progress = []

        pass
```

The code is almost exactly like the code we wrote earlier, with the network layers defined with **nn.Sequential**, a mean squared error loss function and stochastic gradient descent for the optimiser. We've created a **counter** and a **progress** list to keep track of the loss during training, just like we did before.

The network itself is simple. It takes **4** inputs at the input layer. That's because the patterns it will see consist of **four** values. It outputs a single value at the final layer. This value should be **1** for true, or **0** for false. The hidden middle layer has **3** nodes. It really is a very small network!



Just like before, the **forward()** function simply calls the model above, passing it the data and returning the network's output.

```
def forward(self, inputs):
    # simply run model
    return self.model(inputs)
```

The training function **train()** doesn't need to be different to the one we developed earlier.

```
def train(self, inputs, targets):
    # calculate the output of the network
    outputs = self.forward(inputs)

    # calculate loss
    loss = self.loss_function(outputs, targets)

    # increase counter and accumulate error every 10
    self.counter += 1
    if (self.counter % 10 == 0):
        self.progress.append(loss.item())
        pass
    if (self.counter % 10000 == 0):
        print("counter = ", self.counter)
        pass

    # zero gradients, perform a backward pass, update weights
    self.optimiser.zero_grad()
    loss.backward()
    self.optimiser.step()
```

```
    pass
```

We can see the standard pattern for a training function. It first asks the neural network for the **outputs** given the **inputs**. A **loss** is calculated by comparing it to the **targets**. The gradients within the network are calculated from this **loss**, and the learnable parameters are updated by taking a single step of the optimiser. We also keep track of how many times the **train()** function has been called with the **counter**, and we add the **loss** to the **progress** list every **10** calls.

Finally, we can add a function **plot_progress()** to draw a chart of the losses accumulated during training. It is no different to our earlier code.

```
def plot_progress(self):
    df = pandas.DataFrame(self.progress, columns=['loss'])
    df.plot(ylim=(0, 1.0), figsize=(16,8), alpha=0.1, marker='.',
grid=True, yticks=(0, 0.25, 0.5))
    pass
```

The similarity of this code with our MNIST classifier shouldn't surprise us. This discriminator is the same classifier, just with smaller layers and only one output value.

Testing The Discriminator

It is a good idea to test important elements of any machine learning architecture. Here we'll test the discriminator.

We haven't yet built the generator so we can't really test the discriminator as it competes with it. What we can do is check that the discriminator can separate real data from random data.

That might not sound like a useful test, but it is. It tells us that the discriminator has the **capacity** to at least separate real data from random noise. If it can't do that, then it is unlikely to be able to do the harder task of separating real data from fake data that looks like real data. So this test will catch any discriminator which was unlikely to ever be good at competing with the generator.

Let's create a function to generate random noise patterns.

```
def generate_random(size):
    random_data = torch.rand(size)
    return random_data
```

We could have written a function similar to **generate_real()** but this one is more generic and allows us to ask for a tensor of a given size. So **generate_random(4)** will return a tensor with **4** values chosen at random between **0** and **1**. Try it yourself to check it works with different sizes.

Let's now train a discriminator with a training loop that rewards the discriminator if it classifies:

- **1010** patterned data as real, with a target output of **1.0**
- random noise data as false, with a target output of **0.0**

The training loop looks like this.

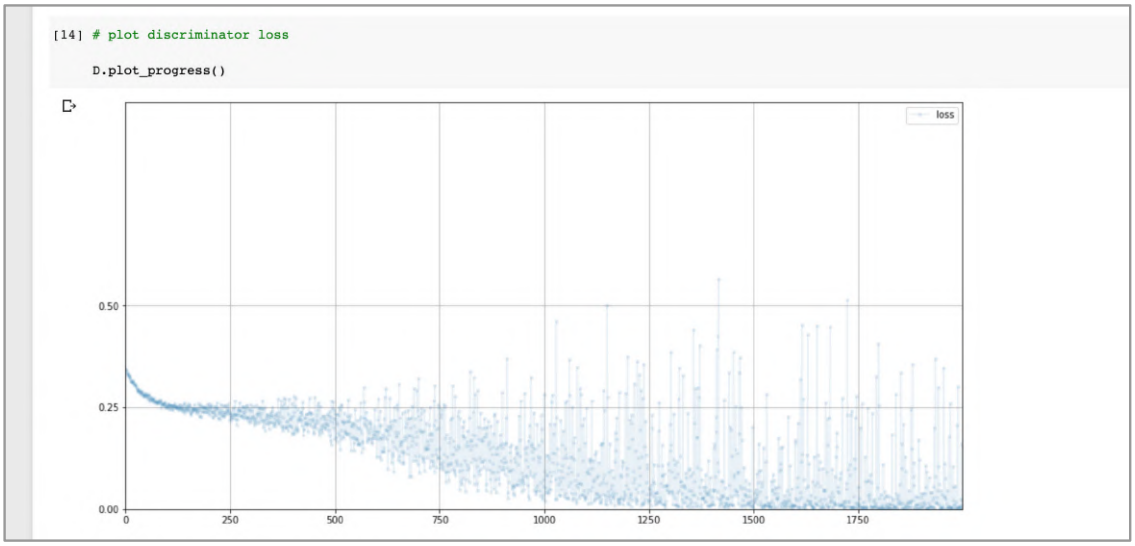
```
D = Discriminator()

for i in range(10000):
    # real data
    D.train(generate_real(), torch.FloatTensor([1.0]))
    # fake data
    D.train(generate_random(), torch.FloatTensor([0.0]))
    pass
```

The training loop runs **10,000** times. The discriminator's **train()** function is passed the real data from **generate_real()**, and a training target which is a tensor with the value **1.0**, for true. This will encourage the network to try to output **1.0** when it sees real data with a **1010** pattern. Similarly, the discriminator's **train()** function is also passed random noise from **generate_random()** and a target value of **0.0**, to encourage it to output **0.0** when it sees data that is not of the form **1010**.

Run the training loop code in a new cell. It should only take about ten seconds to complete. Once it's done, let's draw a chart of the loss to see how the training went.

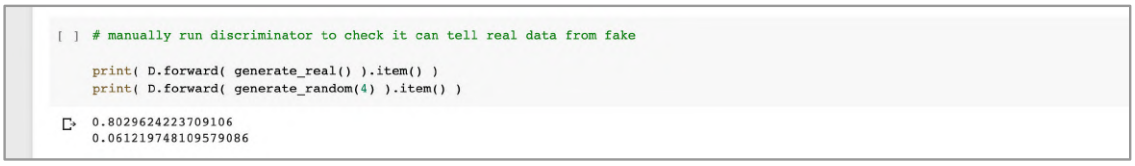
```
D.plot_progress()
```

My chart looks like this. Yours will not be too different.



The loss initially hangs around **0.25** and then falls towards zero as the discriminator gets better and better at separating the **1010** pattern from noise.

Before we move on, let's pass some patterns through the trained discriminator. If we pass a **1010** pattern we should expect an output close to **1.0**. If we pass a random pattern we should expect an output close to **0.0**.



This shows more directly the discriminator is working. Your own high and low values will be slightly different.

Let's pause and think about what we've done. We haven't proved the discriminator will work well competing against the generator. We can't do that yet. What we can do is show the discriminator can at least learn to separate the patterns from the real dataset from random noise. If it couldn't do that, we couldn't expect it to do the harder task of competing against the generator.

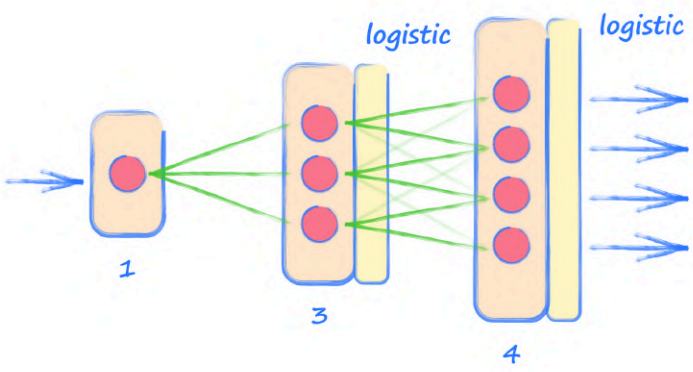
Building The Generator

Building the generator needs a little more thought, so we'll proceed step by step.

The generator is a neural network, and not a simple function, because we want it to learn. We hope its output will get past the discriminator. That means the last output layer needs **4** nodes, to match the real data.

How big should the hidden layer be? What about the generator's input layer? There isn't anything that forces us to use a specific size for these. They should be large enough to learn, but not so big that it takes a very long time, and with an overall aim to match the rate at which the discriminator learns so one doesn't get too far ahead of the other. For these reasons, many researchers start by mirroring the discriminator.

So let's try a generator with an input layer of **1** node, a hidden layer with **3** nodes, and an output layer of **4** nodes, which is the discriminator in reverse.



Like all neural networks, this one needs an input. What do we use as input to the generator? For now, let's do the simplest thing and feed it a constant value. We know overly large values make training harder, and normalising data ranges helps, so let's stick with a value of **0.5**. We can change our minds later if we hit a limitation.

Let's start defining a **Generator** class. We can copy the code from the **Discriminator** class and make changes to it.

```
class Generator(nn.Module):

    def __init__(self):
        # initialise parent pytorch class
        super().__init__()
```

```
        # define neural network layers
        self.model = nn.Sequential(
            nn.Linear(1, 3),
            nn.Sigmoid(),
            nn.Linear(3, 4),
            nn.Sigmoid()
        )

        # create optimiser, simple stochastic gradient descent
        self.optimiser = torch.optim.SGD(self.parameters(), lr=0.01)

        # counter and accumulator for progress
        self.counter = 0
        self.progress = []

        pass


    def forward(self, inputs):
        # simply run model
        return self.model(inputs)
```

The main difference is the definition of the neural network layers.

You may have spotted that we don't have a **self.loss**. That's because we don't need one. If we look back at the GAN training loop, we'll see that the only loss function we use is the one that is applied to the outputs of the discriminator. When the generator is updated, it is based on error gradients flowing back from that discriminator loss.

Let's now think about the generator's **train()** function. Training the generator is a little different to training the discriminator. With the discriminator, we know what the target output should be. With the generator, we don't. What we do have is the back-propagated gradients from the loss calculated from the output of the discriminator from step **3** of the GAN training loop we talked about earlier.

So, training the generator needs the discriminator too. There are several ways to code this. One neat way is to pass the discriminator to the generator's **train()** function. This keeps the training loop neat and simple.

Have a look at the following code.

```
def train(self, D, inputs, targets):
    # calculate the output of the network
    g_output = self.forward(inputs)

    # pass onto Discriminator
    d_output = D.forward(g_output)

    # calculate error
    loss = D.loss_function(d_output, targets)

    # increase counter and accumulate error every 10
    self.counter += 1
    if (self.counter % 10 == 0):
        self.progress.append(loss.item())
        pass

    # zero gradients, perform a backward pass, update weights
    self.optimiser.zero_grad()
    loss.backward()
    self.optimiser.step()

    pass
```

The code is easy to follow. The inputs are passed through the generator's own neural network with **self.forward(inputs)**. The generator's output **g_ouput** is then passed through the discriminator's neural network using **D.forward(g_output)** to give the classification output **d_output**.

The **loss** is calculated from this **d_output** and the desired target. The back-propagation of the error gradients start at this **loss**, which flows back through the computation graph through discriminator and then the generator.

The updates are triggered using **self.optimiser**, and not **D.optimiser**. This way, only the generator's link weights are updated, as intended by step **3** of the GAN training loop.

If you're more experienced with python, you might be concerned that passing an entire complex discriminator object to the generator's **train()** function is problematic. We don't need to be concerned because python doesn't pass a separate copy, it passes a reference to that same object, which is not just efficient, it also allows our generator code to change that object, necessary for back-propagating the error gradients through it. Don't worry if you don't understand this. It's simply a reassurance for readers more experienced with python.

We've also removed the printing of the count in the generator's **train()** function because the discriminator's **train()** does it too, and that one more accurately reflects progress through the real training data.

Finally, we can add the **plot_progress()** function to the Generator class. The code is exactly the same as the one in the Discriminator class.

Checking Generator Output

Again, it is good to verify that the individual elements of a larger machine learning architecture are working. Before we use the generator in training, let's just check it does output what we expect it to.

In a new cell we can run the following code to create a new generator object and pass it a tensor with a single value of **0.5**.

```
G = Generator()
G.forward(torch.FloatTensor([0.5]))
```

We can see the generator's output tensor contains four values, which is what we expect from the generator.

The pattern isn't of the form **1010** because the generator isn't trained yet.

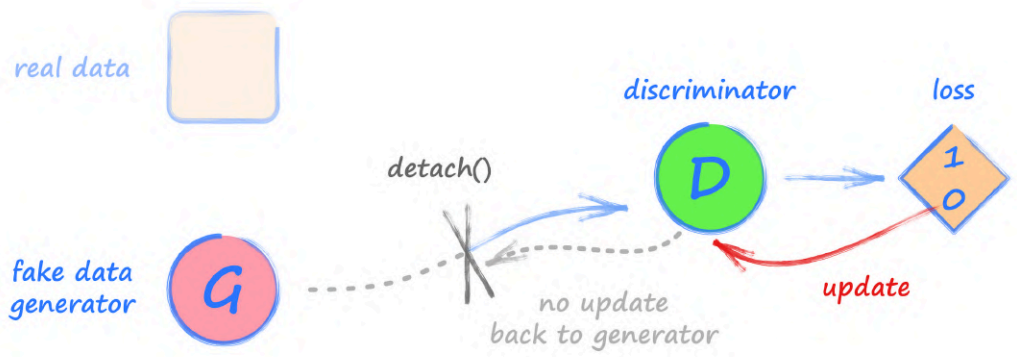
Training The GAN

Finally, we're ready to train the GAN using the **3** step training loop. Have a look at the following code.

```
# create Discriminator and Generator

D = Discriminator()
G = Generator()

# train Discriminator and Generator

for i in range(10000):

    # train discriminator on true
    D.train(generate_real(), torch.FloatTensor([1.0]))

    # train discriminator on false
    # use detach() so gradients in G are not calculated
    D.train(G.forward(torch.FloatTensor([0.5])).detach(),
torch.FloatTensor([0.0]))

    # train generator
    G.train(D, torch.FloatTensor([0.5]), torch.FloatTensor([1.0]))

    pass
```

We first create fresh discriminator and generator objects, before running the training loop **10,000** times. Inside the loop we can see the **3** steps of the GAN training loop we talked about earlier. For step **1**, we can see the discriminator being trained on the real data.

For step **2** we train the discriminator with a pattern from the generator. That **detach()** applied to the output of the generator detaches it from the computation graph. Normally, calling **backwards()** on the discriminator loss causes error gradients to be calculated all the way back along the computation graph - from the discriminator loss, through the discriminator itself, and then back through the generator. Because we're only training the discriminator, we don't need to calculate the gradients for the generator. That **detach()** applied to the generator output cuts the computation graph at that point. The following picture illustrates this.



You might ask why we do this. Surely there's no real harm in calculating the gradients in the generator even if we don't act on them? The benefit isn't huge in our tiny example, but if we have larger networks, the saving on computational effort can be significant.

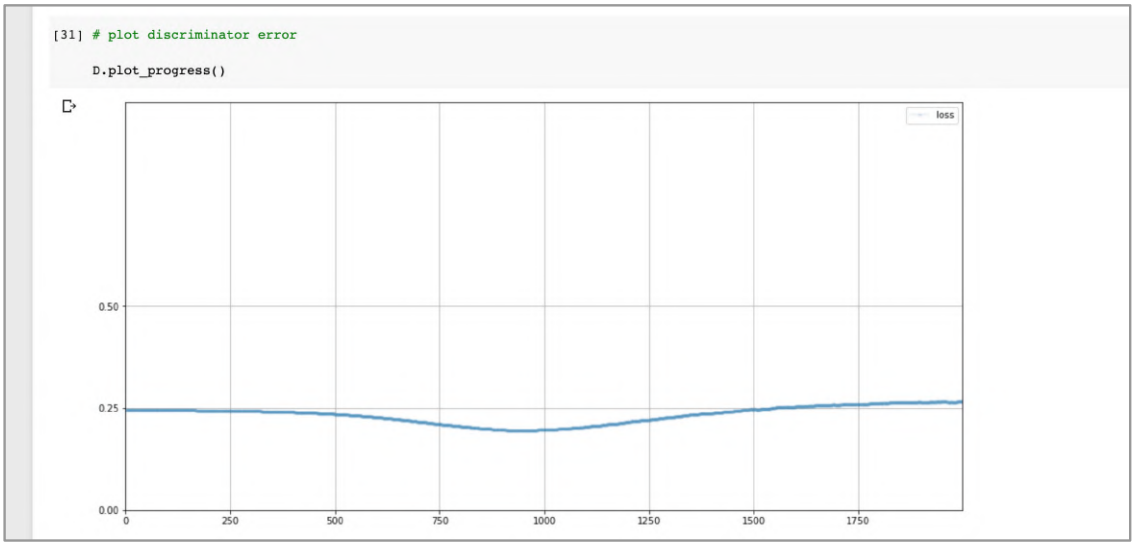
For step **3**, we train the generator, passing it the discriminator object, and a generator input value of **0.5**. This time we don't use **detach()** because we want the error gradients to flow all the way back from the discriminator loss to the generator. The generator's **train()** function only updates the generator's link weights, so we don't need to do anything special to prevent the discriminator being updated. Neat!

Because training a GAN can take more than a few seconds, it is useful to place a **%%time** instruction at the top of the cell to get an idea of how long training will take if we run several experiments.

Run the training code yourself. For me it takes about **16** seconds.

```
[ ] %%time

    # create Discriminator and Generator

    D = Discriminator()
    G = Generator()

    # train Discriminator and Generator

    for i in range(10000):

        # train discriminator on true
        D.train(generate_real(), torch.FloatTensor([1.0]))

        # train discriminator on false
        # use detach() so gradients in G are not calculated
        D.train(G.forward(torch.FloatTensor([0.5])).detach(), torch.FloatTensor([0.0]))

        # train generator
        G.train(D, torch.FloatTensor([0.5]), torch.FloatTensor([1.0]))

        pass

    counter =  10000
    counter =  20000
    CPU times: user 14.8 s, sys: 1.46 s, total: 16.3 s
    Wall time: 16.2 s
```

Let's plot the loss at the discriminator to see how training went using **D.plot_progress()**.



This is interesting!

Previously we expected our loss values to fall towards zero during training as our neural networks got better at their job. Here the loss values remain around **0.25**. What's so special about this number?

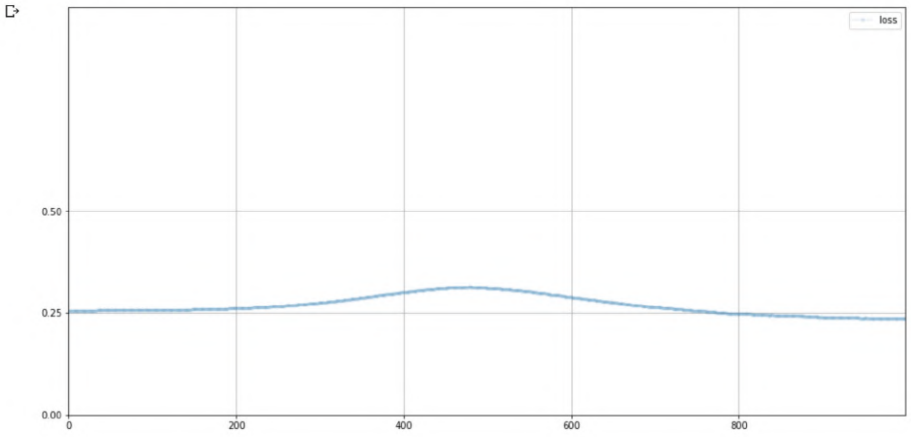
When a discriminator is equally bad at spotting real data from fake data, it is equally uncertain about whether to output **0.0** or **1.0**, so it outputs **0.5**. Because we're using a mean squared error, the loss is **0.5** squared, or **0.25**.

Here, as training progresses, the loss falls slightly, but not by much. It suggests the network has improved at its job a little bit. It's not clear whether this means it has gotten better at recognising real **1010** patterns, or better at spotting the generated patterns as fake, or both.

Towards the end of training the loss value rises back to **0.25**. This is good. It means the generator has learned to generate realistic patterns, and the discriminator can't tell real and generated patterns apart. Which means its output will be **0.5**, halfway between **0** and **1**. And that's why the loss value rises back up to **0.25**.

Let's have a look at the loss values that come from trying to train the generator, using **G.plot_progress()**.

```
[32] # plot generator error
     G.plot_progress()
```



We can see the discriminator starts off not very confident at classifying generated patterns as real or fake. At about halfway through training the loss has risen slightly, which suggests the generator has actually improved and started to fool the discriminator. Towards the end we can see an equalisation between the generator and discriminator.

It's always a good idea to have a look at the loss during training to get an insight into how the training actually went. Here, we didn't see a complete failure of training, nor did we see wild oscillations in loss values suggesting unstable learning.

Let's now try our trained generator to see what pattern it creates. Our first generated data!

```
[33] # manually run generator to see it's outputs
     G.forward(torch.FloatTensor([0.5]))
     tensor([0.9082, 0.0516, 0.9118, 0.0463], grad_fn=<SigmoidBackward>)
```

We can see the generator does indeed create a **1010** pattern with values that are clearly high in the first and third position, and low at the second and fourth position. The high values at around **0.9**, and the low values at around **0.05**, are pretty good.

As an additional experiment, let's visualise how that **1010** pattern evolves during training. To do this we can create an empty list **image_list** before our training loop, and then store the generator's output at every **1000** training cycles.

```
# add image to list every 1000
```

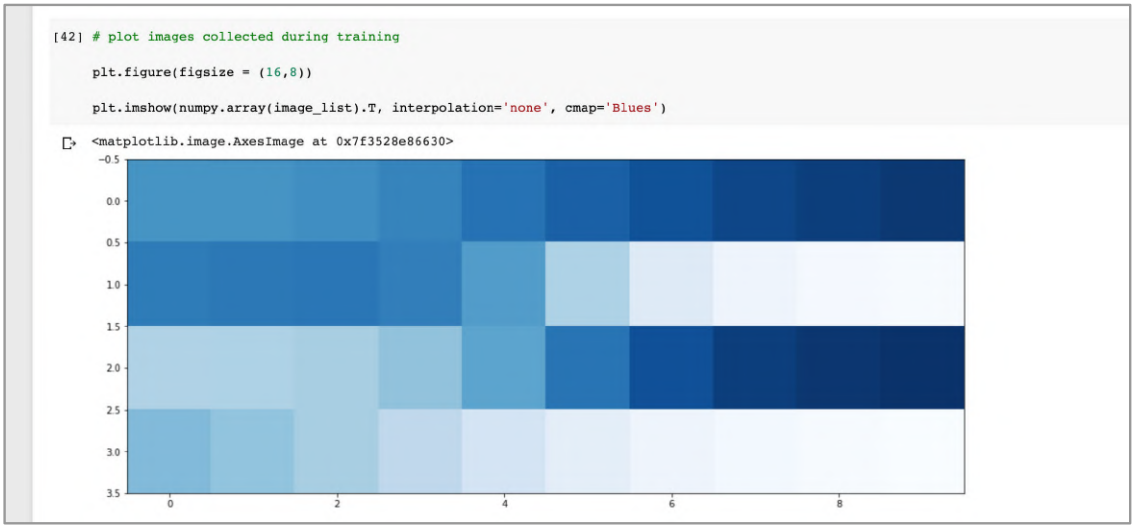
```
if (i % 1000 == 0):
  image_list.append(
G.forward(torch.FloatTensor([0.5])).detach().numpy() )
```

To extract the values from the generator's output tensor, in the form of a numpy array, we need to **detach()** it from the computation graph before using **numpy()**.

After training we should have **10** patterns in our **image_list**. The following code converts that list of **10** patterns, each with **4** values, into a **10** by **4** numpy array, and then flips it diagonally so we can see it evolve towards the right.

```
plt.figure(figsize = (16,8))
plt.imshow(numpy.array(image_list).T, interpolation='none',
cmap='Blues')
```

The code does need us to first import the **numpy** library at the top of our notebook.



This chart shows quite clearly how the generator improves over time. Initially the generator creates a rather indistinct pattern. About half way through training the generator suddenly is able to generate a **1010** pattern which continues to become clearer during the remainder of the training.

The code we've developed is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/06_gan_simple_pattern.ipynb

Take a well-deserved break! We've done a lot of work building our first GAN and training it successfully.

As you take that break, it is worth meditating on the fact that the generator has never seen the training data directly, yet has learned to create a pattern that could easily have come from the real data source.

Learning Points

- A good pattern for developing and training a GAN includes the following steps. (1) **preview** data from the real dataset, (2) **test the discriminator** can at least learn to separate real data from random noise, (3) **test the untrained generator** can create data of the right shape, (4) **visualise the loss** values to understand how well training progressed.

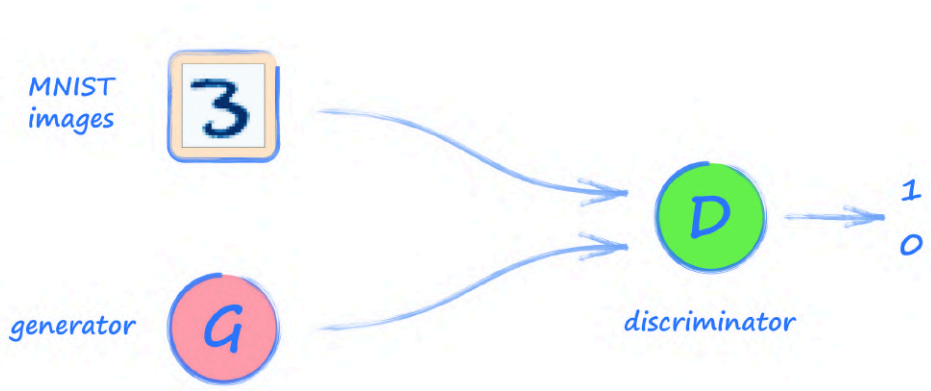
- A successfully trained GAN has a discriminator that can't tell real from generated data. Its output is **0.5**, half way between **0.0** and **1.0**. The ideal mean squared error loss is **0.25**.

- It can be useful to separately visualise the discriminator and generator loss. The **generator loss** is the loss at the discriminator caused by a generated data example.

Handwritten Digits

In the last section we did a lot of work coding the machinery of a GAN, and getting familiar with using it. It means we have less work to do as we progress from the simple **1010** pattern to seeing if we can train a GAN to generate images that look like hand-written digits.

It's always helpful to start with a picture of what we're trying to build.



The overall architecture is still the same. The real images are provided by the MNIST dataset, which we've used before. The generator's job is to create images of the same size. As training progresses, we hope the generator's images become more and more realistic, fooling the discriminator.

Create a new notebook and import the usual libraries.

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset

import pandas, numpy, random
import matplotlib.pyplot as plt
```

As we build our code, we'll find that we're copying code we developed earlier when we built the MNIST classifier and the GAN for **1010** patterns.

Dataset Class

We'll use a PyTorch **torch.utils.data.Dataset** class to load the MNIST data from the source CSV files. We can copy the code for the MnistDataset class we developed before. We don't need to make any changes.

That class packaged up the data as tensors, for each record returning a **label**, image pixel values rescaled to the range **0** to **1**, and a one-hot **target** tensor.

To access the MNIST CSV data files, we'll also need to mount our google Drive as we did before.

Test the Dataset class to ensure it works by plotting images of selected records from the training dataset.

MNIST Discriminator

The discriminator in a GAN is a classifier, and we've already built a classifier for MNIST images. In fact, that MNIST classifier's code was almost identical to the discriminator we used for the **1010** pattern GAN. The only difference was the size of the neural network

Here we can copy the code for the **1010** GAN discriminator we just developed. The only changes we need to make are to the neural network layer sizes.

```
self.model = nn.Sequential(
    nn.Linear(784, 200),
    nn.Sigmoid(),
    nn.Linear(200, 1),
    nn.Sigmoid()
```

```
)
```

Everything else in the Discriminator class can stay exactly the same, including the **forward()**, **train()**, and **plot_progress()** functions.

Testing The Discriminator

Before we build the generator, we should test the discriminator to check that it can at least separate real images from random noise. It should be able to because we've already used a very similar neural network to classify images as digits **0** to **9**.

The following code works through the **60,000** images in the training data and encourages the discriminator to label them as real with an output of **1.0**.

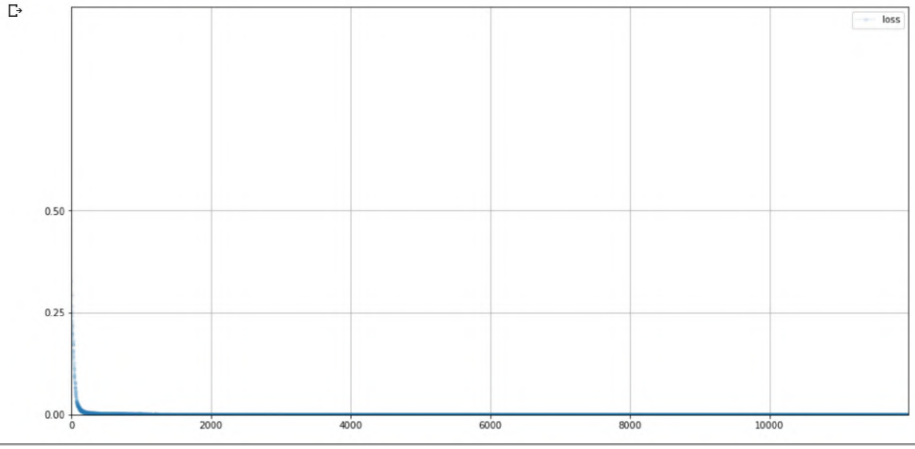
```
D = Discriminator()

for label, image_data_tensor, target_tensor in mnist_dataset:
    # real data
    D.train(image_data_tensor, torch.FloatTensor([1.0]))
    # fake data
    D.train(generate_random(784), torch.FloatTensor([0.0]))
    pass
```

For each real image, another fake image made of entirely random valued pixels is created using **generate_random(784)**. The discriminator is trained to label these as fake with an output of **0.0**.

Use the **%%time** instruction at the top of the cell to get an idea of how long it takes to work through the real and random training data. It took about two and a half minutes for me.

Let's plot a chart of the loss during training.

```
[17] # plot discriminator loss

     D.plot_progress()
```



The loss falls towards zero and stays close to it, which is the behaviour we want to see.

Let's manually test the trained discriminator by running a few randomly selected images from the dataset through the trained discriminator, as well as a few images of random noise.

```
[18] # manually run discriminator to check it can tell real data from fake

     for i in range(4):
       image_data_tensor = mnist_dataset[random.randint(0,60000)][1]
       print( D.forward( image_data_tensor ).item() )
       pass

     for i in range(4):
       print( D.forward( generate_random(784) ).item() )
       pass

     0.9968416690826416
     0.9824121594429016
     0.9948784112930298
     0.9970876574516296
     0.005318928975611925
     0.0044152457267045975
     0.006996214855462313
     0.005926067009568214
```

We can see the real images result in a high output value, meaning the discriminator thinks they are real. Similarly, the random noise images are given a low value.

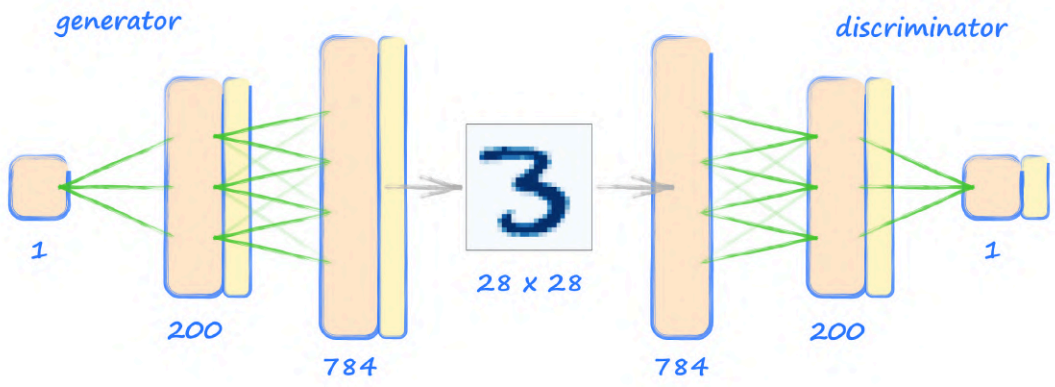
This confirms the discriminator has the capacity to separate real from random images. This doesn't surprise us as we've already shown that a very similar network can classify the images amongst ten classes.

MNIST Generator

Let's now turn our attention to the generator, which will be more interesting.

We want the generator to create images of the same format as those in the MNIST dataset. That is, **784** pixel values for **28** by **28** images.

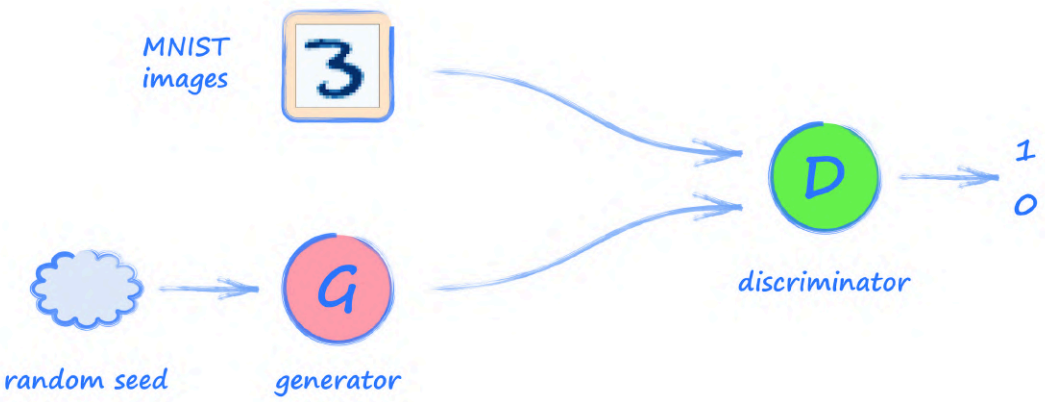
Reversing the discriminator's neural network is a good starting point. That gives us a network with an output layer of **784** nodes, a hidden layer of **200** nodes, and an input later of **1** node. The following picture shows the generator and discriminator networks next to each other. We can see clearly how the generator's output of **784** pixel values is precisely what the discriminator expects as input.



With our previous **1010** GAN, the generator was trained to produce a **1010** pattern every time it was used. Here, we don't want our generator to always produce the same output every time it is used. We want it to produce diverse images that represent all the digits seen in the training data. We want it to generate images that look like threes, sevens, fours, nines, and so on.

Let's think about how to do that. We know that a neural network will always produce the same output for a given input. Remember, it is the training of a neural network that is partly random, not the calculation of an output for a given input.

This points us to changing the generator input so it's no longer the constant **0.5** we used previously. We can use random inputs to the generator for each training cycle. Let's update our architecture picture to show this **random seed**.



Why do we think a random seed into the generator will help it create different images?

Well, we don't yet know for sure, but we can expect the generator learns to create different outputs for different ranges of the input. For example, it might learn to create an image of a three for an input in the range **0.0** to **0.2**, or nine for an input in the range **0.4** to **0.6**.

The code for our generator can be copied directly from the **1010** GAN generator. We only need to change the neural network layer sizes.

```
self.model = nn.Sequential(
    nn.Linear(1, 200),
    nn.Sigmoid(),
    nn.Linear(200, 784),
    nn.Sigmoid()
)
```

Check Generator Output

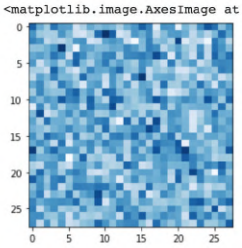
Before training the GAN, let's check the generator does output data of the right format.

```
G = Generator()
output = G.forward(generate_random(1))
img = output.detach().numpy().reshape(28,28)
plt.imshow(img, interpolation='none', cmap='Blues')
```

We first create a new generator object, and then run it with a random seed as input to give us an **output** tensor. You can check it has **784** values using **output.shape**.

```
[18] # check the generator output is of the right type and shape

     G = Generator()

     output = G.forward(generate_random(1))

     img = output.detach().numpy().reshape(28,28)

     plt.imshow(img, interpolation='none', cmap='Blues')
```

```
<matplotlib.image.AxesImage at 0x7f9d10dcf2b0>
```



Viewed as an image, we can see it is pretty random, which makes sense because the generator is not yet trained. If there was a pattern in the image, it would mean we'd made an error somewhere.

Training The GAN

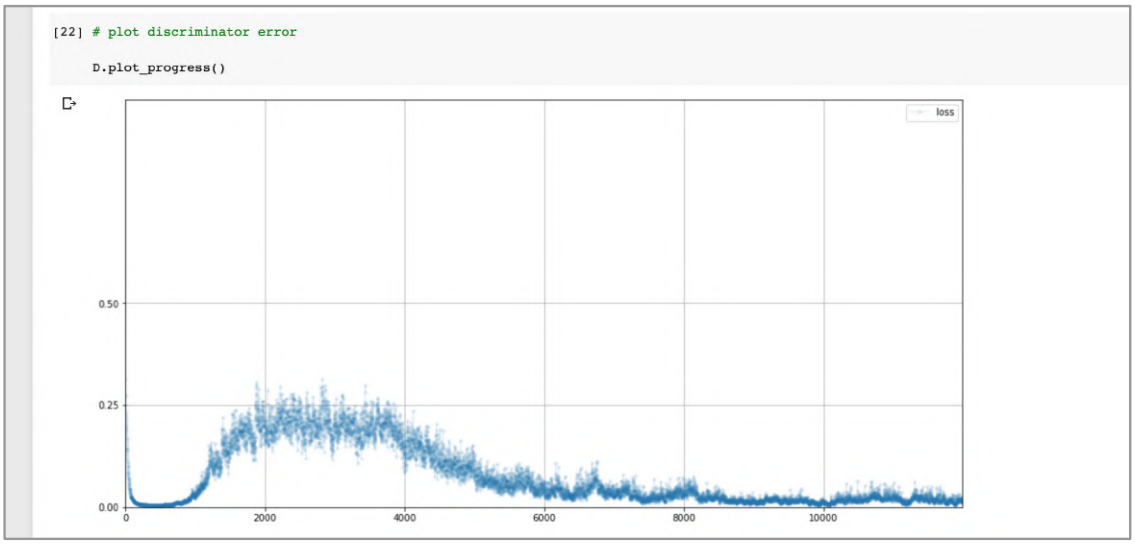
Let's train this GAN. The training loop is the same as before. The only changes are the data passed to the discriminator and generator.

```
# create Discriminator and Generator

D = Discriminator()
G = Generator()

# train Discriminator and Generator

for label, image_data_tensor, target_tensor in mnist_dataset:

    # train discriminator on true
    D.train(image_data_tensor, torch.FloatTensor([1.0]))

    # train discriminator on false
    # use detach() so gradients in G are not calculated
    D.train(G.forward(generate_random(1)).detach(),
torch.FloatTensor([0.0]))

    # train generator
    G.train(D, generate_random(1), torch.FloatTensor([1.0]))
```

```
    pass
```

It takes a short while to complete the training. For me it took just over four minutes. The counter is printed every **10,000**, and grows to **120,000** because the discriminator is trained with **60,000** MNIST images and **60,000** generated images.

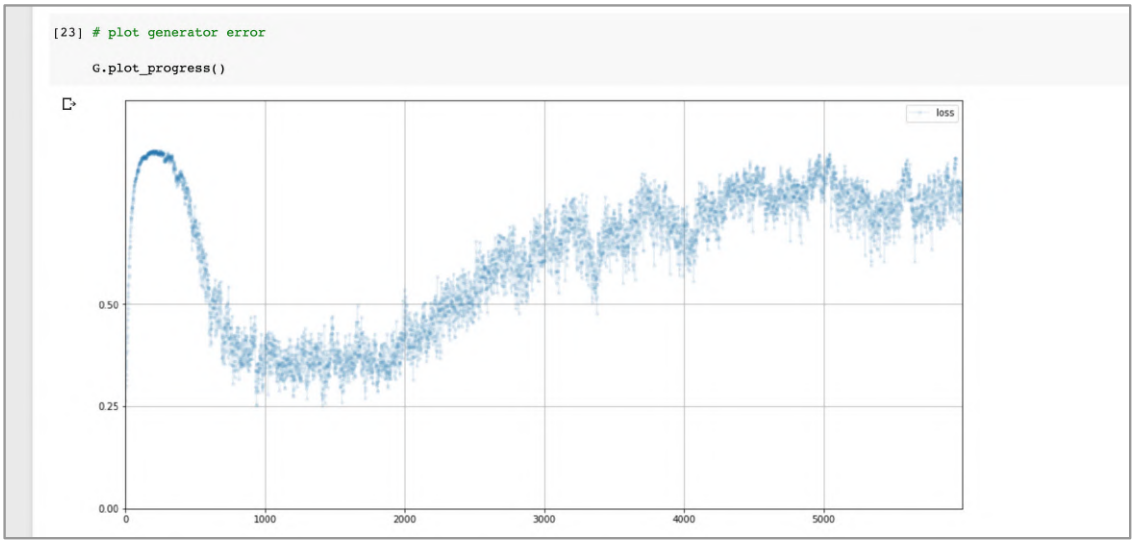
Let's plot the **loss** values from training the discriminator.



That's an interesting chart! The loss values fall towards zero and stay low for a while, suggesting the discriminator is ahead of the generator. Then the loss values rise to approximately just under **0.25**, which suggests the discriminator and generator are being balanced. Unfortunately, the discriminator then gets ahead again and the loss values fall and remain low.

Remember, we want the **loss** values to be around **0.25** for a balanced discriminator and generator, where the discriminator is not confident at telling real images from generated images. If the **loss** values fall towards zero, it suggests the generator has not learned to fool the discriminator.

Let's look at the **loss** values that come from training the generator.



Initially the discriminator identifies the generated images correctly and that's why the loss values are high. Then the loss falls towards **0.25** as the generator and discriminator approach balance for a period. The second half of the training period sees the loss rise as the discriminator gets stronger than the generator again.

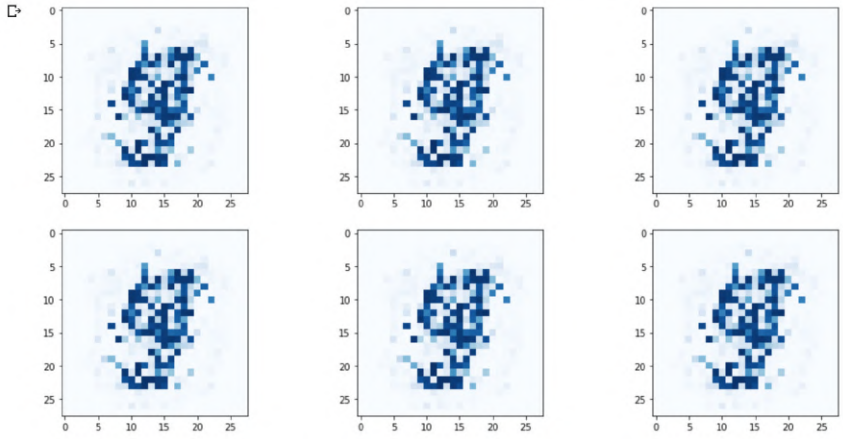
Let's have a look at the actual images the generator makes, not just for interest, but in case there is something we can learn from them.

Because we're expecting different images to be produced by different random seeds, we should make a plot of more than one output image.

```
# plot a 3 column, 2 row array of generated images
f, axarr = plt.subplots(2,3, figsize=(16,8))
for i in range(2):
    for j in range(3):
        output = G.forward(generate_random(1))
        img = output.detach().numpy().reshape(28,28)
        axarr[i,j].imshow(img, interpolation='none', cmap='Blues')
        pass
    pass
```

This code uses matplotlib's ability to create a plot as a grid of several different charts. Here we create a **3** by **2** grid of separately generated images.

```
[32] # plot several outputs from the trained generator

     # plot a 3 column, 2 row array of generated images
     f, axarr = plt.subplots(2,3, figsize=(16,8))
     for i in range(2):
         for j in range(3):
             output = G.forward(generate_random(1))
             img = output.detach().numpy().reshape(28,28)
             axarr[i,j].imshow(img, interpolation='none', cmap='Blues')
             pass
         pass
```



The first thing we notice about the generated images is that they aren't random noise, but do have a definite shape. The dark areas are in the middle of the image, just like the real images of hand-written digits. This is good. Even better, the image does seem to contain what could be the beginnings of a recognisable digit. I'd say the image was almost a **9**, but you could also see a **5** emerging.

This is a good start. Even if the image isn't a perfectly drawn digit, we've still achieved a significant milestone with very simple code. Remember, the generator has not directly seen the images in the MNIST dataset, and yet it has learned to create images which are not random noise, but the beginnings of recognisable hand-written digits.

This a moment to pause and celebrate. Our first GAN generated images!

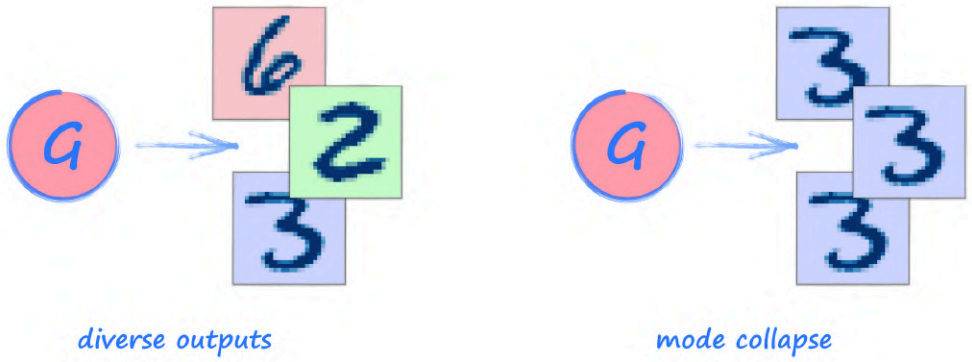
Our first attempt at writing a GAN for generating images of hand-written digits is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/07_gan_mnist_first_attempt.ipynb

Now that we've paused and celebrated, let's look at those images again. We can see that all the generated images look the same. If they are different, the differences must be so small that we can't tell just by looking at them.

Mode Collapse

What we've just seen is unfortunately a very common problem with GAN training called **mode collapse**.



In our MNIST example, we want the generator to be able to create images of all ten digits. When mode collapse happens, the generator will only produce one, or a small number, of the possible ten digits. We can see our own generator suffered from mode collapse because it created the same image again and again.

The reasons why mode collapse happens are not perfectly understood. Research is ongoing and active, and some ideas are more established than others.

One way of looking at mode collapse is to see that the generator has raced ahead of the discriminator and found a single image that always gets classed as real, before the discriminator has learned to provide good quality feedback to the generator. To remedy this, it is tempting to try ideas like training the discriminator more often than the generator. In practice this has been found not to work well. This suggests the solution lies not just in the quantity, but in the quality of training.

In our example, the rising loss values from the generator show that it is not learning, perhaps because the discriminator is not doing a good job providing it with good feedback. Again, this points to training quality as the challenge.

In the next section we'll experiment with some ideas to improve the quality of the feedback that the discriminator gives to the generator.

Improving GAN Training

Make a copy of the previous notebook which was our first attempt at a GAN for generating images that look like handwritten digits.

Here we'll try to fix the mode collapse and image clarity problems by trying to improve the training quality in our GAN. We've already seen some ideas for doing this when we developed refinements to our MNIST classifier.

The first refinement is to use the binary cross entropy **BCELoss()** instead of the mean squared error **MSELoss()** for the loss function. We've already talked about how this loss makes more sense when our network is performing a classification task. It also punishes incorrect answers, and rewards correct ones, more strongly than the **MSELoss()**.

```
self.loss_function = nn.BCELoss()
```

The next refinement we can make is to use **LeakyReLU()** activation functions in both the discriminator and generator. We'll only apply them after the middle layer, and keep the **Sigmoid()** for the final layer as we want the outputs to be in the range **0** to **1**. We've previously talked about how the **LeakyReLU()** activation reduces the problem of vanishing gradients for large signal values. It's a popular way of improving the quality of training neural networks in general.

Another refinement is to take the signals in the neural network and normalise them to ensure they are centred around a mean of zero, and have their variance limited to avoid network saturation from large values. We've already seen how **LayerNorm()** has a positive effect on training.

The following code describes this improved discriminator neural network.

```
self.model = nn.Sequential(
    nn.Linear(784, 200),
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 1),
    nn.Sigmoid()
)
```

The code for the generator has the same changes.

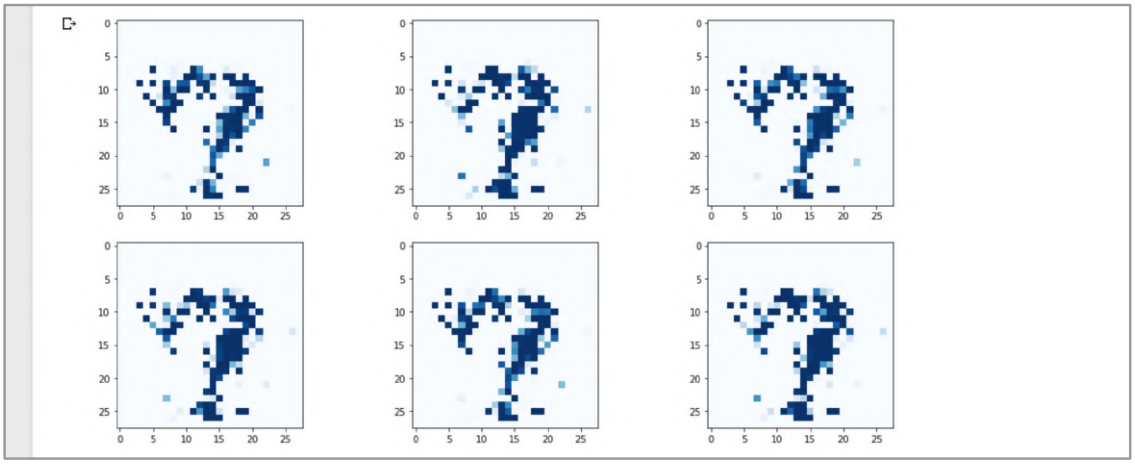
```
self.model = nn.Sequential(
    nn.Linear(1, 200),
```

```
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 784),
    nn.Sigmoid()
)
```

Another refinement we tried earlier was the Adam optimiser. Let's use it for both the discriminator and the generator.

```
self.optimiser = torch.optim.Adam(self.parameters(), lr=0.0001)
```

Let's see the effect of these four changes.



Sadly, we still have mode collapse. The images themselves are clearer with more distinct structure, but still not an obvious digit.

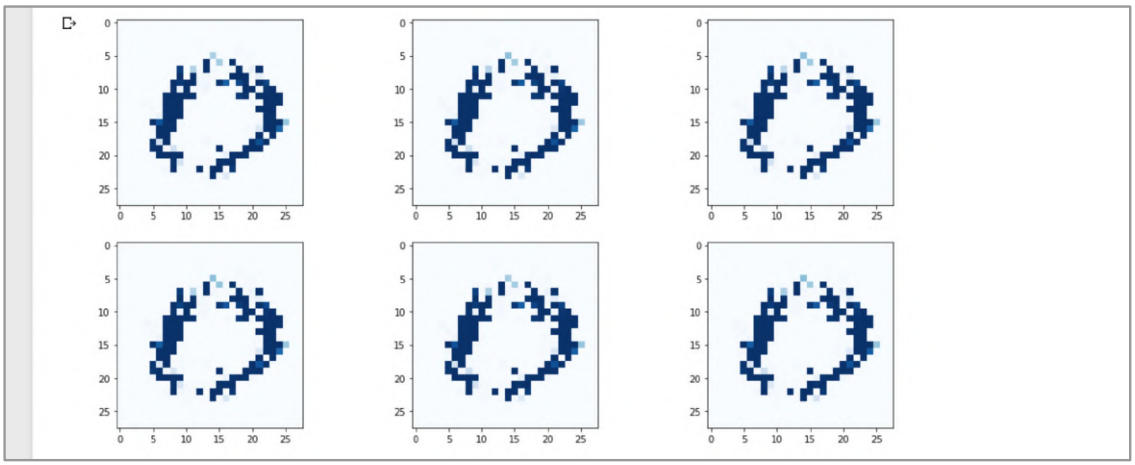
Let's think a little more deeply about how we can improve our GAN architecture.

The start of the generation process is the seed. We first had a constant value of **0.5**, and then we changed it to be a random value because we know that any neural network will always give the same output for a fixed input. Perhaps the generator's neural network finds it too hard to translate that single value into **784** pixel values that represent each of the **10** digits.

We could make it easier by providing a larger number of input seeds. We could try **100** input nodes, each of which will be a random value. Let's update the definition of the generator's neural network.

```
self.model = nn.Sequential(
    nn.Linear(100, 200),
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 784),
    nn.Sigmoid()
)
```

Let's see the effect this has.



The images are much cleaner and look much more like handwritten digits. These particular outputs look like a zero. Sadly, all the generated images are effectively the same. We're still suffering from mode collapse.

Don't take this personally - even the best GAN researchers struggle with mode collapse!

If we keep thinking about the random values we use to seed the generator, and also to feed the discriminator, we realise they shouldn't be the same.

- The random values used as image pixel values fed to the discriminator should be uniformly chosen from the range **0** to **1**. The range is **0** to **1** because that's the range for pixels from the real dataset. Because we're testing the discriminator's performance against neutral randomness, the values should be uniformly chosen, not biased like a normal distribution.

- The random values chosen to feed the generator don't have to be in the range **0** to **1**. We know that normalising signals in a network to be centred around **0** and have a limited variance can help training. We first discussed this in *Make Your Own Neural Network*, where we used it to initialise the weights of a neural network. It makes sense to pick the seeds from a normal distribution centred around **0** with a variance of **1**.

So let's separate out our function for generating random data into two. The functions look identical but the difference is **torch.rand()** and **torch.randn()**.

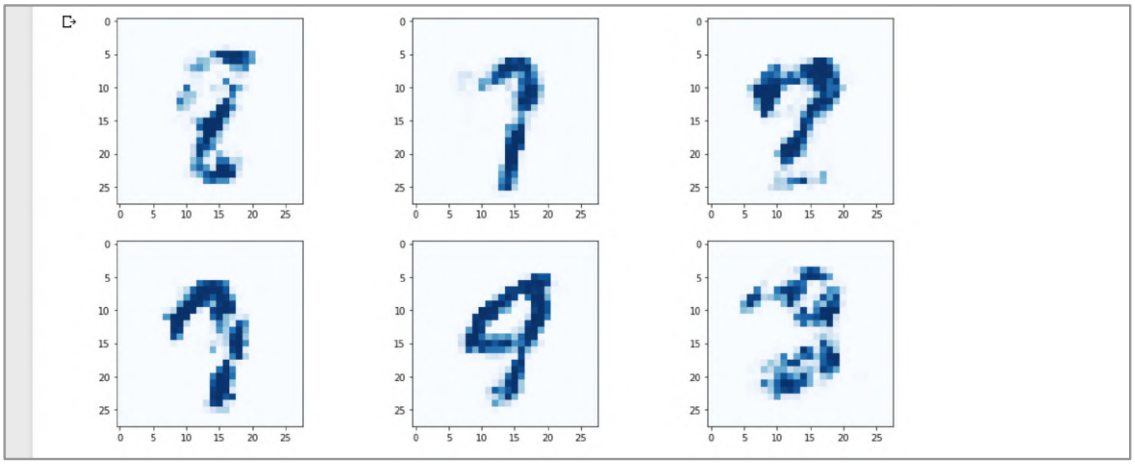
```
def generate_random_image(size):
    random_data = torch.rand(size)
    return random_data


def generate_random_seed(size):
    random_data = torch.randn(size)
    return random_data
```

We'll need to use **generate_random_image(784)** whenever we feed the discriminator, and **generate_random_seed(100)** whenever we feed the generator.

The following shows the GAN training loop with these changes.

```
D = Discriminator()
G = Generator()

# train Discriminator and Generator

for label, image_data_tensor, target_tensor in mnist_dataset:

    # train discriminator on true
    D.train(image_data_tensor, torch.FloatTensor([1.0]))

    # train discriminator on false
    # use detach() so gradients in G are not calculated
    D.train(G.forward(generate_random_seed(100)).detach(),
torch.FloatTensor([0.0]))

    # train generator
    G.train(D, generate_random_seed(100), torch.FloatTensor([1.0]))

    pass
```

Let's see if this improves the results.



Yippee! This seems to have fixed the mode collapse issue. We now have different digits being generated. We can see shapes that look like an **8**, a **2**, and a **3**. Some are ambiguous, one of them looks like it could be a **4** or **9**.

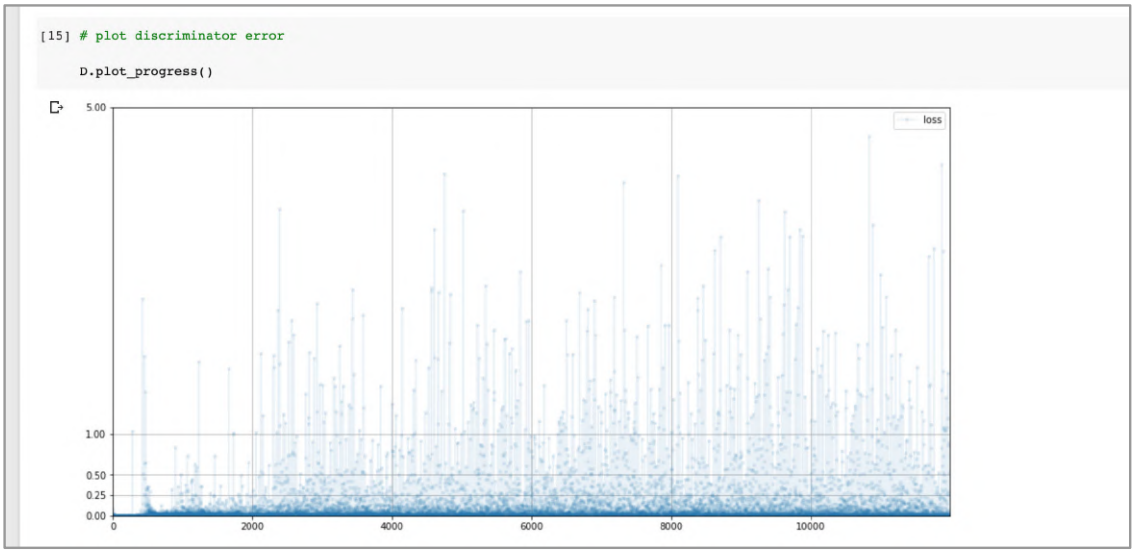
Let's state again what we've achieved. We've trained a generator to draw images of digits that look like they could be from the training data, but without having seen any real images directly. That's pretty cool. And there's more. That single trained generator can create several kinds of digit, just by varying the random seed.

This is quite an achievement. Fixing mode collapse is sometimes extremely difficult, and in too many cases no solution is found.

Let's also have a look at the loss charts to see if they give us any insight. Because we're now using **BCELoss()**, the values aren't always within the range **0** to **1**, so we'll update the **plot_progress()** function in both the discriminator and generator to remove the upper limit on the loss range, and add more horizontal grid lines too.

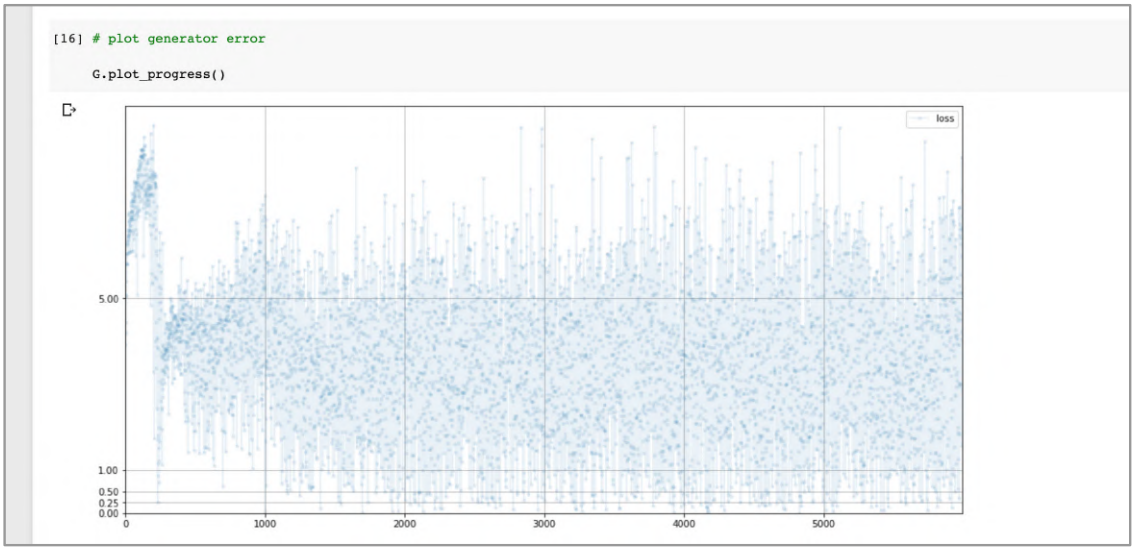
```
def plot_progress(self):
    df = pandas.DataFrame(self.progress, columns=['loss'])
    df.plot(ylim=(0), figsize=(16,8), alpha=0.1, marker='.', grid=True,
yticks=(0, 0.25, 0.5, 1.0, 5.0))
    pass
```

The following shows the discriminator loss.



We can see the loss falls rapidly towards zero, and stays near zero with occasional jumps during training. This suggests we still haven't achieved balance between the generator and discriminator.

The following chart shows the loss from training the generator.



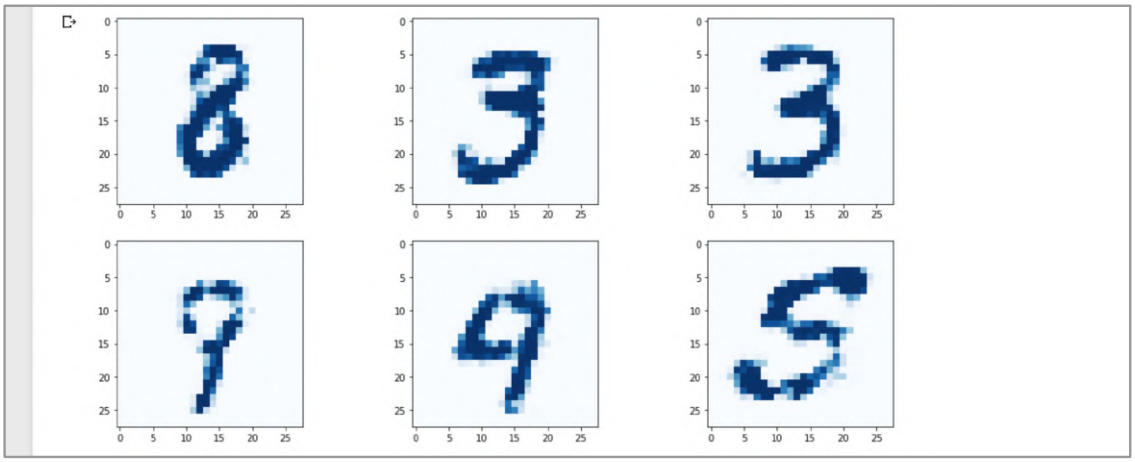
We can see the loss initially rising, which means the generator has fallen behind the discriminator early in training. After that the loss falls and stays around about **3**. Remember, unlike the MSELoss, the BCELoss doesn't have an upper limit of **1.0**.

These loss charts may look discouraging because they contain a wider spread of loss values, but they are better than the loss charts before these improvements were made. Those charts showed the discriminator loss falling to zero fairly neatly, that is, without much spread in the loss values. They also showed the generator loss rising, again fairly neatly. The neatness might look desirable but a rising generator loss is not what we want. It is preferable to have the generator loss varying but mostly averaging around a finite value.

A good question to ask is what the BCELoss should be if we did reach equilibrium. If we run our simple **1010** GAN, which we know reaches equilibrium, but use BCELoss, we'll see both generator and discriminator losses approach a value of about **0.69**. Try it yourself. By using the mathematical definition of binary cross entropy for a perfectly uncertain classifier, the ideal loss can be worked out to be **ln(2)** or **0.693**. You can read more about this in Appendix A.

It's great that we've fixed the mode collapse problem, but the images themselves aren't amazing. Let's see if longer training with more epochs can help. We can easily wrap our GAN training loops with an outer loop for epochs.

The following images are from **4** epochs of training, that is, working through the training data four times. It took about **30** minutes to complete.



The images are much better now. If you have the time, why not try **8** epochs of training, which should take about an hour.

We could continue exploring further refinements but we'll stop here because we've fixed the mode collapse problem, and have managed to get good quality images from the generator.

The code we've just is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/08_gan_mnist.ipynb

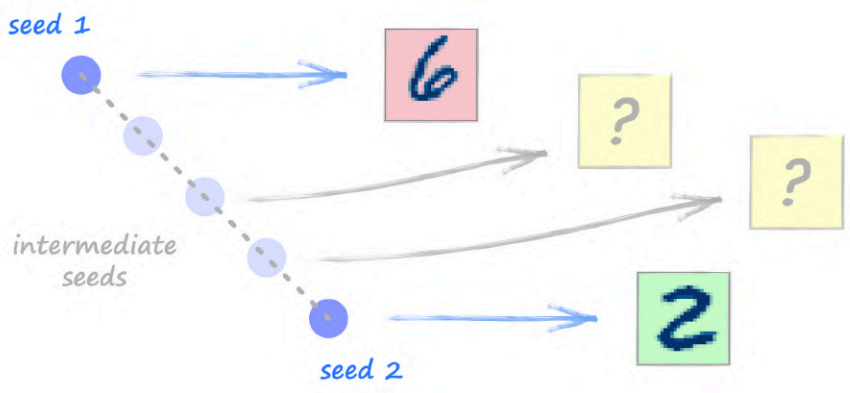
You might be asking whether the magic fix for mode collapse was using **randn()** for the generator seeds. If we went back to our earlier code which used only the vanilla options for the GAN architecture, and changed the seed to use **randn()** we'd find the problem of mode collapse was not fixed. It was fixed through a combination of most, or all, of the refinements. For example, only using **randn()** with a generator seed of size **100** doesn't fix mode collapse. Try it yourself if you're not convinced.

You might also be wondering why we're satisfied with not achieving that balance between the generator and discriminator that we saw with the simple **1010** GAN. Our loss charts show that the discriminator loss rapidly fell close to zero and stayed there, and the generator loss stayed high. In many real life GAN scenarios, we don't achieve this balance, but still have a generator that produces good-enough images. And that's the overall aim, the creation of images that look like they could be real. If there's anything we can do to improve that balance, we should certainly try. We'll keep producing the loss charts because they give us a feel for how the training actually went. For example, our MNIST loss charts tell us the training wasn't chaotic and unstable.

Experimenting with Seeds

So far, we've thought of the seed that feeds the generator as just a random number. After a GAN has been trained, that seed can gain some interesting properties. Let's see for ourselves.

Imagine two different seeds, **seed1** and **seed2**. We can generate images from both of these seeds. Now imagine a seed that was half way between **seed1** and **seed2**. What would the generated image look like? In fact, what would images look like for seeds at regular intervals between **seed1** and **seed2**?



Let's try it. We'll need a GAN already trained on the MNIST data. It's easiest to continue from our previous notebook.

The following code picks a random seed and keeps it as **seed1** so we can use it later. It also plots the image generated from that seed.

```
seed1 = generate_random_seed(100)
out1 = G.forward(seed1)
img1 = out1.detach().numpy().reshape(28,28)
plt.imshow(img1, interpolation='none', cmap='Blues')
```

The following code does the same but for a separate seed **seed2**.

```
seed2 = generate_random_seed(100)
out2 = G.forward(seed2)
img2 = out2.detach().numpy().reshape(28,28)
plt.imshow(img2, interpolation='none', cmap='Blues')
```

Not every image from the generator is a clear digit, so repeat the code until you get a distinct digit.

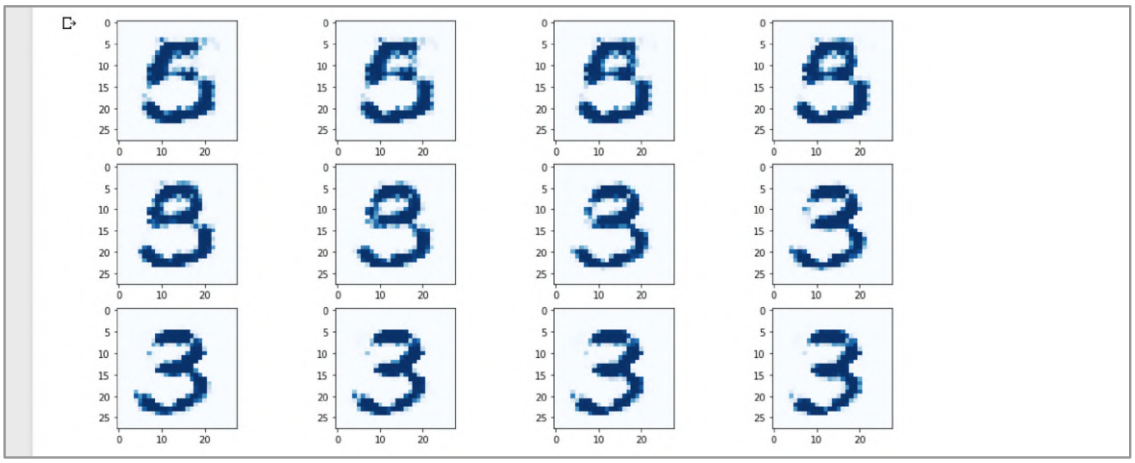
For my own experiment, the following image is generated from **seed1**. It looks like a **5**.

And similarly, the following image is generated from **seed2**. It looks like a **3**.

Let's now write some code that calculates **12** seeds that are evenly spaced between **seed1** and **seed2**.

```
count = 0

# plot a 3 column, 2 row array of generated images
f, axarr = plt.subplots(3,4, figsize=(16,8))
for i in range(3):
    for j in range(4):
        seed = seed1 + (seed2 - seed1)/11 * count
        output = G.forward(seed)
        img = output.detach().numpy().reshape(28,28)
        axarr[i,j].imshow(img, interpolation='none', cmap='Blues')
        count = count + 1
        pass
    pass
```

The code might look complicated, but all it is doing is taking **12** steps from **seed1** in the direction of **seed2**, and plotting the image generated from that seed.



We can see that the image of a **5** evolves into a **3**, more or less smoothly as the seed changes from **seed1** to **seed2**. This is a nice property of the seed values.

Let's try another experiment. What image do we get from adding the seeds?

```
seed3 = seed1 + seed2
out3 = G.forward(seed3)
img3 = out3.detach().numpy().reshape(28,28)
plt.imshow(img3, interpolation='none', cmap='Blues')
```

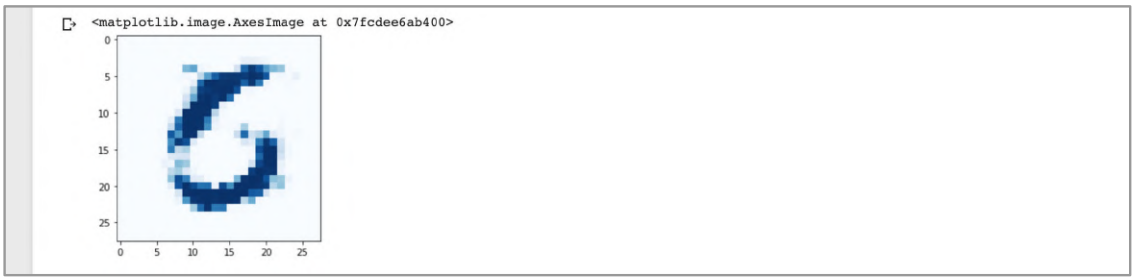
The code is simple to follow. A new **seed3** is created from **seed1 + seed2**, and this is used to feed the generator.

The result looks almost like an **8**. This makes some sense as that's the form we'd get if we overlaid a **5** with a **3**. Again, this suggests a really nice property of seeds, that adding them adds their images too.

We've seen the effect of adding seeds. Let's see what happens if we subtract them.

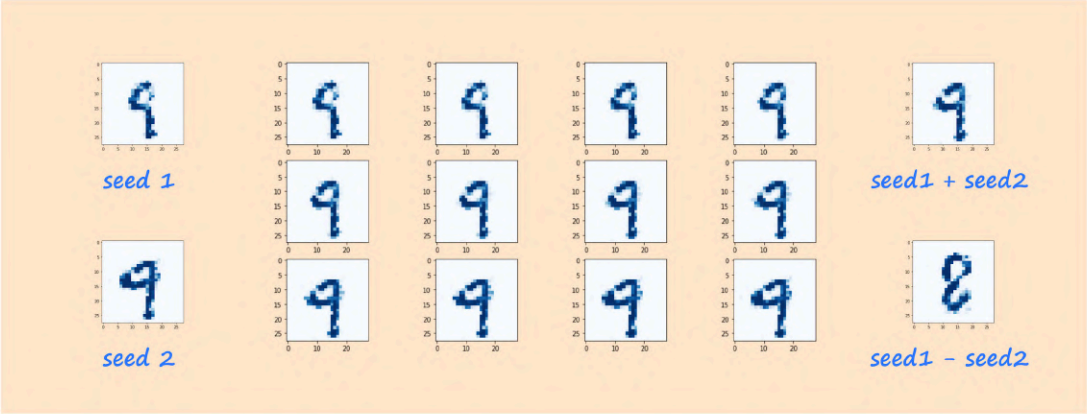
```
seed4 = seed1 - seed2
out4 = G.forward(seed4)
img4 = out4.detach().numpy().reshape(28,28)
plt.imshow(img4, interpolation='none', cmap='Blues')
```

The difference between **seed1** and **seed2** is fed to the generator.



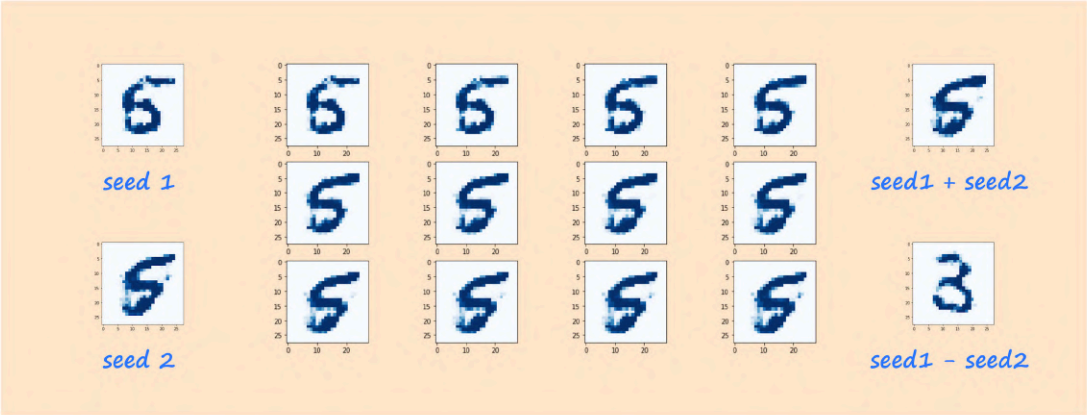
The image still looks like a **5** but could also be a **6**. This isn't quite as logical as subtracting the strokes of a **3** from a **5**, so perhaps the properties of seeds are not so simple.

Let's try another example. The following chart shows the images from the starting seeds, the interpolated seeds, and also the sum and difference of the seeds.



We can see the two starting seeds both give us similar images of a **9**. Seeds interpolating between them also give images of a **9**. It is no surprise the image generated from the sum of the seeds is also a **9**. What is surprising is that the difference between the seeds gives us an **8**. How odd!

Here's another example where two seeds give very similar images of a **5**, but the difference gives us a very different image of a **3**.



This suggests that doing arithmetic with seeds is not quite as simple as we thought.

Try these experiments yourself. Is there a logic that decides how the image will look when we add or subtract seeds?

A notebook with this additional code for experimenting with seeds is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/09_gan_mnist_seed_experiments.ipynb

Learning Points

- Working with **monochrome images** doesn't need a change in neural network design. The 2-dimensional array of pixel values are simply unrolled or reshaped into a 1-dimensional list to be fed to the input layer of the discriminator. How this is done isn't important, but **consistency** is.
- **Mode collapse** is when a generator only produces output of one class when several are possible. Mode collapse is one the most common challenges for GAN training. The causes, and remedies, are still not well understood, and are an active area of research.
- A good starting point for designing GANs is to **mirror** the generator and discriminator neural network architectures with the aim of balancing them so one doesn't get too far ahead of the other during training.
- Experimental evidence suggests the **quality**, and not just the **quantity**, is key to successful GAN training.
- Smoothly interpolating between generator **seeds** leads to smoothly interpolated images. Summing seeds seems to correspond to additively combining image features. Images from subtracted seeds don't follow an intuitive pattern.
- The theoretical ideal MSE loss for a perfectly trained GAN is **0.25**. For BCE loss it is **ln(2)** or about **0.69**.

Human Faces

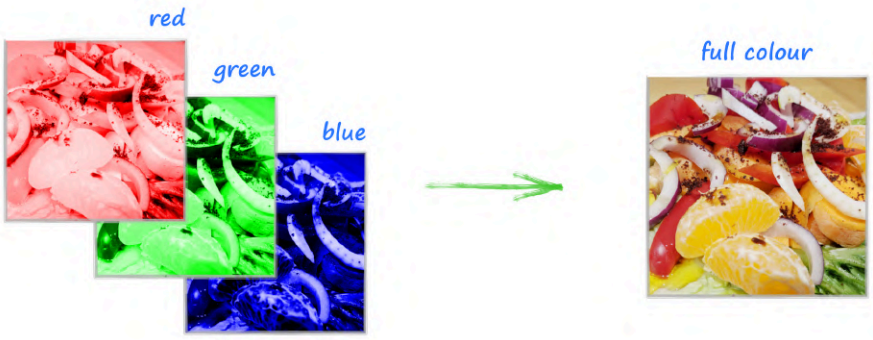
In this section we'll try to train a GAN to generate images of human faces. We'll be taking on two new challenges compared to generating monochrome handwritten digits. We'll be

- training with full colour images, and learning to generate full colour images.
- using a training dataset of photographs which will be much more diverse and contain much more, potentially distracting, detail.

Colour Images

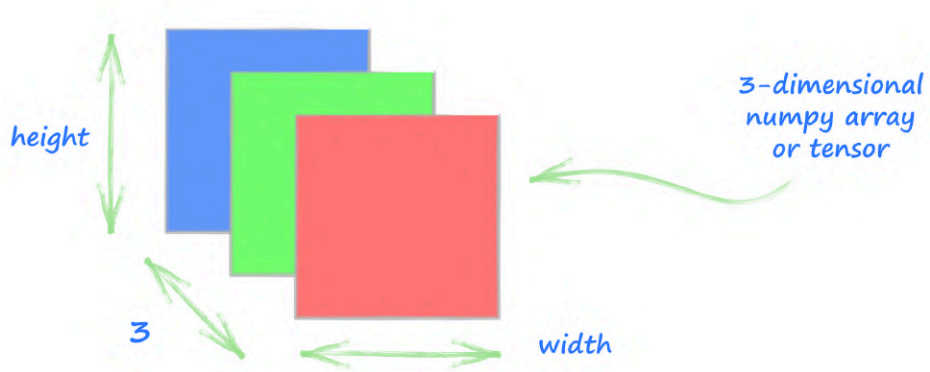
There are many options for representing colour in digital images. The most popular one describes colour as a mixture of **red**, **green** and **blue** light. You have probably used this mixing when picking colours in an image or photo editor. Almost all digital displays, like a laptop or smartphone screen, have pixels made of tiny red, green and blue lights.

The following shows how these red, green and blue layers combine to make a full colour image of a delicious salad.



That picture is helpful because it suggests we can keep the red, green and blue colour information in separate arrays of the same size. We know a monochrome image of size **28** by **28** can be represented by an array of size **28** by **28**. A colour image of the same size can be represented by **3** separate arrays, each of size **28** by **28**. One will contain the red light values, another the green values, and the third will contain the blue values.

The following picture shows a colour image represented by a 3-dimensional array or tensor. One dimension is the image **height**, another is the image **width**. The third dimension is always **3** because there are **3** layers, the red, green and blue values.



It's handy that many python libraries also use this data format for colour images, including matplotlib and its **imshow()** function.

CelebA Dataset

A real challenge for training GANs is ensuring there is enough training data. We can't train a GAN to generate human faces with just tens, or even hundreds of faces. Luckily, there is a popular dataset we can use called the **CelebA** dataset, which contains **202,599** photos of celebrity faces, aligned and cropped so the eyes and mouth are roughly centred.

You can read more about the CelebA dataset and see sample images at its home:

- http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html

The dataset is intended to be used only for non-commercial research and educational use.

We won't reproduce any images directly from the database itself, and only show GAN generated images. This is a good excuse to draw cartoon faces and show those instead!

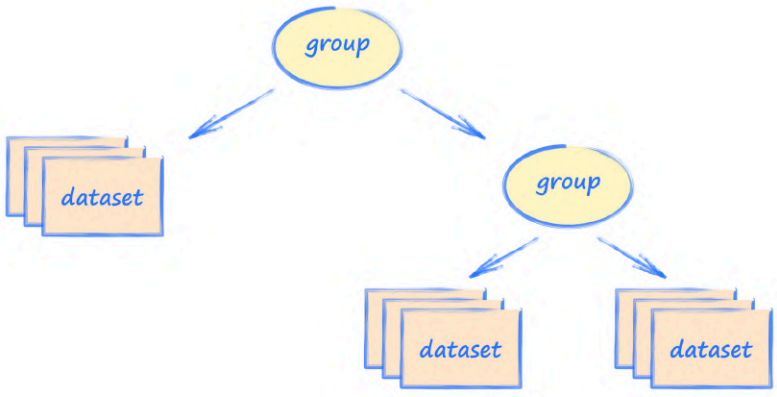
Hierarchical Data Format

The CelebA dataset contains thousands of separate image files in JPEG format. We could unpack them into a folder and have our GAN code open and close all of these images separately as we worked through the training data. This would work, but would be very slow as opening and closing thousands of image files individually is not very efficient. The problem is made much worse if we use a mounted google Drive because the path between our code and the data itself is even less direct.

To help ease these performance challenges, we can package the data into a format that is designed to support this kind of repeated access more efficiently. We'll use a packaging format called **HDF5**.

The **Hierarchical Data Format** is a mature and open data format designed to store very large amounts of data and enable efficient access to it. It is commonly used in science and engineering applications.

It is called **hierarchical** because a HDF5 package can contain one or more **groups**, and within each group there can be one or many **datasets**, or even more groups. This way of organising data is very much like the folders and files we're already used to.



The HDF5 format, and the libraries used for accessing data in that format, have many features designed to ensure good performance. One of these is compressing the data so less is transferred to and from slow storage. Another one is that the data is intelligently mapped to fast RAM memory which can reduce requests to slow storage. Without these features, working with thousands of image files in google Drive would be impractically slow.

Even if you're using your own storage and not using google Drive, it is a good idea to check whether a format like HDF5 would improve the performance of machine learning tasks which repeatedly access lots of data, particularly if it doesn't all fit in RAM.

Getting The Data

The following notebook contains code to download the CelebA dataset, extract **20,000** images and package them up into a HDF5 file. As we're focussed on learning and experimenting, we won't use the full **202,599** images.

- https://github.com/makeyourownneuralnetwork/gan/blob/master/10_celeba_download_make_hdf5.ipynb

Before you run the code make sure to create a new folder called **celeba_data** next to the **mnist_data** folder and at the same level as your notebooks. In the code, change the HDF5 file path **hdf5_file** to match your own folder names. Your work might be in a folder called **gan** and not **myo_gan** like mine.

Extracting **20,000** images should take about four minutes. The HDF5 file containing these images should be saved into the **celeba_dataset** folder, and take up just under **2 Gb** of storage space. The downloaded files are only stored temporarily in the virtual machine, and will disappear when the colab virtual machine is shut down.

Looking At The Data

Let's learn how to access data in a HDF5 file, and then check our file does contain photos of celebrities.

Create a new notebook and use the first cell to mount the google Drive.

```
from google.colab import drive
drive.mount('./mount')
```

In the next cell import the **h5py** library for working with HDF5 files, as well as numpy and matplotlib like before.

```
# import h5py to access data
import h5py

import numpy
import matplotlib.pyplot as plt
```

The h5py library gives us access into a HDF5 package in a very pythonic way by presenting the hierarchical structure of groups and datasets as dictionary-like objects. If that sounds confusing rather than helpful, we'll see what it means next.

Have a look at the following code.

```
with h5py.File('mount/My Drive/Colab
Notebooks/myo_gan/celeba_dataset/celeba_aligned_small.h5py', 'r') as
file_object:

  for group in file_object:
    print(group)
    pass
```

We use the h5py library to open the HDF5 file in read-only mode using a file object. This looks just like how we normally access files in python using **open()**. We then loop over the file object and print out the names of any groups found at the top of the hierarchy.

It looks like there's only one group **img_align_celeba**. We can access the data inside the file object as if we were using a python dictionary **file_object['img_align_celeba']**. This will give us a **dataset** which contains all the image data.

We can again use this dictionary-style look up to get at individual images, for example **dataset['7.jpg']**. The image data itself will still be in a HDF5 format but it is really easy to convert it into a numpy array. We already know how to plot numpy arrays as images using matplotlib's **imshow()**.

```
with h5py.File('mount/My Drive/Colab
Notebooks/myo_gan/celeba_dataset/celeba_aligned_small.h5py', 'r') as
file_object:
  dataset = file_object['img_align_celeba']
  image = numpy.array(dataset['7.jpg'])
  plt.imshow(image, interpolation='none')
  pass
```

Run the code to check that the image extracted from the HDF5 file is indeed a photo of a person. I've drawn a cartoon over the photo to avoid reproducing images from the dataset.

The image you see is in full colour. Let's examine the shape of the **image** numpy array to see how it contains all the colour information.

As expected, it has a height of **218** pixels, a width of **178** pixels, and has **3** layers for the red, green and blue information.

The image names run from **0.jpg**, **1.jpg**, **2.jpg** all the way up to **19999.jpg**. Have a look at a few of them to get a sense of how diverse the dataset is. You'll see there are faces with different styles of hair, different skin colour, different kinds of clothing and also different poses. You should also see that some photos have unusual angles or contain detail which can distract from the main face, making the task of machine learning a little more challenging.

The code we've just written is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/11_celeba_data.ipynb

Dataset Class

To work with images from the CelebA dataset we'll need to modify the Dataset class we used with the MNIST images.

The changes are fairly simple because we've already seen how to open a HDF5 file and extract images from it. Have a look at this CelebADataset class.

```
class CelebADataset(Dataset):

    def __init__(self, file):
        self.file_object = h5py.File(file, 'r')
        self.dataset = self.file_object['img_align_celeba']
        pass

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, index):
        if (index >= len(self.dataset)):
          raise IndexError()
        img = numpy.array(self.dataset[str(index)+'.jpg'])
        return torch.FloatTensor(img) / 255.0

    def plot_image(self, index):
        plt.imshow(numpy.array(self.dataset[str(index)+'.jpg']),
interpolation='nearest')
        pass

    pass
```

The constructor **__init__()** opens the HDF5 file and also opens the group named **img_align_celeba**, ready for individual images to be accessed later. The **__len__()** method is

also easy to complete because running **len()** on a HDF5 group conveniently gives us the number of data items inside it.

The **__getitem__()** method retrieves the image data by converting the **index** to the name of the image. The numpy array values are divided by **255.0** so they're in the range **0** to **1**.

The extra bit of code in **__getitem__()** checks to see if the **index** is greater than or equal to the size of the dataset, and if it is, an IndexError exception is raised. We've not needed this in our earlier examples because if we tried to access an array or dataframe with an index that was out of bounds, an IndexError was raised anyway. Because we're accessing the HDF5 dataset, an out of bounds request would cause an error but it wouldn't be the IndexError expected by PyTorch.

It's good practice to have a method for seeing the data, and here we have a **plot_image()** function just like we did with the MNIST data. The code is simpler because we don't need to reshape the data into a rectangle, it already has the shape **(218, 178, 3)**.

That wasn't too difficult at all, and this Dataset class is simpler than the MNIST class.

Let's test our Dataset class by creating a dataset object and viewing an image chosen by index.

```
# create Dataset object
celeba_dataset = CelebADataset('mount/My Drive/Colab
Notebooks/myo_gan/celeba_dataset/celeba_aligned_small.h5py')

# check data contains images
celeba_dataset.plot_image(43)
```

Try different values for the index to check you can see different images. Remember the index ranges from **0** to **19,999**.

Discriminator

The discriminator will take an image and try to classify it as real or generated. This is what the MNIST discriminator does so let's copy that code as a starting point.

The only difference between the MNIST images and the CelebA images are their size and colour depth. The size difference just means we need to change the number of input nodes to the discriminator neural network. The MNIST images were **28** by **28** pixels in size so we had **28 * 28 = 784** input nodes. The CelebA images are **218** by **178** pixels in size. That should mean **218 * 178** input nodes.

But we mustn't forget that each colour pixel actually has **3** values, one each for the red, green and blue levels. That means each image has a total of **218 * 178 * 3** values. We need to feed all of these to the discriminator because we don't want to leave any valuable information out. That means the discriminator needs **218 * 178 * 3 = 116,412** input nodes.

A good question to ask is how we should arrange these **116,412** values when we feed them to the discriminator. With the MNIST classifier and discriminator we just fed the values to the network in the order they appeared in the dataset. Those values followed each pixel along a row in the image, and wrapped round to the next row when the end of a row was reached. As long as we are consistent, the ordering doesn't actually matter because the neural network layers are fully connected, every node in one layer is connected to every node in the next. This means there is no benefit to a pixel value being at any particular position in the input tensor.

We can do the same for colour images. We can unroll, or reshape, the **218** by **178** by **3** image tensor to a 1-dimensional tensor of length **116,412**, and feed that to a fully connected neural network. It doesn't really matter how we unroll or reshape, as long as we're consistent and do it the same way every time we feed the discriminator.

Have a look at the following.

```
self.model = nn.Sequential(
    View(218*178*3),

    nn.Linear(3*218*178, 100),
    nn.LeakyReLU(),

    nn.LayerNorm(100),

    nn.Linear(100, 1),
    nn.Sigmoid()
)
```

Most of the code is familiar. We have a linear layer that connects all **3 * 218 * 178** input nodes to a much smaller **100** middle layer nodes. We use a LeakyReLU activation function and a layer normalisation before another linear layer connects these **100** nodes to a single output node. A sigmoid activation is applied to the output of the final node.

There's a new bit of code **View(3*218*178)** at the top of the definition. This reshapes the 3-dimensional image tensor of shape **(218, 178, 3)** to a 1-dimensional tensor of shape **(218*178*3)**.

Being able to reshape tensors in the Sequential list with View is really convenient. PyTorch hasn't provided such a function as a module yet. We can use the following one based on code kindly shared on the PyTorch github issues tracker. Notice how it defines View as a class that inherits from **nn.Module** so it can work like any other module used in a Sequential list.

```
# modified from https://github.com/pytorch/vision/issues/720

class View(nn.Module):
    def __init__(self, shape):
        super().__init__()
        self.shape = shape,

    def forward(self, x):
        return x.view(*self.shape)
```

We'll need to include this code in a separate cell near the top of our notebooks together with our other helper functions like the random seed and image generators.

Testing The Discriminator

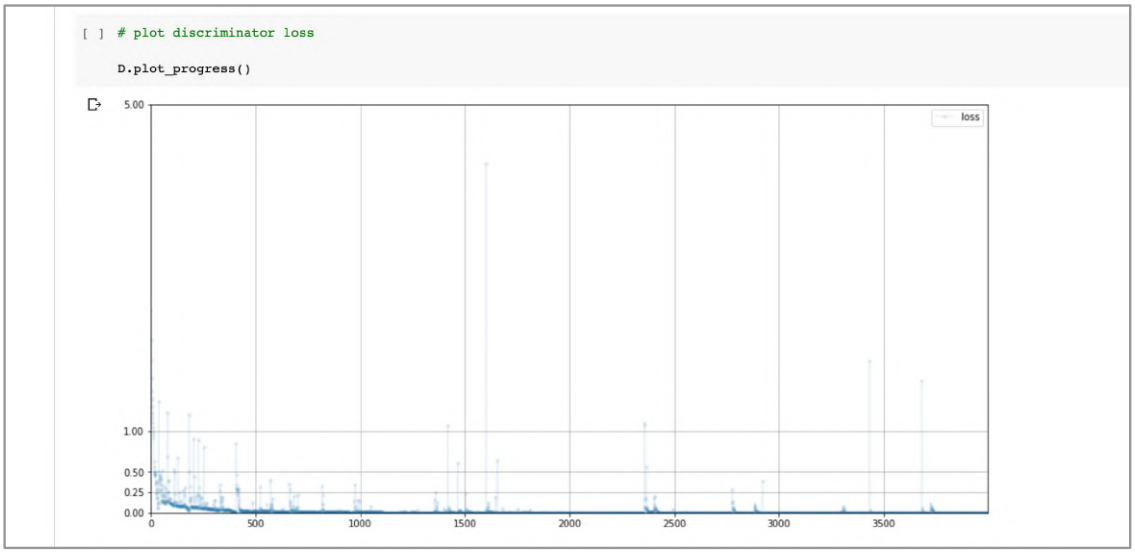
Let's test the discriminator to make sure our network can at least separate real images from images with random pixel values.

```
%%time
# test discriminator can separate real data from random noise

D = Discriminator()

for image_data_tensor in celeba_dataset:
    # real data
    D.train(image_data_tensor, torch.FloatTensor([1.0]))
    # fake data
    D.train(generate_random_image((218,178,3)),
torch.FloatTensor([0.0]))
```

```
    pass
```

The code is simpler than our MNIST code because we're not having to deal with labels and target values.

Plotting the discriminator loss values shows the network does indeed learn to separate real from random images.



If you run this test yourself, you'll find it takes a very long time. My own test took **1** hour and **19** minutes to work through **20,000** images. The MNIST dataset has **60,000** images and took about **4** minutes. The reason for this difference is that the CelebA images are much larger at **116,412** pixel values compared with MNIST images which have **784** values. That's about **150** times larger.

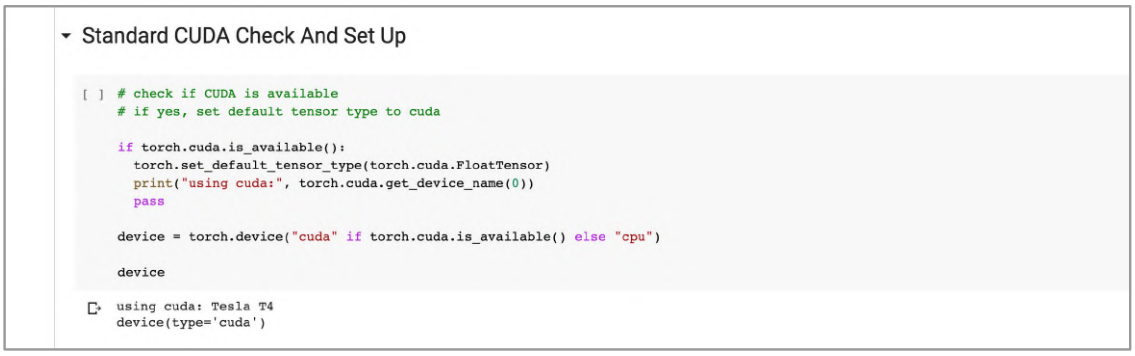
This is a good point to adapt our code to take advantage of GPU acceleration.

GPU Acceleration

We've already seen how to move our tensors to the GPU and perform calculations there. Here we'll be adapting our code to do as many of the calculations on the GPU.

First we'll need to change the runtime type so the colab virtual machine is attached to a GPU. From the notebook menu, under **Runtime**, choose **Change runtime type**, and select **GPU**. This will reset the virtual machine so we'll have to run the code again from the top.

After the cells which mount the google Drive and import the modules, in a separate cell we'll run our standard test to see if CUDA is available, and if it is, set the default float tensor type to be **torch.cuda.FloatTensor**.



Looks like I was lucky enough to get a Tesla T4 GPU this time.

Next we'll need to modify our CelebADataset class so the **__getitem__()** function returns a CUDA tensor. Only the return line needs to be changed.

```
return torch.cuda.FloatTensor(img) / 255.0
```

We don't need to update our **generate_random_image()** and **generate_random_seed()** functions because the default float is now a CUDA tensor. Let's check to see for ourselves.

The Discriminator class doesn't need any changes, but we do have some small changes to the discriminator test training loop.

```
D = Discriminator()
# move model to cuda device
D.to(device)

for image_data_tensor in celeba_dataset:
    # real data
    D.train(image_data_tensor, torch.cuda.FloatTensor([1.0]))
    # fake data
    D.train(generate_random_image((218,178,3)),
torch.cuda.FloatTensor([0.0]))
```

```
        pass
```

The first change is that the discriminator object **D**, which includes the neural network, is moved to the GPU. This is important because we want the neural network training to be done on the GPU. The other changes are using **torch.cuda.FloatTensor** for the target values.

If we now run the discriminator test again, it should be much quicker. My test took **3** minutes and **48** seconds. That's about **20** times faster!

```
counter =  37000
counter =  38000
counter =  39000
counter =  40000
CPU times: user 2min 58s, sys: 45 s, total: 3min 43s
Wall time: 3min 48s
```

This really shows how GPUs can make working with neural networks much more practical.

Check the loss graph still falls towards zero in the same way as before. Using the GPU should not change our neural network calculations in any significant way.

Generator

The generator also needs to be changed because we're now generating not just a larger image, but also a colour image. This means the output needs to be a 3-dimensional tensor of size **(218, 178, 3)**.

Have a look at the following code for the generator's neural network.

```
self.model = nn.Sequential(
    nn.Linear(100, 3*10*10),
    nn.LeakyReLU(),

    nn.LayerNorm(3*10*10),

    nn.Linear(3*10*10, 3*218*178),

    nn.Sigmoid(),
    View((218,178,3))
)
```

The input is the simple seed of size **100** we used before. It is fully connected to a middle layer with **3*10*10 = 300** nodes. That middle layer is then fully connected to a final output layer which has **3*218*178** nodes. The very last step is to reshape the 1-dimensional tensor of size **3*218*178** to a 3-dimensional tensor of shape **(218, 178, 3)** to look like a colour image.

Check Generator Output

As usual it is good practice to check the generator produces output of the right shape, and without error, before we get too involved in training it.

```
G = Generator()
# move model to cuda device
G.to(device)

output = G.forward(generate_random_seed(100))
img = output.detach().cpu().numpy()
plt.imshow(img, interpolation='none', cmap='Blues')
```

The code is very familiar now. We create a new generator object and move it to the GPU. We feed the generator a random seed and generate an **output**. Before we can display that **output** as an image, we need to **detach()** it from PyTorch's computation graph, move it from the GPU to the CPU, and then convert it to a numpy array.

We can see the output is of the right shape and indeed does contain random colour. If the image was dominated by a particular colour or contained definite patterns, we'd need to look for errors in our code. An untrained generator should generate random data.

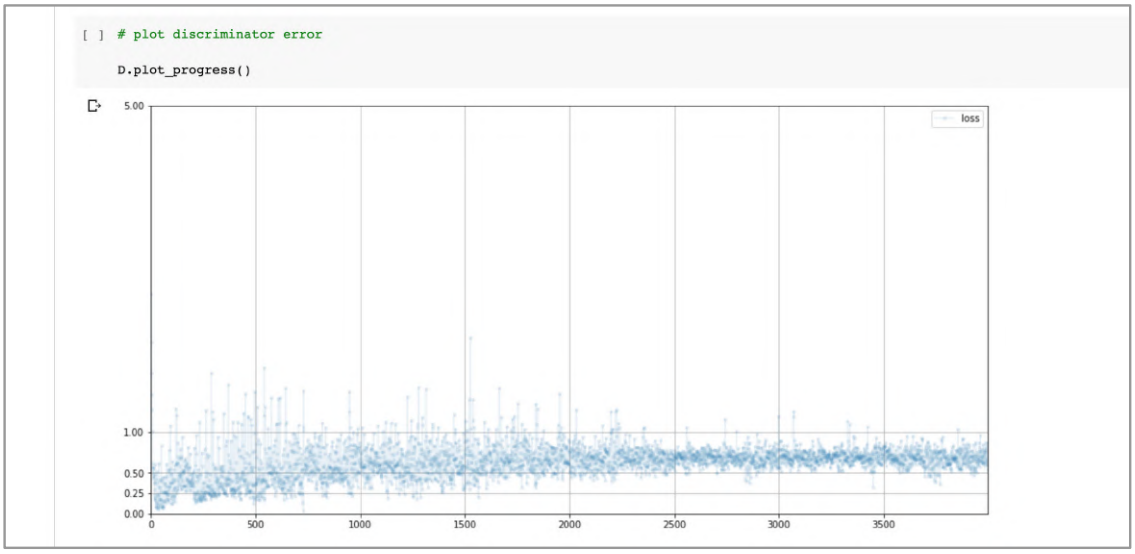
Training The GAN

We're finally ready to train the GAN. The training loop code has the same structure as before, with minor changes to move the discriminator and generator to the GPU and also ensure the target values are of type **torch.cuda.FloatTensor**.

```
%%time

# create Discriminator and Generator

D = Discriminator()
D.to(device)
G = Generator()
G.to(device)

epochs = 1

for epoch in range(epochs):
  print ("epoch = ", epoch + 1)

  # train Discriminator and Generator

  for image_data_tensor in celeba_dataset:
    # train discriminator on true
    D.train(image_data_tensor, torch.cuda.FloatTensor([1.0]))

    # train discriminator on false
    # use detach() so gradients in G are not calculated
    D.train(G.forward(generate_random_seed(100)).detach(),
torch.cuda.FloatTensor([0.0]))

    # train generator
    G.train(D, generate_random_seed(100),
torch.cuda.FloatTensor([1.0]))

    pass

  pass
```

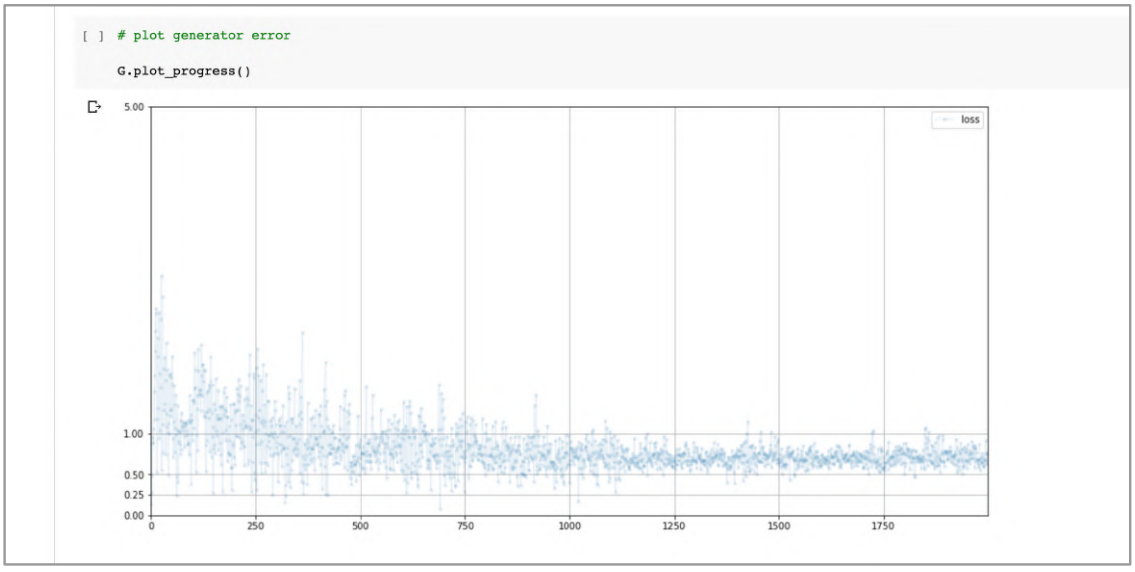
Nothing else needs to change which tells us we have written a very reusable training loop. Running the training loop for one epoch takes me just under **10** minutes. Without GPU acceleration that might have taken three hours!

Let’s look at the loss charts. The following is the discriminator loss.



This looks good because we can see the training is not unstable and chaotic. The loss values are approximately converging to a value that isn’t too large.

Let’s now look at the generator loss.



Again, this looks good. The training seems to be fairly stable and the loss values are converging to a similar value to the discriminator loss.

In fact the loss values both seem to be approaching the ideal loss for binary cross entropy **ln(2) = 0.69** as discussed in Appendix A.

The signs are really positive!

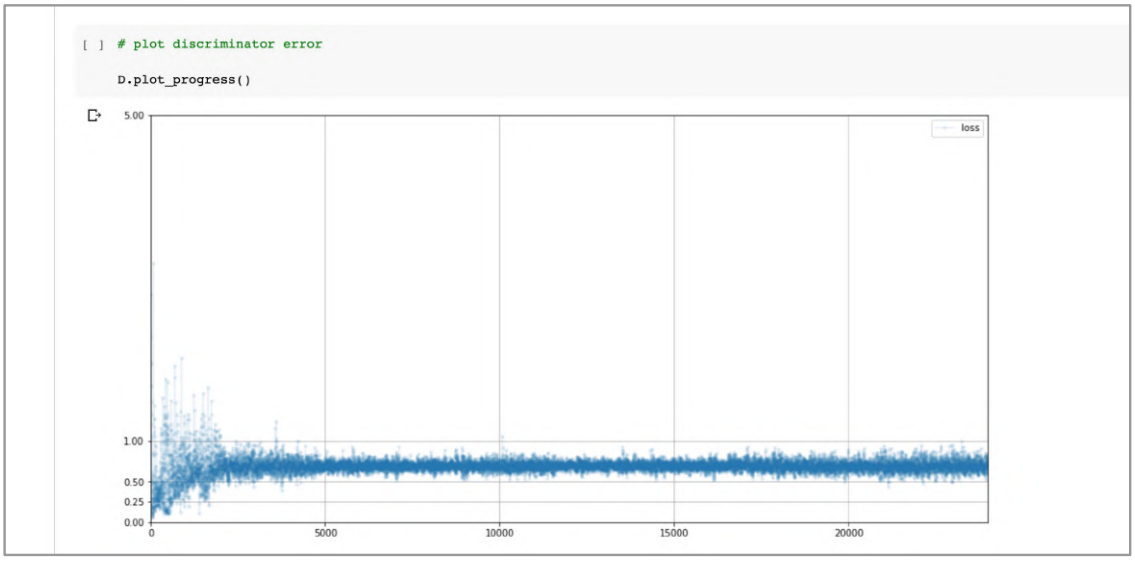
Let's have a look at the images that emerge from the trained generator. The following code plots a **3** by **2** grid of **6** images, each separately output from the generator.

```
# plot a 3 column, 2 row array of generated images
f, axarr = plt.subplots(2,3, figsize=(16,8))
for i in range(2):
    for j in range(3):
        output = G.forward(generate_random_seed(100))
        img = output.detach().cpu().numpy()
        axarr[i,j].imshow(img, interpolation='none', cmap='Blues')
        pass
    pass
```

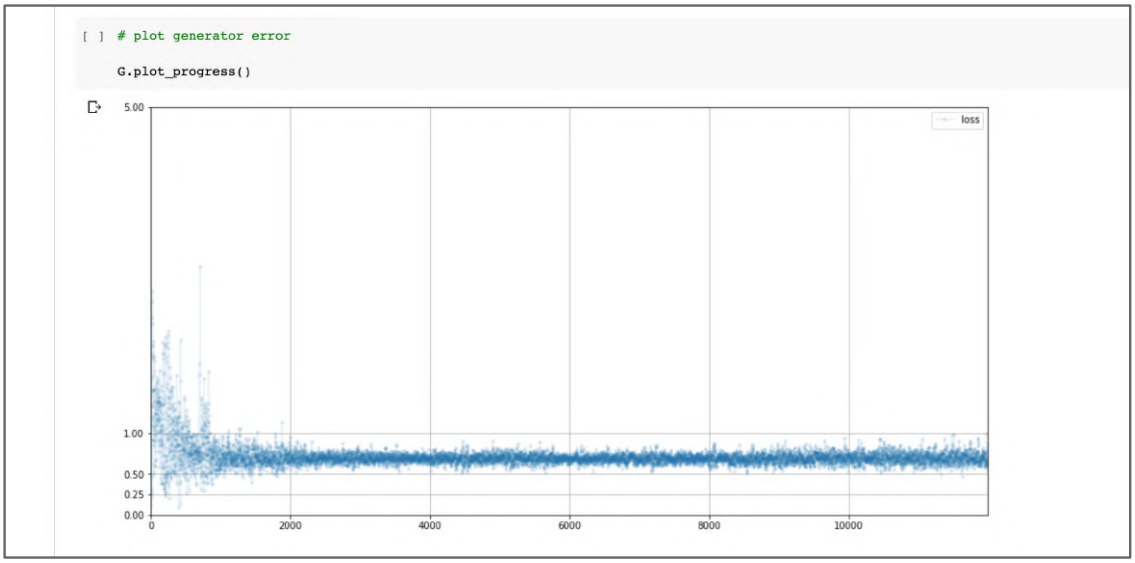
The reason for showing several generated images together is so we can get a sense of how diverse the generated images are. The following picture shows the results from the above code run twice so we have **12** images in total.

Yippee! We have faces. Not only do we have faces, the faces are quite diverse. Our generator has learned to create portraits of people with different hair styles, different face shapes and even slightly different poses. Sure, there are some images which are poor, but such good results from very simple code is surprising, and a real achievement. Well done!

Let's train the generator for a longer **6** epochs to see if we can improve image quality. For me **6** epochs took **58** minutes. The following chart shows the discriminator loss.

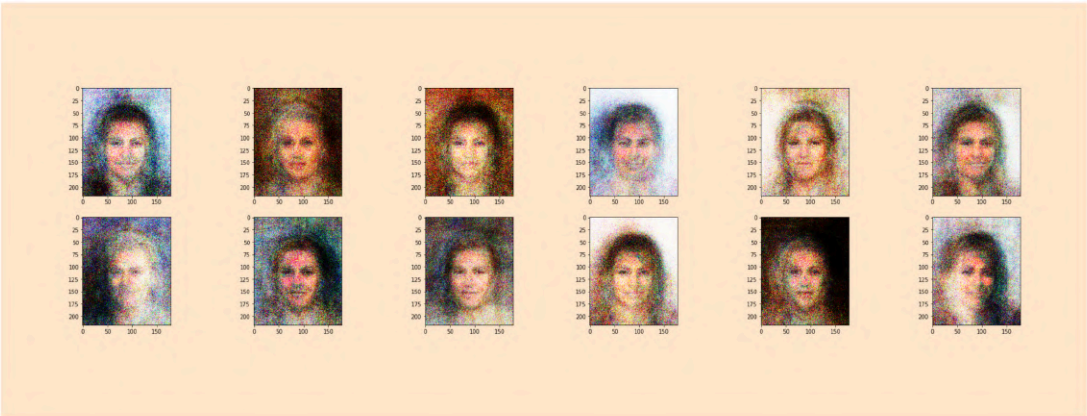


The loss has stayed fairly stable around the theoretical ideal value, which is a good sign, and tells us training hasn't become unstable. Let's also look at the generator loss.



Again, the loss has remained stable around the ideal value **ln(2) = 0.693**.

Let's have a look at **12** images from this generator trained for **6** epochs.



The images have improved a little. The facial features and details are more distinct and the incorrect green or blue faces are now rare. We are also starting to see faces which are distinctly male or female. The hair styles in particular remain diverse.

Try your own experiments, perhaps using larger neural networks, or using different optimisation methods and loss functions to see if you can improve the image quality further.

You should pat yourself on the back. We've done a lot of work, and with a fairly small amount of code, succeeded in training a neural network to generate portraits that all look different. That's not a small achievement!

These diverse portraits prompt a good question - what does the generator actually learn? We've already said the generator does not memorise entire faces or smaller features like eyes and noses from the training data. What the generator has learned is to create images that match the **likelihood** of features seen in the training data. You can read more about this in Appendix B.

The code we've developed is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/12_gan_celeba.ipynb

Learning Points

- **Colour** is often modelled with **red**, **green** and **blue** light values. Colour images are often represented by arrays with **3** layers of pixel values, one for each red, green and blue channel, and have shape **[height, width, 3]**.

- When working with datasets consisting of many individual files, opening and closing each file is inefficient, particularly in virtual environments. It is good practice to repackage the data in a format designed for frequent and random access to lots of data. The mature **HDF** format is common in scientific computing.

- A GAN doesn't memorise examples from the training data. It also doesn't copy and paste elements from training examples. It learns the **probability distribution** of features in the training data and generates data which looks like it could have come from that distribution.

Part 3

We extend the core GAN concept, applying it to convolutional neural networks, and developing a conditional GAN for generating data of a desired class.

Convolutional GANs

In this section we're going to improve on the CelebA GAN we just developed for two reasons.

- The images it generated look fuzzy. The areas we expect to be fairly smoothly coloured are covered in a high-contrast pattern of pixels.

- Fully connected neural networks consume a lot of memory. Even moderately larger images or networks will soon hit the limit of our GPU and prevent training. Most consumer GPUs will have much smaller memory than the Tesla T4 or P100 provided by Google's colab service.

Memory Consumption

Before we explore a new GAN technique, let's check how much memory our GAN consumes. If we run the entire notebook again, the discriminator and generator networks will have consumed memory. To be more precise, the input data, and information flowing through the networks, the outputs, as well as the learnable parameters, will all have been tensors that have required memory on the GPU.

We can check how much memory is currently allocated using the following code.

```
# current memory allocated to tensors
torch.cuda.memory_allocated(device) / (1024*1024*1024)
```

We divide by **1024*1024*1024** to convert from **bytes** to **gigabytes**.

We can see that after the all the notebook code has run, about **0.70 gigabytes** of GPU memory remains allocated to tensors. This is mostly because the discriminator and generator objects still exist, ready to be used again if needed.

That number doesn't paint the complete picture because some memory will have been released after the GAN code has run. The following slightly different code tells us the peak memory consumed by tensors during the run.

```
# total memory allocated to tensors during program (in Gb)
torch.cuda.max_memory_allocated(device) / (1024*1024*1024)
```

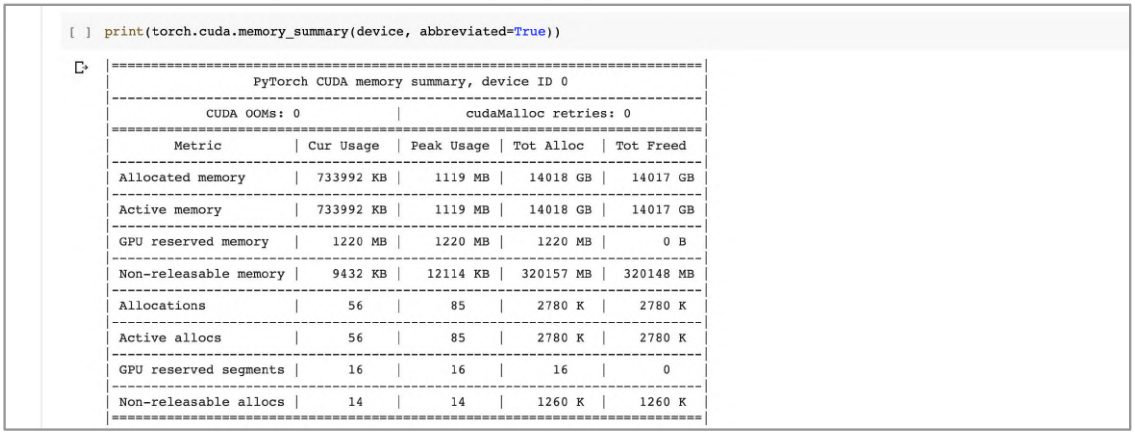
This peak consumption should be a larger number.

We can see that the peak memory used by tensors during the running of our code is a larger **1.09 gigabytes**.

If you're interested in more statistics on memory you can use the following code.

```
# summary of memory consumption
print(torch.cuda.memory_summary(device, abbreviated=True))
```

Amongst the information is the current and peak usage which we've just talked about.



The **733992** kilobytes is the same **0.70 gigabytes** of current consumption, and the **1119** megabytes is the same **1.09 gigabytes** of peak usage.

These numbers will be a good baseline to see if our improved method does indeed require less memory.

Localised Image Features

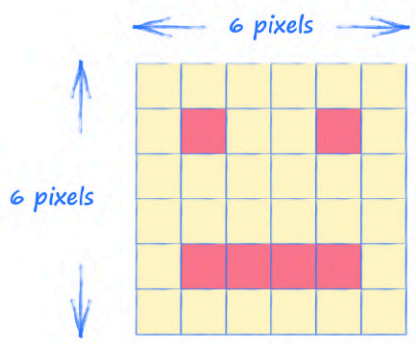
A golden rule of machine learning is to make maximum use of any knowledge about the problem we're trying to solve. This **domain knowledge**, can reduce the size of the problem by excluding possibilities that we know aren't valid. This makes machine learning easier because the space of learnable parameters in which we have to find a good combination is smaller.

If we think a little deeper about images, we realise that meaningful **features** are **localised** in the image. For example, the pixels representing an eye or a nose are close together. That's the information which can be useful in classifying that image as a face. We should be able to take advantage of this locality by designing a classifier neural network that looks more narrowly at groups of neighbouring pixels.

Our previous MNIST and CelebA classifiers don't do this. They consider all the pixels of an image together. That's not incorrect, and we know that those networks can learn the right link weights to pick out the right features to help classify the images. By not taking advantage of the locality of image features, we're just making the learning task harder.

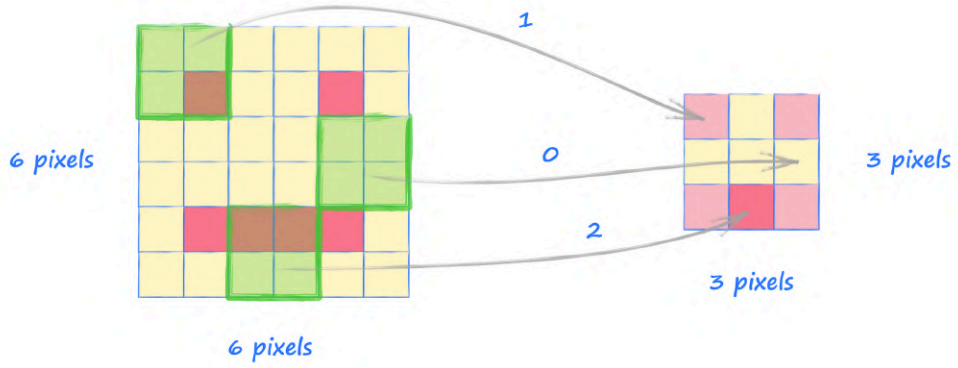
Convolution Filters

Have a look at the following **6** by **6** pixel image of a face.



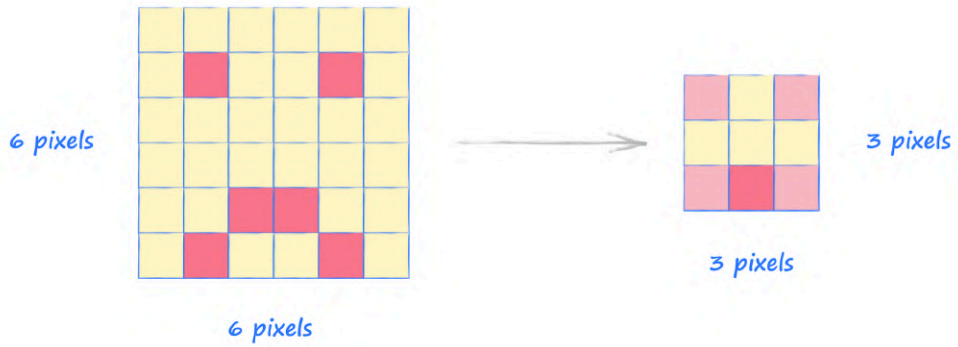
Imagine we had a magnifying class that could only see a small **4** by **4** pixel portion of any image we pointed it at. We could move it over the image and see an eye or part of a smile. This magnifying glass is giving us the **locality** we just talked about.

If we moved over the image and counted how many dark pixels we saw in that **4** by **4** view, we would create a new grid which summarises that local information. The following picture shows this process.



That smaller grid is **3** by **3** pixels in size and summarises what the magnifying glass found in each region of the image. We can see it found an eye in the top left and top right of the image. It also found dark pixels in the bottom, with the bottom middle being particularly dense. It also found that the middle left and middle right don't have dark pixels.

We can see how that summary grid can, in a basic way, be useful in helping to classify whether an image is a face or not. Have a look at the following picture which shows the process applied to a slightly different face.



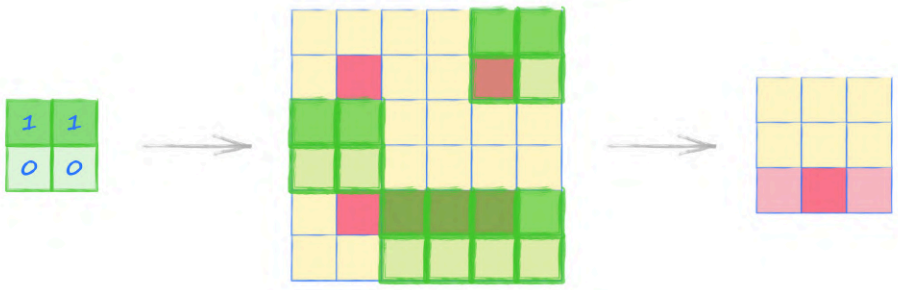
That different face has resulted in the same summary grid. This process is looking quite powerful because it identifies local features, and is also robust to small changes in the image.

This moving over an image and summarising it into a new grid is called **convolution**. The word comes from a mathematical process which finds the similarity between the shape of two

functions or signals. Here we're finding the similarity between the magnifying glass view and the image pixels in that view.

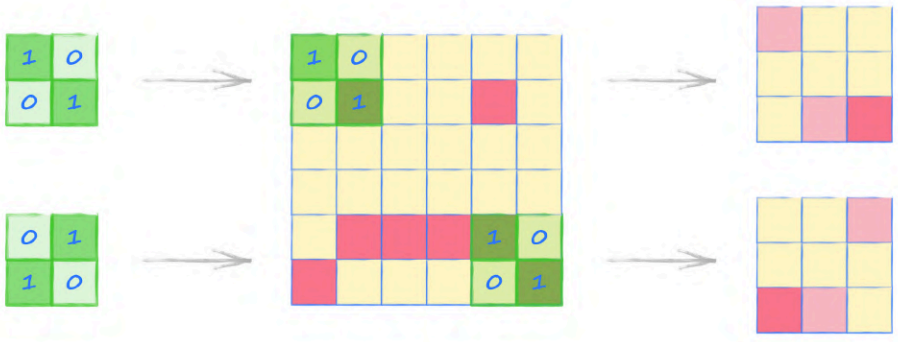
This process can be more sophisticated. If our magnifying glass was biased and gave higher scores to some pixels, and lower scores to others, it can be used to pick out specific patterns.

The following shows a magnifying glass which counts the top two pixels in its view, but ignores the lower two pixels. It does this by multiplying the top pixel values by **1**, and the bottom pixel values by **0**. By doing this, it should pick out horizontal lines in the image being examined.



The resulting summary grid only shows a strong value in the middle bottom, which corresponds to where there is indeed a horizontal line of dark pixels in the face image.

The proper name for that magnifying glass is **convolution kernel**. The picture below shows two convolution kernels being applied to a slightly different image. The kernels are biased in a diagonal direction.



We can see the kernels have picked out areas of the image which have diagonal features that match their directions.

You can find worked examples of convolutions in Appendix C.

Learning Kernel Weights

A good question to ask is how these convolution kernels are useful when we're trying to train a classifier. We've seen how different kernels can pick out different patterns in an image, and this information should be very helpful in classifying that image. For example, a horizontal dark feature in the bottom middle together with dark features near the top left and right suggest the image is a face.

We could have a selection of different kernels and learn the importance of each one, a bit like a link weight from the image to each kernel.

A better approach is to not have a selection of previously designed kernels, but instead learn what the best scores, or weights, inside the kernels should be. This is the approach taken by many machine learning frameworks, including PyTorch.

In essence, we decide how many kernels we want to have, say **20**. During training, the weights inside each of these **20** kernels are adjusted. If training is successful, we end up with kernels which pick out the most useful details from the images. The rest of the neural network will combine this information to classify the image. Not all kernels will be useful, and a low link weight will reduce the impact of these kernels.

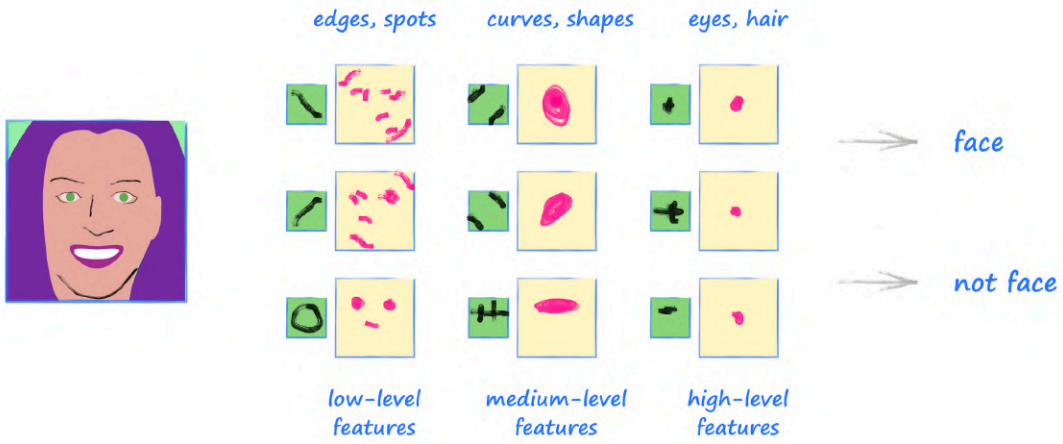
Hierarchy of Features

We've just talked about how a layer of convolution kernels can pick out low-level features like edges or spots to give us grids of summary information, properly called **feature maps**.

If we apply another layer of convolution kernels to these feature maps, we can find higher level features which are a combination of some of those low-level features. For example, the right combination of spots and edges could be an eye or a nose.

We can apply yet another layer of convolution kernels to find even higher-level features that are a combination of those medium-level features. The right combination and orientation of eyes and nose features could be a face.

The following picture shows a hierarchy of convolution layers finding low-level, medium-level and high-level features. The contents of the kernels and feature maps are just for illustration.



Scientific opinion on how human brains understand what we see with our eyes is unsettled and divided, but many do believe it is similar to this kind of hierarchical analysis.

In any case, it is clear that this approach of building up the contents of an image from medium-level features, themselves derived from small patterns of neighbouring pixels, can make the task of image classification more efficient. In fact, **convolutional neural networks**, or CNNs, have long been the state of the art in image classification.

The key point about CNNs is that we let the network learn what the kernels should be, rather than specify them ourselves. In effect, we let the network find the most useful low, medium and high level image features itself.

MNIST CNN

To build our familiarity with convolutions in a neural network, let's make an MNIST classifier before we try making a GAN that uses them.

Start by making a copy of our previous MNIST classifier which is online at:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/04_mnist_classifier_refinements.ipynb

We'll only need to change the definition of the classifier neural network. The rest of the code for loading the data, viewing images, training the network, and checking classification performance shouldn't need to change much.

That neural network had an input layer of **784** nodes, fully connected to a middle layer of **200** nodes, which itself was fully connected to an output layer of **10** nodes. The middle layer had a

LeakyReLU activation and a layer normalisation applied after it. The output layer simply had a sigmoid activation applied. That network achieved a really good **97%** accuracy on the MNIST test data.

We now need to think about how to replace this with convolution filters. The first thing that strikes us is that convolutions work on 2-dimensional images, whereas we were feeding this network a simple 1-dimensional list of pixel values. A quick and easy fix is to reshape the **image_data_tensor** to have shape **(28, 28)** whenever we pass it to the network.

What we actually have to do is use a 4-dimensional tensor because PyTorch's convolution filters expect data tensors to have 4 elements **(batch size, channels, height, width)**. We're using a batch size of **1**, and the MNIST images are monochrome so only have **1** channel, so our MNIST data needs to be shaped as **(1, 1, 28, 28)**. We can easily do this using the **view()** function.

Have a look at the following code for a convolutional neural network.

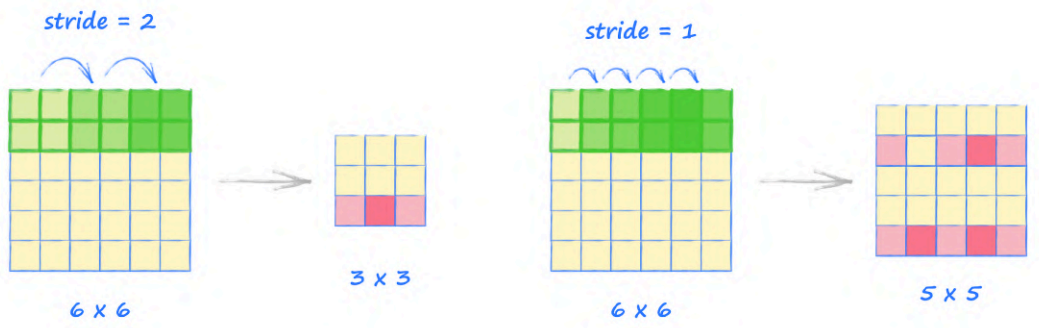
```
self.model = nn.Sequential(
    # expand 1 to 10 filters
    nn.Conv2d(1, 10, kernel_size=5, stride=2),
    nn.LeakyReLU(0.02),
    nn.BatchNorm2d(10),

    # 10 filters to 10 filters
    nn.Conv2d(10, 10, kernel_size=3, stride=2),
    nn.LeakyReLU(0.02),
    nn.BatchNorm2d(10),

    View(250),
    nn.Linear(250, 10),
    nn.Sigmoid()
)
```

The first element of this neural network is a convolution layer **nn.Conv2d**. The first parameter is the number of input channels, which is **1** for our monochrome images. The second parameter is the number of output channels. What this means is that we've created **10** convolution kernels from which will emerge **10** feature maps.

The next parameter **kernel_size** is the size of the kernel. In our earlier discussion we used a **2** by **2** square kernel. Here we set the kernel to be a **5** by **5** square.

The last parameter **stride**, sets the step size as the kernel moves along the image. In our discussion above, we had our **2** by **2** kernel move in steps of size **2**. Here we'll also set the stride to be **2**.

The following picture illustrates how the stride works, showing an example with stride **2** and another with stride **1**. Notice how with stride **1**, the areas covered by the kernels do overlap, and that's fine.



With an MNIST image of size **28** by **28**, a convolution **5** by **5** kernel with stride **2**, results in an output feature map of size **12** by **12**.

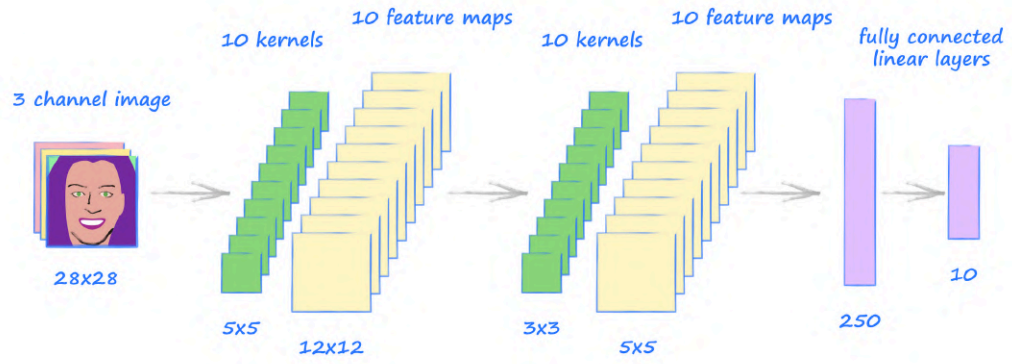
As usual with the output of a layer, we need a non-linear activation function. We can stay with **LeakyReLU(0.02)** which we've seen work well before.

We then apply a normalisation to these feature maps. We don't use **LayerNorm()** but **BatchNorm2d()** which normalises over each channel in the layer.

The next bit of code is similar. It is another convolution layer followed by a normalisation and a non-linear activation function. This time we take the **10** feature maps and apply a kernel to each one to get **10** new feature maps. These kernels are smaller at size **3**. With the same stride of **2**, we get output feature maps of size **5** by **5**.

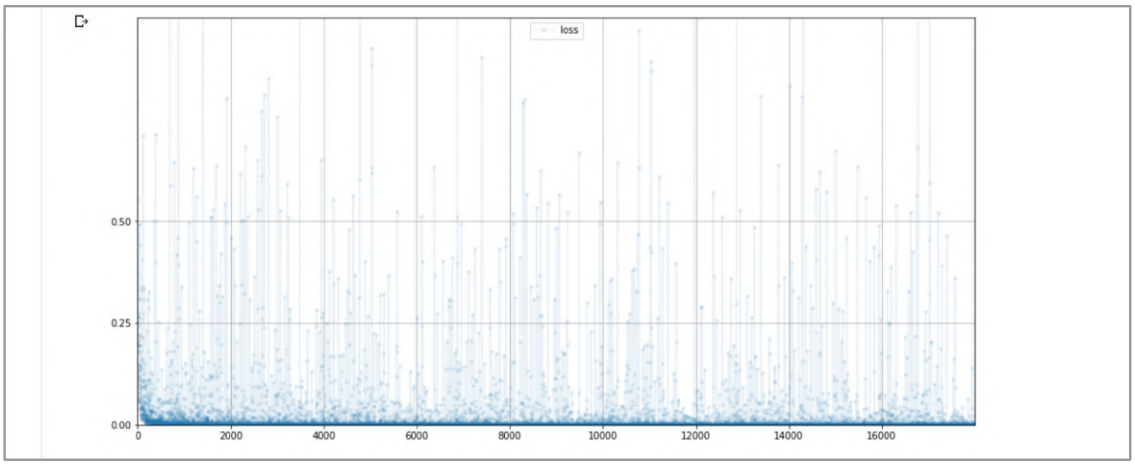
The last part of the network takes these **10** feature maps, each of size **5** by **5**, and reshapes these **10 * 5 * 5 = 250** values to a 1-dimensional list of **250** values. We'll need to use our own helper class again to use **View()** inside a Sequential list like this. The final fully connected layer takes these **250** values and maps them to **10** output nodes, each with a sigmoid activation. We want **10** output nodes because we're classifying the image as one of ten possible digits.

Here's a picture of our network architecture.



Let's train this CNN and test its performance in the same way as our fully connected MNIST classifier.

The first difference we notice is that this CNN is quicker to train. Mine took approximately **9.5** minutes compared to the **13.5** minutes for the fully connected network.



The loss chart looks very similar to the chart for the fully connected network. The loss falls rapidly towards zero and the bulk of the loss values remain close to zero.

```
print(score, items, score/items)
```

```
9814 10000 0.9814
```

The performance score of **98%** is higher than our previous record of **97%**. It might not look like much of an increase but for MNIST classifiers scores above **98%** are not easy to achieve. This

convolutional classifier takes us right up to that **98%** with a very simple design and a small amount of code.

The code we've just developed for a convolutional neural network classifier is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/13_cnn_mnist.ipynb

CelebA CNN

Now that we've practised using convolution layers to make a classifier, let's use them for a GAN.

We'll start with the code we developed for the CelebA GAN which is also online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/12_gan_celeba.ipynb

The CelebA images are a rectangular **217** by **178** pixels in size. To keep our convolutions simple, we'll work with square **128** by **128** images. That means we'll need to crop the training images to this size.

The following code is a helper function for cropping a numpy image array to a given size, with the crop centred on the supplied image.

```
def crop_centre(img, new_width, new_height):
    height, width, _ = img.shape
    startx = width//2 - new_width//2
    starty = height//2 - new_height//2
    return img[  starty:starty + new_height, startx:startx + new_width,
:]
```

To create a square **128** by **128** image from the centre of a larger image **img**, we'd use **crop_centre(img, 128, 128)**.

We'll need to move our helper functions above the Dataset class in our notebook because we'll need to use **crop_centre()** in its definition. The following code updates the **__getitem__()** and **plot_image()** methods. In both cases, an image is retrieved from the HDF5 dataset and then cropped to a **128** by **128** square.

```
def __getitem__(self, index):
    if (index >= len(self.dataset)):
      raise IndexError()
    img = numpy.array(self.dataset[str(index)+'.jpg'])
```

```
        # crop to 128x128 square
        img = crop_centre(img, 128, 128)
        return torch.cuda.FloatTensor(img).permute(2,0,1).view(1,3,128,128)
/ 255.0

    def plot_image(self, index):
        img = numpy.array(self.dataset[str(index)+'.jpg'])
        # crop to 128x128 square
        img = crop_centre(img, 128, 128)
        plt.imshow(img, interpolation='nearest')
        pass
```

The **__getitem__()** needs to return a tensor, and we know it now needs to be a 4-dimensional tensor of the form **(batch size, channels, height, width)**. The numpy arrays are 3-dimensional of the form **(height, width, 3)**. The **permute(2,0,1)** reorders the numpy array to **(3, height, width)**, and **view(1,3,128,128)** adds an additional dimension for batch size, set to **1**.

Let's check the Dataset class crops the images correctly.

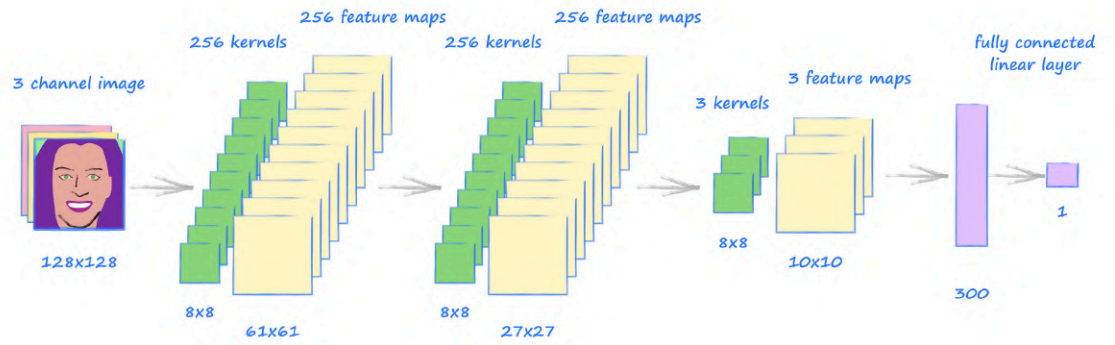
We can see the images are indeed cropped to a smaller **128** by **128** square.

Unlike fully connected layers, the size of what emerges from a convolution layer isn't immediately obvious. A pencil and paper is really helpful when designing convolutional networks. Some people like to work it out by drawing sketches of the input tensors, kernels, and strides, like we did earlier for very small tensors. Some will just experiment with the code, using the error messages to guide how they adjust the kernel and stride sizes. Some will use formulae like those in Appendix C or those on the PyTorch reference page for **nn.Conv2d()** to directly calculate the size of the output.

How many layers should our network have? How many kernels should we have in the middle layers? As we've discussed earlier and in *Make Your Own Neural Network*, there is no simple answer to this question. We should try to build the smallest possible network to make training easier, but not so small that it doesn't have the capacity to learn the given task. We should also

be mindful of whether our network is trying to reduce data, like a classifier, or expand data like a generator, as this will guide the shape of our networks.

The following network has **3** convolutional layers and a final fully connected layer.



The first convolutional layer takes the 3-channel colour images and applies **256** kernels to output **256** feature maps. Because the kernels are **8** by **8** and have stride **2**, the feature maps are **61** by **61** pixels in size, smaller than the **128** by **128** input images. The next convolution layer is the same, with **256** kernels of size **8** by **8** and stride **2**. We still have **256** feature maps emerging from this layer, but the sizes are reduced further to **27** by **27**. As we approach the end of the network, we need to think about reducing the data. The next convolution layer has only **3** kernels, but of the same **8** by **8** size and stride **2**, which gives us **3** feature maps of size **10** by **10**. That's **300** values which we need to reduce down to a single discriminator output value. We could use more convolution layers to reduce the data but **300** is small enough to have summarised key features of the images, and we can use an easy fully connected linear layer to map them to the output.

Phew! That was a lot to talk through, but it is worth going through once so we fully understand what's happening.

The following is the code for this discriminator network design.

```
self.model = nn.Sequential(
    # expect input of shape (1,3,128,128)
    nn.Conv2d(3, 256, kernel_size=8, stride=2),
    nn.BatchNorm2d(256),
     nn.LeakyReLU(0.2),

    nn.Conv2d(256, 256, kernel_size=8, stride=2),
    nn.BatchNorm2d(256),
```

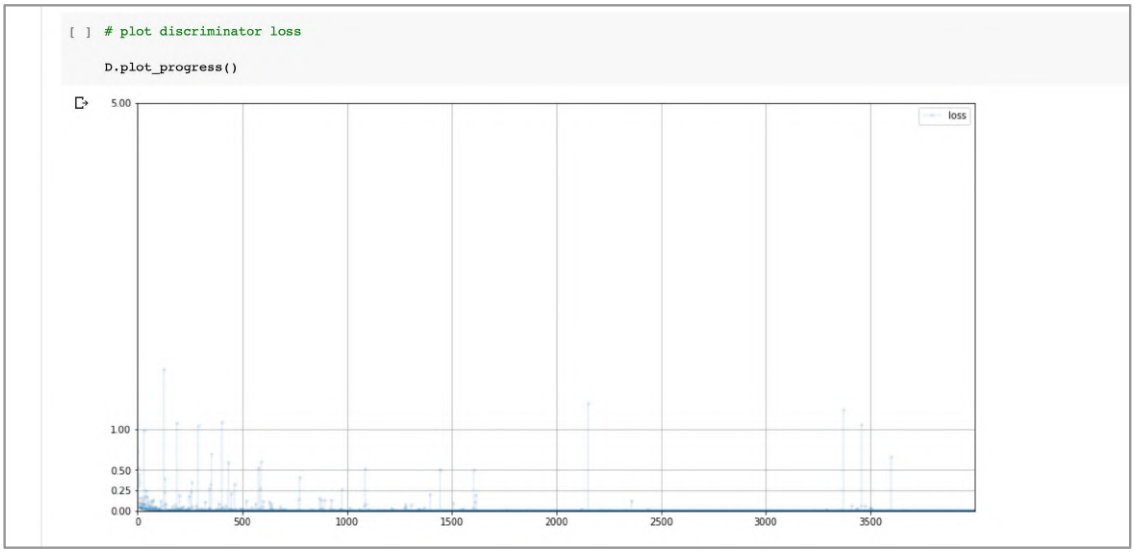
```
        nn.LeakyReLU(0.2),

        nn.Conv2d(256, 3, kernel_size=8, stride=2),
        nn.LeakyReLU(0.2),

        View(3*10*10),
        nn.Linear(3*10*10, 1),
        nn.Sigmoid()
    )
```

There isn't anything new in the code to discuss, we've seen all the elements already. It is worth noting that we use **View()** to reshape the last feature maps of size **(1, 3, 10, 10)** to a simple 1-dimensional tensor of size **300** so it can be passed to the linear layer. You might have noticed the code sometimes uses **3*10*10** instead of saying **300**. This is to help anyone reading our code understand where that number came from - a good habit.

To test the discriminator against images of random pixel values, we'll need to update the code so that **generate_random_image()** creates the expected 4-dimensional tensors of size **(1, 3, 128, 128)**. This change is easy because we wrote our random value helper functions to take shapes as parameters.

```
D.train(generate_random_image((1,3,128,128)),
torch.cuda.FloatTensor([0.0]))
```

This training loop takes about **10** minutes. The following chart shows the loss during training.



We can see the loss falls very rapidly towards zero. It is remarkable how little noise there is, with very few jumps to higher values that were typical of our previous networks.

Let's manually run the discriminator on real and random images to see the scores. Remember to update the random image function to ask for tensors of size **(1,3,128,128)**.

```
[ ] # manually run discriminator to check it can tell real data from fake

    for i in range(4):
      image_data_tensor = celeba_dataset[random.randint(0,20000)]
      print( D.forward( image_data_tensor ).item() )
      pass

    for i in range(4):
      print( D.forward( generate_random_image((1,3,128,128))).item() )
      pass
```

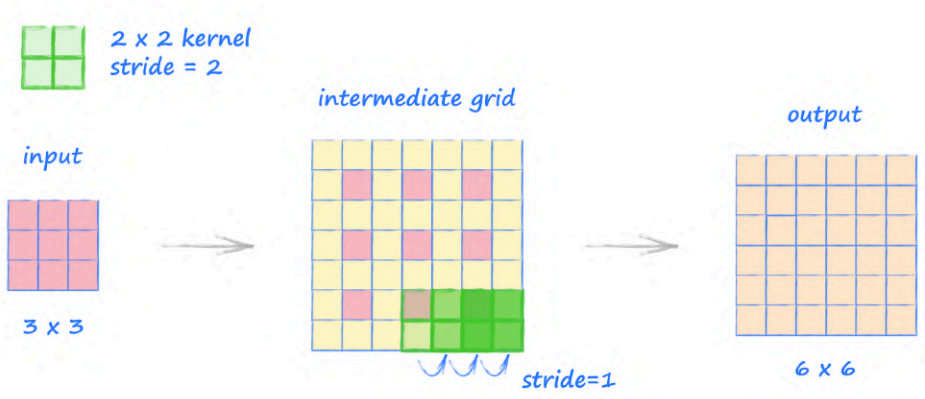
```
1.0
1.0
1.0
1.0
8.684115755386301e-07
1.302601582153784e-09
5.1147814872365416e-09
1.8210079133496038e-07
```

The scores are really very confident. It looks like we have a very effective discriminator network.

Let's now think about the generator network. That means it's time to pull out our pencil and paper again. We'll be guided by the principle that the generator should be a mirror of the discriminator so that one is not stronger or weaker than the other.

As you start to sketch out designs, you might be asking what the opposite of a convolution step is. Our convolutions reduce larger tensors into smaller ones. The opposite of a convolution will need to expand smaller tensors into larger ones. PyTorch calls this opposite a **transposed convolution**, and the module it provides is called **nn.ConvTranspose2d**.

The following picture illustrates how this transposed convolution works.

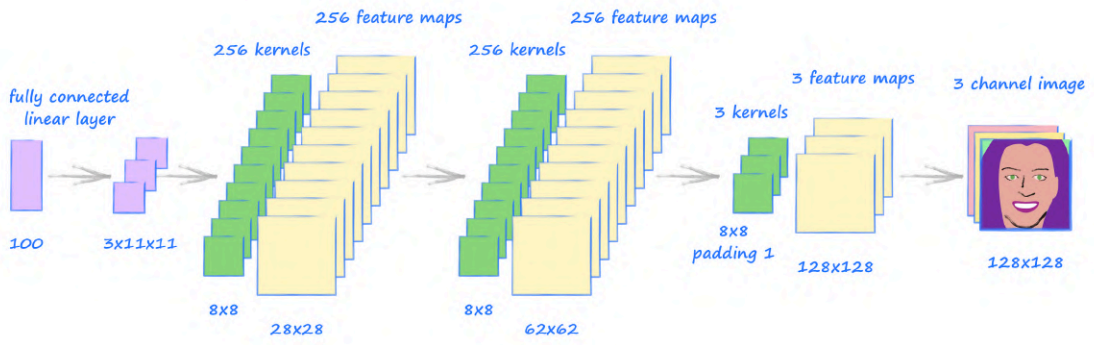


The input tensor is of size **3** by **3**. The kernel is **2** by **2**, and has a stride set to **2**. The way transpose convolution works is that the kernel moves over an intermediate grid with a stride of **1**, not **2**. That intermediate grid is the input tensor with additional grid squares of value **0** added in between the original squares. These separate the original values by the size of the stride, which here is **2**. Then, the maximum amount of additional **0** valued squares are added around the edge so the kernel still covers one of the original squares. The picture shows how the output tensor has size **6** by **6**.

This might seem like an overly complicated method of expanding a tensor. The key benefit of this method is that it undoes the effect of normal convolution configured with the same options. For example, if we take that output **6** by **6** tensor and perform a normal convolution with a **2** by **2** kernel with stride **2**, we get a **3** by **3** tensor again. There are special cases where the reversal isn't exact and requires an additional padding.

You can follow more step by step worked examples in Appendix C.

The following picture shows a convolutional network architecture that takes a seed of size **100** and eventually produces a tensor shaped **(1, 3, 128, 128)**.



Designing a convolutional network which expands tensors to exactly the right size takes a few attempts. This network does mostly look like a mirror of the discriminator, which is good. The beginning of the network has a fully connected layer which maps the **100** seed values to **3*11*11**, which are then reshaped to the required 4-dimensional **(1,3,11,11)** ready for the transpose convolution layers. The last transpose convolution layer needs an extra configuration which is a **padding** of **1**, which for transpose convolutions removes the outer squares from the intermediate grid. Without this, it is much more difficult to get the output to be of size **(1,3,128,128)** without growing the network itself.

It can be tempting to shortcut the design of a convolutional generator network by adding a fully connected layer at the end just to get the desired output size. We should avoid this if we can as we want our images to be constructed by localised features.

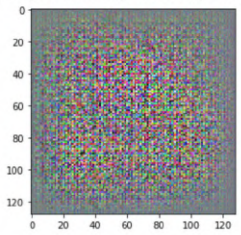
The following code defines the generator neural network.

```
self.model = nn.Sequential(
    # input is a 1d array
    nn.Linear(100, 3*11*11),
    nn.LeakyReLU(0.2),

    # reshape to 4d
    View((1, 3, 11, 11)),

    nn.ConvTranspose2d(3, 256, kernel_size=8, stride=2),
    nn.BatchNorm2d(256),
    nn.LeakyReLU(0.2),

    nn.ConvTranspose2d(256, 256, kernel_size=8, stride=2),
    nn.BatchNorm2d(256),
    nn.LeakyReLU(0.2),

    nn.ConvTranspose2d(256, 3, kernel_size=8, stride=2, padding=1),
    nn.BatchNorm2d(3),

    nn.Sigmoid()
)
```

The code follows the design we drew. It starts by using a fully connected layer to map the **100** seed values to **3*11*11**, which are then reshaped to the 4-dimensional **(1, 3, 11, 11)** required by PyTorch's convolution modules. There are three transpose convolutions, each of kernel size **8** and stride **2**. The first two have **256** kernels, and the last reduces this to **3**, because our output tensor needs to have **3** channels, one each for red, green and blue pixel values. The last convolution has the additional **padding=1** option set.

Let's check the untrained generator does produce an image of random pixel values, and is of the right shape. We'll need to reorder the 4-dimensional tensor from the generator so it is in the normal shape for images ready for plotting, just as we did before using **permute(0,2,3,1)** and **view(128,128,3)**.

```
[ ] # check the generator output is of the right type and shape

    G = Generator()
    # move model to cuda device
    G.to(device)

    output = G.forward(generate_random_seed(100))

    img = output.detach().permute(0,2,3,1).view(128,128,3).cpu().numpy()

    plt.imshow(img, interpolation='none', cmap='Blues')

    <matplotlib.image.AxesImage at 0x7fa0b071f160>
```

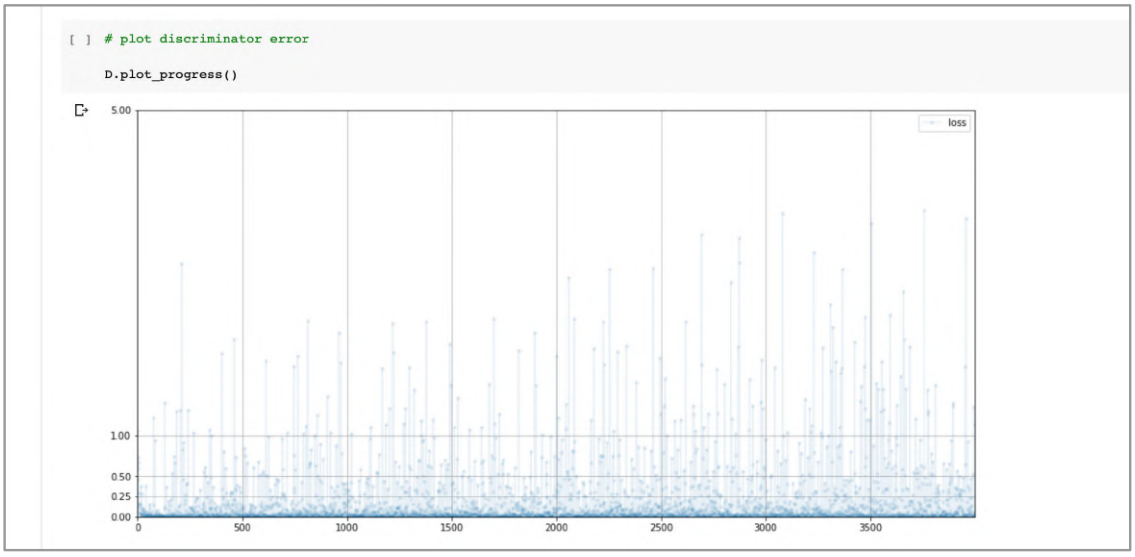


The untrained generator does create an image of the right size. It looks like it has random pixel values, but if we look closer we can see there seems to be a chequerboard pattern. There's also a dark vignette around the edges of the image. Did we make a mistake in our code?

We haven't made a mistake. When we construct an image through a series of transpose convolutions, the overlapping of feature maps, especially when the stride isn't a multiple of the kernel size, leads to a chequerboard pattern. The reason for the darker border is that there are simply less overlaps, and so fewer contributing values, around the edges of the image. As we train the generator, it will learn the right weights to compensate for this.

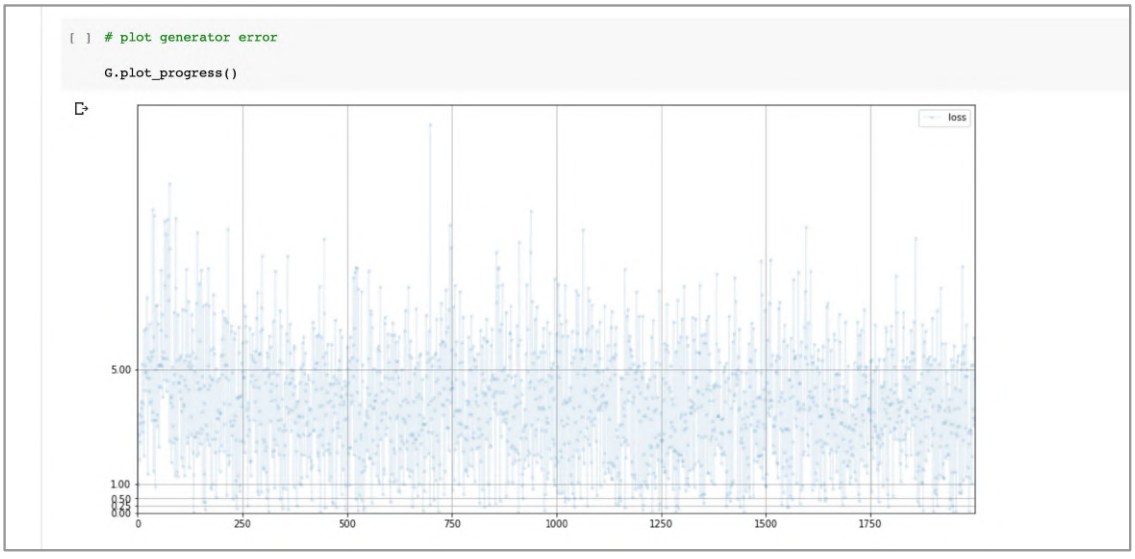
After all that preparation, let's finally train our GAN for one epoch. We don't need to make any changes to the training loop code.

The following chart shows the discriminator loss during training, which takes about **15** minutes for one epoch.



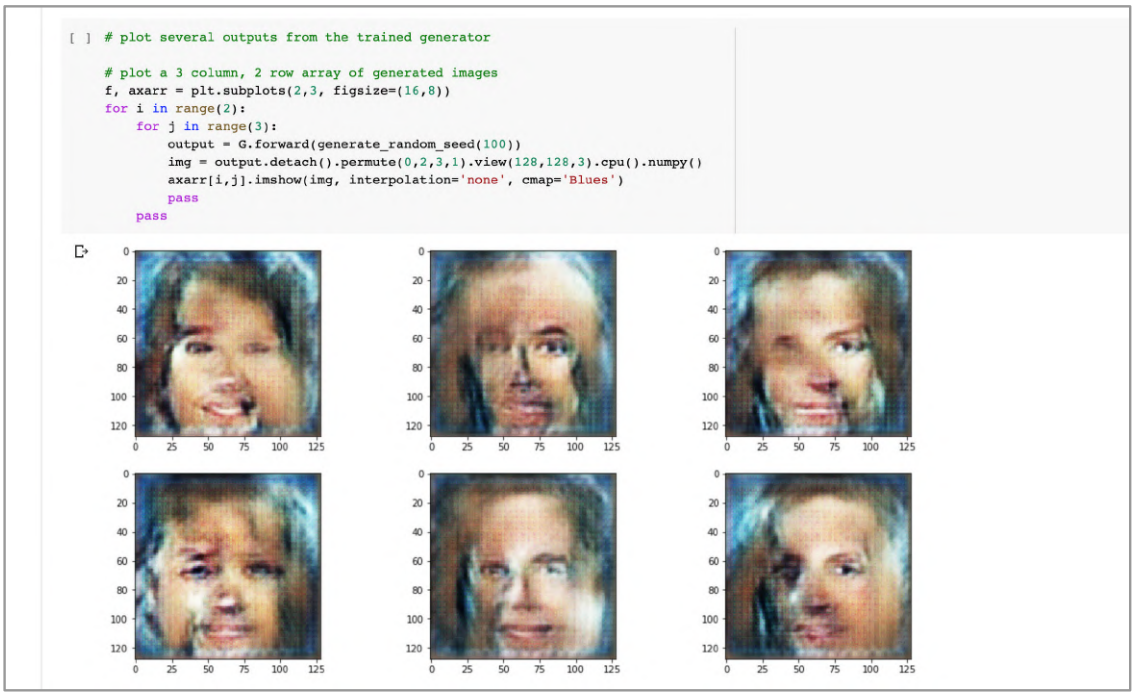
The loss very rapidly falls towards zero and stays low. This is better than a very chaotic unstable loss, but a loss approaching the ideal value **0.693** would have been better. There are small signs the loss is starting to rise towards the end of this training session.

Let's have a look at the generator loss.



Here the loss is not chaotic which is good. Although the loss is higher than the ideal value, it is slowly falling. Perhaps longer training is needed?

Let's finally look at the images produced by the generator after this one epoch of training.



```
[ ] # plot several outputs from the trained generator

    # plot a 3 column, 2 row array of generated images
    f, axarr = plt.subplots(2,3, figsize=(16,8))
    for i in range(2):
        for j in range(3):
            output = G.forward(generate_random_seed(100))
            img = output.detach().permute(0,2,3,1).view(128,128,3).cpu().numpy()
            axarr[i,j].imshow(img, interpolation='none', cmap='Blues')
            pass
        pass
```

Hurray! Our convolutional GAN has produced images that have the basic features of a face. There are two eyes, a nose, a mouth, and in many cases, hair. The image quality isn't very high, but what we've achieved is worth thinking about. These images are composed through a hierarchy of high, medium and low-level localised features learned by the convolutional neural network. And of course, these features are not copied from the training data. Their arrangement, eyes above nose and nose above mouth, have been learned to get past the discriminator. That's pretty cool!

We should also consider ourselves very lucky, or very talented, because we seem to have avoided mode collapse. The generated images are diverse.

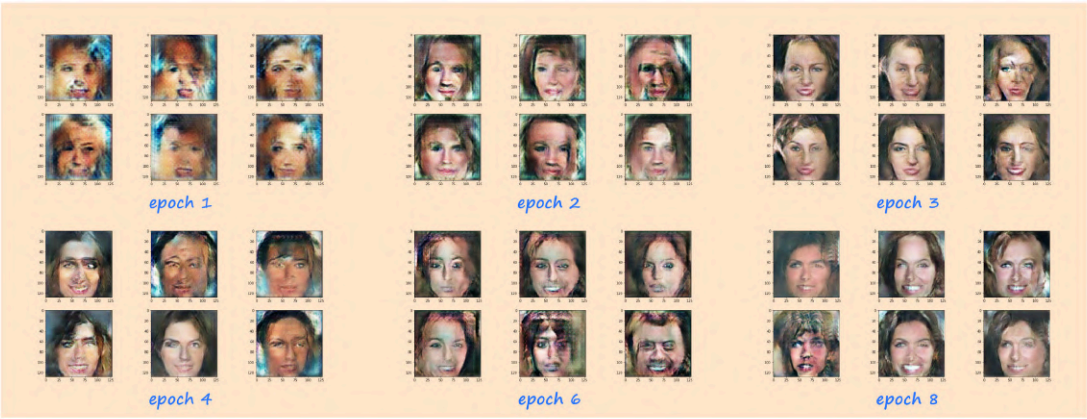
While we're congratulating ourselves, let's check the memory consumption of this convolutional GAN so we can see if it is lower than the fully connected GAN we started with.

```
[ ] # current memory allocated to tensors (in Gb)

    torch.cuda.memory_allocated(device) / (1024*1024*1024)

    0.1423473358154297

[ ] # total memory allocated to tensors during program (in Gb)

    torch.cuda.max_memory_allocated(device) / (1024*1024*1024)

    0.2035832405090332
```

The memory still allocated to tensors after a full run of the notebook is **0.14 gigabytes**. This is primarily the discriminator and generator objects which still exist. The fully connected GAN had a value of **0.70 gigabytes**. So our convolutional GAN consumes about **5** times less memory than the fully connected GAN. This is an impressive improvement.

Looking at the peak memory consumption for tensors, this value of **0.20 gigabytes** is much lower than the previous **1.1 gigabytes**, also about **5** times lower.

Let's see if more training improves the images. The following shows the images from **1, 2, 3, 4, 6** and **8** epochs of training. They're shown together so we can compare image quality.



The image quality isn't great to start with, but with training the faces start to become very well constructed. Some of the faces have more realistic smooth skin. We've also managed to avoid mode collapse, and the diversity of faces and pose angles is encouraging. Not every image is good, and it isn't clear whether much longer training will fix this. Perhaps our very simple network architecture has inherent limitations.

Looking again at these images, they do look like they were put together as a set of localised patches. We can see, for example, that one eye can be very different to another eye, or that half of a hairstyle is different to the other half. We didn't see this happen so much with the fully connected GAN, and this is because in a convolutional network each feature is generated

without a full view of the image from the previous layer. The intentional narrowing of focus using convolutions has disadvantages as well as great advantages.

The code we've developed to generate faces using a convolutional GAN is online:

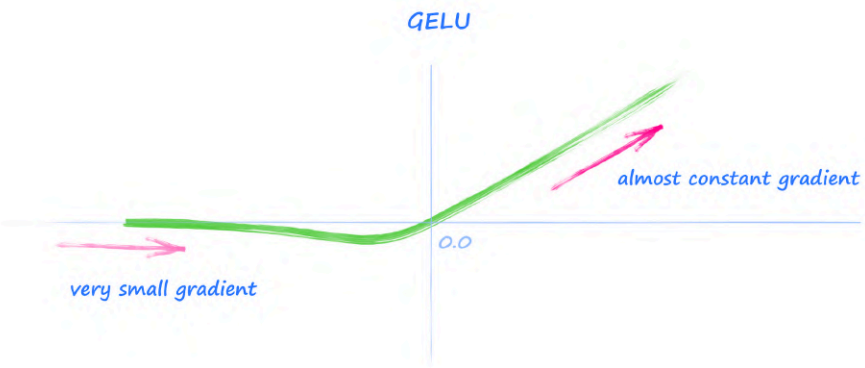
- https://github.com/makeyourownneuralnetwork/gan/blob/master/14_gan_cnn_celeba.ipynb

Your Own Experiments

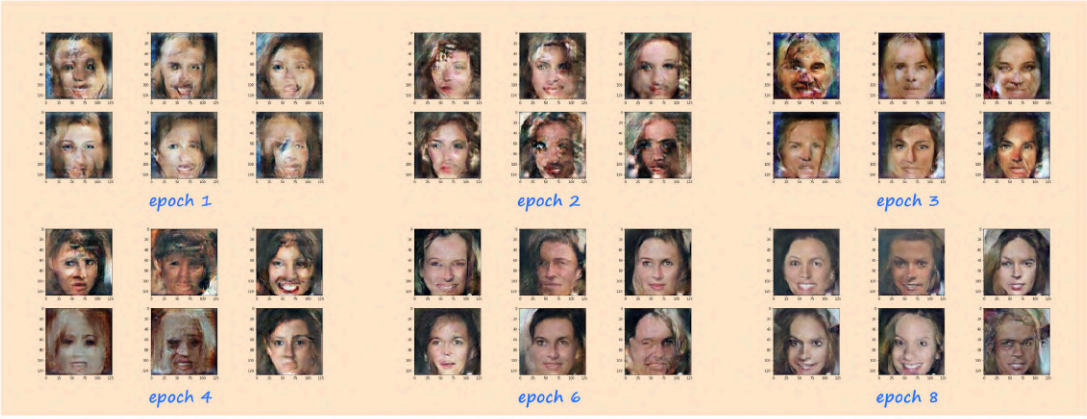
If you've worked through to this stage you've covered enough basics to meaningfully try your own ideas for improving the GAN process.

You could try different kinds of loss functions, different sizes of neural network, perhaps even variations of the standard GAN training loop. Perhaps you might try to discourage mode collapse by including a measure of diversity over several outputs in the loss function. If you're very confident, you might try to implement your own optimiser, one which is better suited to the adversarial dynamics of a GAN.

The following is one of my own simple experiments with an activation function called GELU, that is like a ReLU but has a softer corner. Some have suggested that such activation functions are now the state of the art because they provide good gradients, and don't have a sharp discontinuity around the origin.

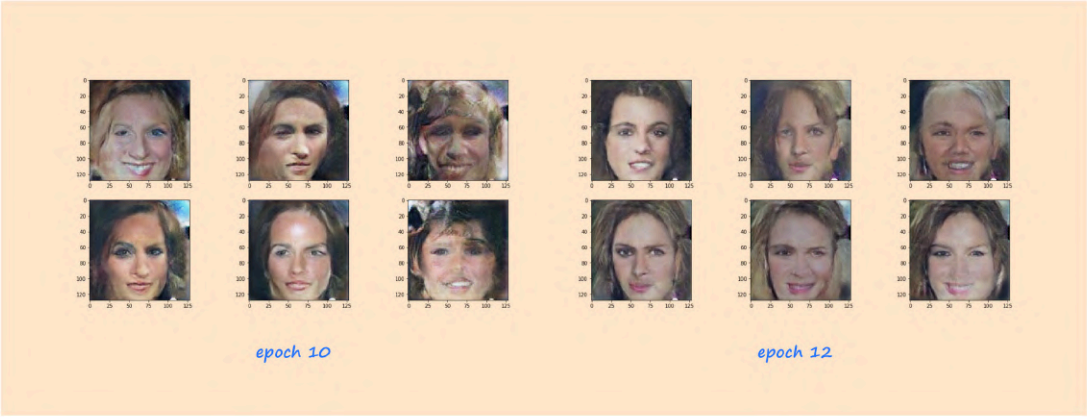


The following images are generated just like the above examples, but with **nn.LeakyReLU(0.2)** replaced with **nn.GELU()**.



Although not a scientific test, a visual comparison suggests the images from using the GELU activation lead to slightly higher quality images.

The following are images from further training to epochs **10** and **12**.



The fidelity is really becoming impressive in some of these images. Further training could lead to further improved image quality but in reality many simple GAN architectures eventually collapse and the images degrade and suffer mode collapse.

You can read a slightly theoretical discussion of why GAN training using gradient descent might be fundamentally flawed in Appendix D.

The code for this experiment with is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/15_gan_cnn_celeba_refinements.ipynb

Learning Points

- State of the art image classifying networks take advantage of the fact that meaningful image features are **localised**, and that recognisable objects are composed of a **hierarchy** of low-level detail features, which combine to form medium-level features, which themselves combine to form high-level objects.
- **Convolutions** apply a **kernel** over an image to output **feature maps**. Specific kernels can pick out specific localised patterns in an image.
- Convolution layers in a neural network can learn good kernels for achieving the given task. That is, the network learns which images features are most useful without us engineering those features directly. Neural networks that use convolutional layers generally perform better at image classification than equivalent fully connected networks.
- Where a convolution reduces data, the equivalently configured **transposed convolution** reverses this reduction, making it ideal for generative networks.
- GANs based on convolutional networks construct images by composing low-level features into medium-level features, which are composed into high level features. Experiments show the images they produce are of higher quality than equivalent fully connected GANs.
- Convolution GANs require less **memory** than a fully connected GAN. This can be a factor when working with medium to large sized images within the constraints of GPU memory. We saw a **5** times reduction in memory consumption.
- One disadvantage of convolutional generators is that they can lead to images composed of unmatching elements, for example, faces with different eyes. This is because the information flow through a convolutional network is intentionally localised, and global relationships aren't learned.

Conditional GANs

The MNIST GANs we developed earlier generated a wide variety of output images. This was good because a constant challenge for designing GANs is avoiding low diversity and mode collapse.

It would be useful if we could somehow encourage our GAN to produce images that were diverse, but also restricted to one class of the training data. We could then ask our GAN to produce different images of the digit **3**, for example. If we were training with face images, we might be able to ask a GAN to produce only happy faces, if emotion was a class in the training data.

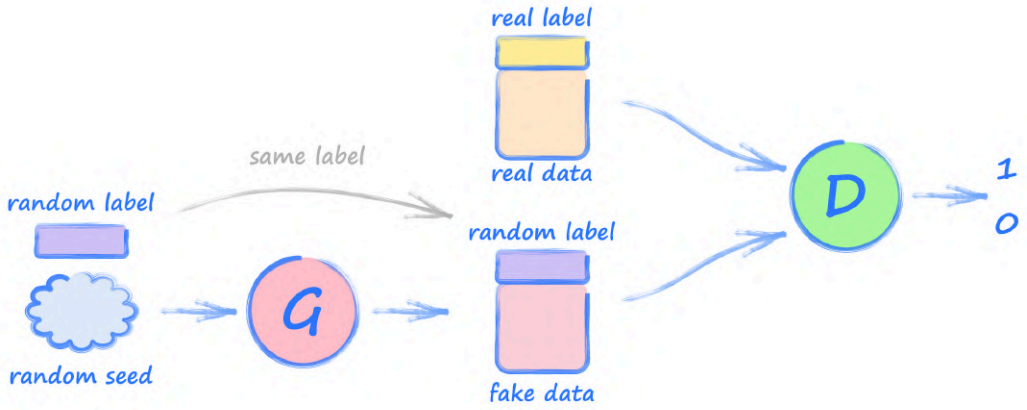
Conditional GAN Architecture

Let's think about what such an architecture would look like.

If we want the generator of a trained GAN to output an image of a given class, it needs to be told which class we want. That means we need to include it as part of the input to the generator, alongside the random seed.

The discriminator needs a little more thought. Previously its only job was to try to separate real images from generated images. Now we want it to also learn to associate a class label with an image. If it doesn't do this, there is no feedback it can provide to the generator to get it to also associate images with class labels. This means we also need to provide the class label to the discriminator alongside the image.

The following shows the architecture of what is called a **conditional GAN**.



The key change is that both the generator and discriminator now have the class label in addition to the image data as input.

Discriminator

Let's develop this GAN by making changes to our earlier fully connected MNIST GAN.

- https://github.com/makeyourownneuralnetwork/gan/blob/master/08_gan_mnist.ipynb

We need to update the discriminator to take both image pixel data and the class label information. A simple way to do this is to extend the **forward()** function to take both the image tensor and a label tensor, and simply join them together. That label tensor is the one-hot tensor already prepared for us by the Dataset class.

```
def forward(self, image_tensor, label_tensor):
    # combine seed and label
    inputs = torch.cat((image_tensor, label_tensor))
    return self.model(inputs)
```

The **torch.cat()** function simply extends one tensor by another. The image tensor returned from the Dataset class is of length **784**, the label tensor has length **10**, so the concatenation will be of length **794**.

Because we've extended the size of the input, we need to update the first layer of the neural network definition to expect an input of **784 + 10** values.

```
self.model = nn.Sequential(
    nn.Linear(784+10, 200),
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 1),
    nn.Sigmoid()
)
```

It is worth writing the new size as **784 + 10**, and not **794**, so others reading our code can see the change, and also see the reason for that change - a good coding habit.

One final change to the discriminator is in the **train()** function where we need to add the label tensor to the **forward()** call. The following shows only the top of the **train()** function.

```
def train(self, inputs, label_tensor):
```

```
    # calculate the output of the network
    outputs = self.forward(inputs, label_tensor)
```

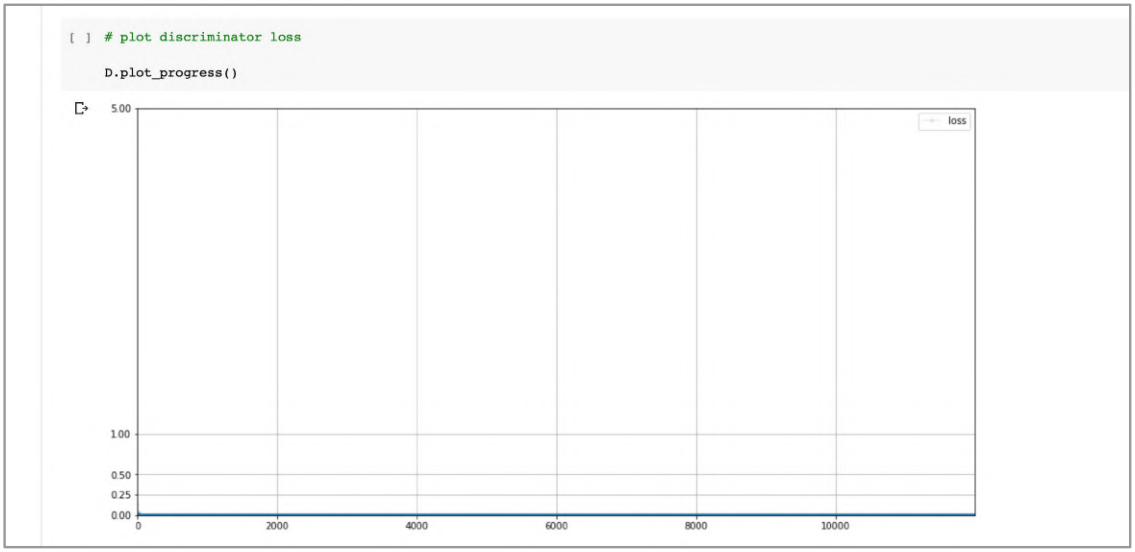
Let's test the discriminator in the usual way. We'll need to update the training loop code to pass the additional label tensor to the **train()** function.

```
for label, image_data_tensor, label_tensor in mnist_dataset:
    # real data
    D.train(image_data_tensor, label_tensor, torch.FloatTensor([1.0]))
    # fake data
    D.train(generate_random_image(784), generate_random_one_hot(10),
torch.FloatTensor([0.0]))
    pass
```

Writing that code has shown we need a random class label to go with the random image. So let's create a convenience function **generate_random_one_hot()** for generating a random one-hot label vector.

```
# size here must only be an integer
def generate_random_one_hot(size):
    label_tensor = torch.zeros((size))
    random_idx = random.randint(0,size-1)
    label_tensor[random_idx] = 1.0
    return label_tensor
```

It'll be interesting to see the effect, if any, on the loss values.



The loss chart hasn't changed significantly, and looks almost identical to the original discriminator.

Generator

Let's now think about the generator. Because we're feeding it the seed and the label tensor, we need to update the **forward()** function to combine them before passing them through the neural network.

```
def forward(self, seed_tensor, label):
    # combine seed and label
    inputs = torch.cat((seed_tensor, label_tensor))
    return self.model(inputs)
```

The first layer of the network will need to change to accommodate the **10** additional values.

```
self.model = nn.Sequential(
    nn.Linear(100+10, 200),
    nn.LeakyReLU(0.02),

    nn.LayerNorm(200),

    nn.Linear(200, 784),
    nn.Sigmoid()
)
```

Finally the **train()** function will also need to accept the label tensor too.

```
def train(self, D, inputs, label_tensor, targets):
    # calculate the output of the network
    g_output = self.forward(inputs, label_tensor)

    # pass onto Discriminator
    d_output = D.forward(g_output, label_tensor)
```

We use the same label tensor when passing the inputs to the generator's own **forward()** function, and when passing the generated image to the discriminator's **forward()** function. If we didn't do this, the discriminator would be providing feedback to the generator relevant to a different class label.

Training Loop

The main GAN training loop is also updated to pass a label tensor to the discriminator and generator. The following shows only the code inside the epoch loop.

```
for label, image_data_tensor, label_tensor in mnist_dataset:
    # train discriminator on true
    D.train(image_data_tensor, label_tensor, torch.FloatTensor([1.0]))

    # random 1-hot label for generator
    random_label = generate_random_one_hot(10)

    # train discriminator on false
    # use detach() so gradients in G are not calculated
    D.train(G.forward(generate_random_seed(100),
random_label).detach(), random_label, torch.FloatTensor([0.0]))

    # different random 1-hot label for generator
    random_label = generate_random_one_hot(10)

    # train generator
    G.train(D, generate_random_seed(100), random_label,
torch.FloatTensor([1.0]))

    pass
```

Notice how we create a **random_label** as a variable so we can use the same random label tensor to feed both the generator and the discriminator when we train the discriminator on generated images.

Plotting Images

Once the generator is trained, it will be useful to ask it to produce several images of a chosen label. Let's add this as a new **plot_images()** method to the generator class.

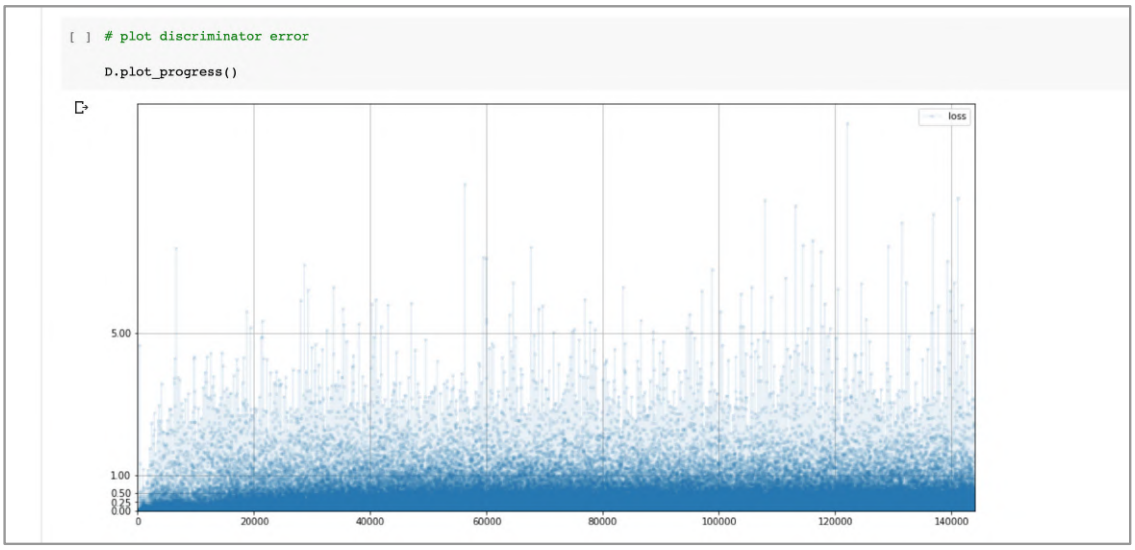
```
def plot_images(self, label):
    label_tensor = torch.zeros((10))
    label_tensor[label] = 1.0
    # plot a 3 column, 2 row array of sample images
    f, axarr = plt.subplots(2,3, figsize=(16,8))
    for i in range(2):
        for j in range(3):
```

```
          axarr[i,j].imshow(G.forward(generate_random_seed(100),
label_tensor).detach().cpu().numpy().reshape(28,28),
interpolation='none', cmap='Blues')
          pass
        pass
    pass
```

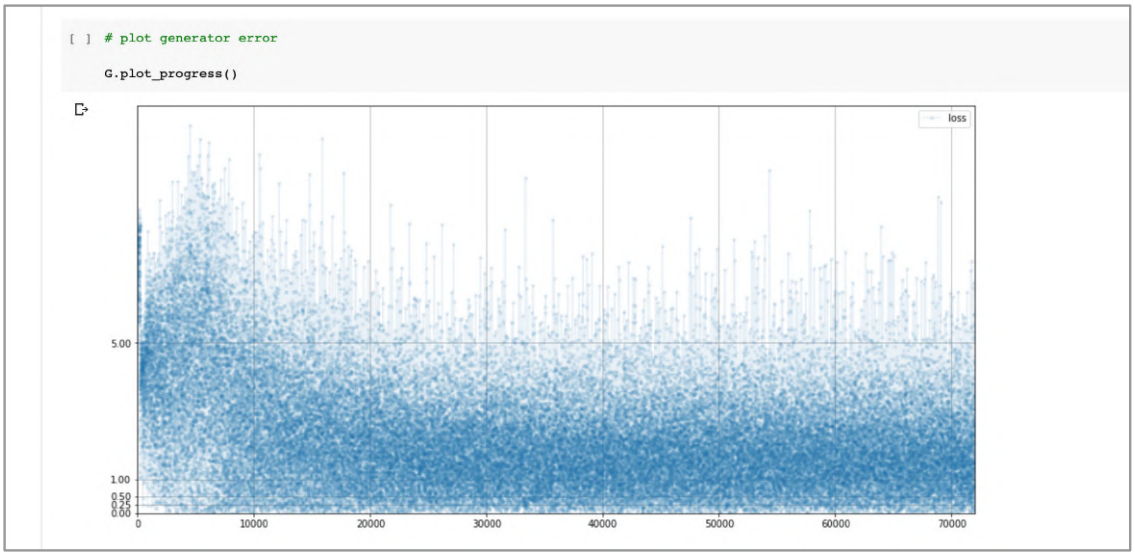
This function will take a **label**, provided as an integer, and prepare a one-hot tensor from it, which is then fed to the generator. Six different random seeds are used to generate six output images, plotted in a grid.

Conditional GAN Results

Training the GAN for **12** epochs takes me about **1** hour **30** minutes. At first glance, the discriminator loss looks similar to the loss from the earlier GAN. Looking closer, the loss values aren't entirely close to zero and in fact seem to be rising. This is encouraging when we remember that the ideal loss for a GAN is not zero.

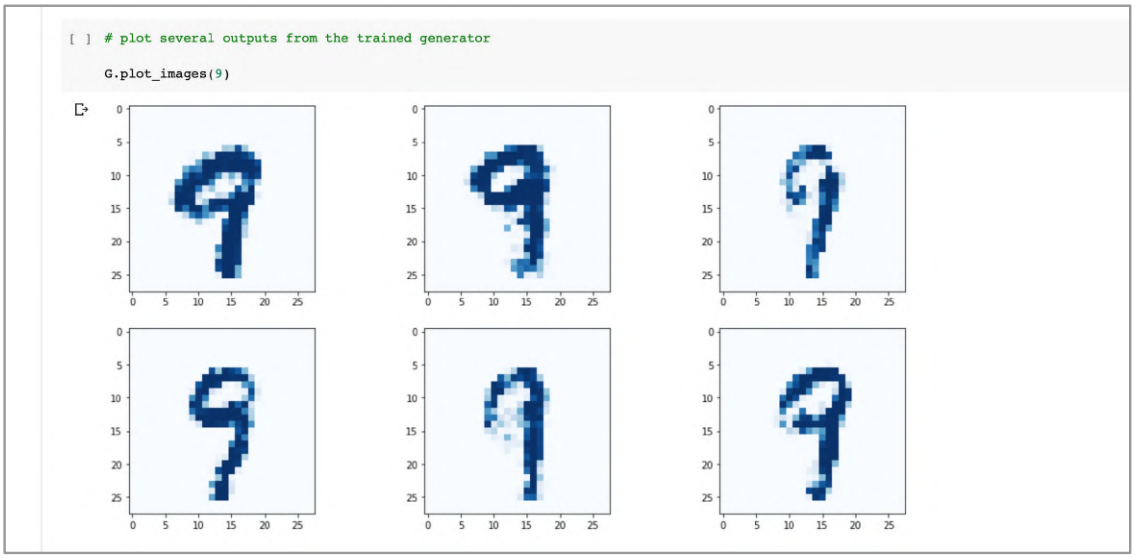


The generator loss also looks similar to the original GAN, and if we look closely, the average seems to be non-zero, which is good.



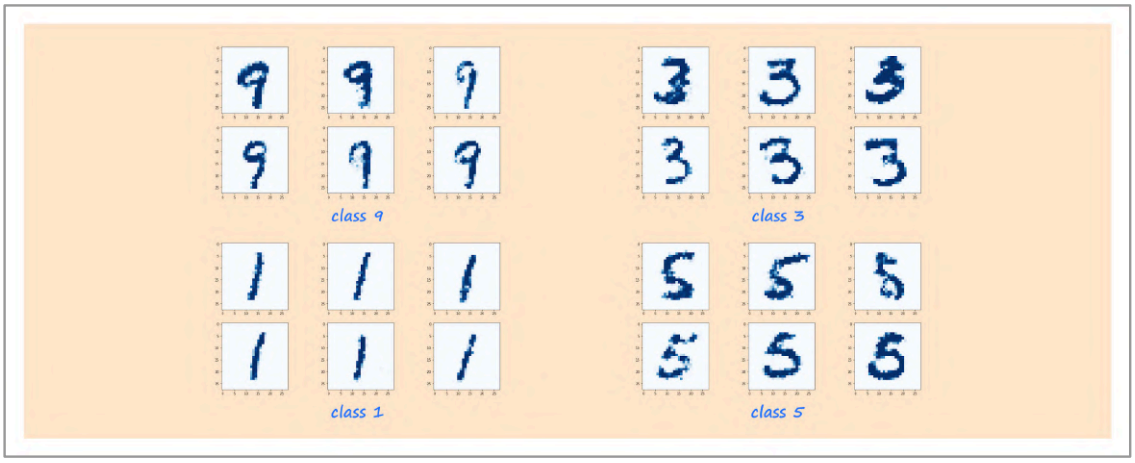
This suggests that using the additional label information helps GAN training. This makes sense because the discriminator has more valuable information to help it decide if an image is real, and this is then fed back to the generator.

Finally, let's ask the GAN to create images of a **9** using **plot_images(9)**.



It worked! Our conditioned GAN has indeed generated images of the digit **9**, and even better, the images are not all the same.

The following shows six generated images for several randomly chosen classes **9**, **3**, **1** and **5**.



We can see the GAN produces images of the desired digit, and those images are not all the same.

The ability to generate diverse images of a given class is really powerful. We can imagine many applications, for example generating faces with a given emotional expression, or flowers of a chosen colour. The key requirement is that the training data does need to be labelled with the classes we're interested in.

The code for this conditional GAN for MNIST digits is online:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/16_cgan_mnist.ipynb

Learning Points

- Unlike normal GANs, **conditional GANs** can be directed to produce output of a desired class.
- Conditional GANs are trained by augmenting the images fed to the discriminator, and the seed fed to the generator, with the **class label**.
- Conditional GANs generally produce images of higher quality than the equivalent GANs without label information.

Conclusion

Well done!

... on working through this book!

You've developed a solid understanding of PyTorch basics. Even better, you've gained hands-on experience using this knowledge to build and train several kinds of neural networks. You should feel confident in using your PyTorch experience to develop more sophisticated architectures.

You've also developed an understanding of GANs, currently one of the most exciting areas of machine learning. You've gained actual experience of designing networks, seeing them fail, and remedying that failure. Not many people have this experience.

You've also been lucky enough to work at the cutting edge of machine learning, where it is entirely possible for one of your ideas to become a breakthrough. That's rather exciting!

Future Directions

In this guide we've deliberately kept the content to a minimum. We focussed on images, but GANs can learn other kinds of data too, including sound, video, and even natural language text. The applications might be different but the core concepts are the same.

There has been a lot of research activity trying to explore new ways to improve GAN training, and also to further develop the theoretical basis for how and why GAN training can fail. There is a whole zoo of different ideas and approaches, from novel loss functions to interesting architectures designed to punish low output diversity. You should dip in and explore some yourself.

One of the most promising areas of research, in my opinion, is the idea that gradient descent is fundamentally the wrong approach to optimisation where two or more agents are trying to achieve opposing goals.

Even basic questions are still open. How do we measure and compare the quality and diversity of images produced by a GAN? How do we compare the effectiveness of different GAN architectures in a scientifically sound way?

Responsible Use

With GANs we can create realistic looking - but entirely synthetic - data. The potential for misuse or accident is significant. Several organisations are developing guidelines and frameworks to help you ensure our use of machine learning is safe, responsible and ethical. Take a look, and do adopt them if your use of machine learning could affect people.

Machine Learning Is Amazing!

Let's say it again - GANs, even very simple ones written in a few lines of code, can learn to generate realistic images without ever having directly seen the training data. Those images are not made of copied and pasted bits of the training examples, and neither are they some sort of average of the training examples.

After all that hard work thinking about the concepts and writing code, that fact is still amazing.

Have fun!

Appendices

Appendix A: Ideal Loss Values

When training a GAN, the ideal state we want to reach is a balance between the generator and the discriminator. When this happens, the discriminator is no longer able to separate real data from generated data. This is because the generator has learned to create data that looks like it could have come from the real dataset.

Let's work out what the discriminator loss should be when this balance is reached. We'll do this for both the **mean squared error** loss, and the **binary cross entropy** loss.

MSE Loss

The mean squared error loss has a simple definition. The difference between the value that emerges from an output node and desired target value is the error. It can be positive or negative. If we square this error, the value is always positive. The mean squared error is the mean average of these squared errors.

This loss is written mathematically as follows. For each of the **n** output layer nodes, the actual output is **o**, and the desired target is **t**,

$$loss = \frac{1}{n} \sum_{n} (t-o)^2$$

Because a discriminator only has one node, we can simplify this.

$$loss = (t-o)^2$$

When the discriminator can't tell the real data from the generated data, it doesn't output **1** because that would mean it is totally confident the data is real. It doesn't output **0** because that would mean it is fully confident the data is generated.

The discriminator outputs **0.5** because it is equally not confident about the data being real or generated.

If the output is **0.5**, and the target is **1**, the error is **0.5**. When the target is **0**, the error is **-0.5**. When these errors are squared, they both become **0.25**.

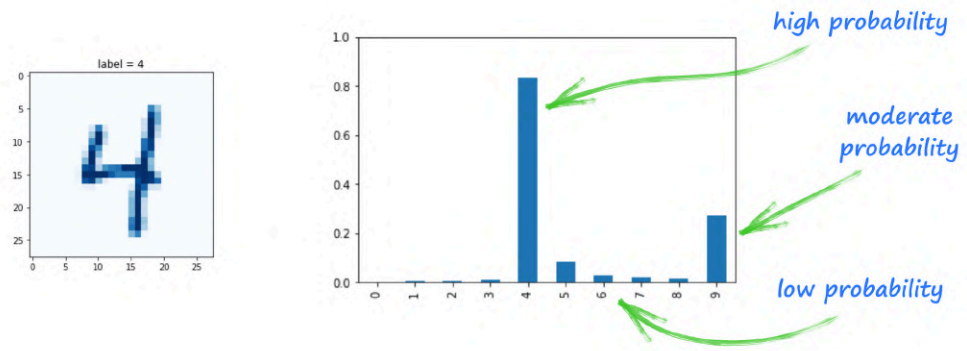
So the MSE loss of a balanced GAN is **0.25**.

BCE Loss

The binary cross entropy loss is based on the ideas of probability and uncertainty. Let's take it step by step.

Looking back at the MNIST classifier, the neural network has **10** output nodes, one for each possible classification. If the trained network thought an image was of the digit **4**, the fourth output node would have a high value, and the other nodes would have a low value.

We previously talked about these values as a measure of confidence in the classification. It is an easy step to think of these values as **probabilities**. This step is made easier by the fact that the output nodes only have values between **0** and **1**, just like probabilities.



The above shows an image of a **4**, and the outputs of the classifier. The network has assigned a high probability to the fourth node, which means it thinks the image is highly likely to be a **4**. It has also assigned a moderate probability to the ninth node, to say it thinks the image could be a **9**. It has assigned very low probabilities to other nodes because it doesn't think the image looks at all like a **2** or a **3**, for example.

Have a look at the following table which shows examples of an output node's value **x** and the desired output **y**.

output x	target y	comment

0.9	1.0	almost correct
0.1	1.0	very wrong

In the first row, the neural network has given a classification with a probability of **0.9**. The target is **1.0**, so it's almost correct. A good loss function would have a low value for this output.

In the second row, the classification has a very low probability **0.1** which is the network saying it doesn't really think the classification applies. The target is **1.0**, so the network has got this very wrong. A good loss function would have a high value for this output.

Let's now move from probability onto uncertainty.

Entropy is a mathematical idea for describing **uncertainty**. If we had a very biased coin with both sides having a head, the chance of getting a head is **100%**. Similarly the chance of getting a tail is **0%**. In both cases we're **100%** certain of the outcome. The uncertainty is zero, so we say the entropy is **0**.

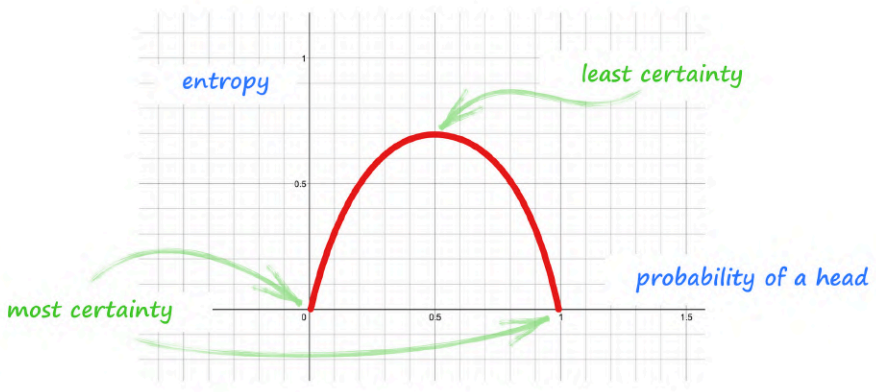
If we had a fair coin, with a head on one side and a tail on the other, we're maximally uncertain about the outcome. The entropy is highest.

The mathematical expression that gives us this entropy is

$$entropy = \sum -p \cdot \ln(p)$$

The sum is over all potential outcomes, and **p** is the probability of those outcomes. We won't go into where this expression comes from, but we will see visually why it has the right shape.

The following graph shows the entropy, calculated using the above expression, for a coin where the probability is the chance of a head.



The graph shows us that when we have a coin where both sides are a head and so **p(head) = 1**, the uncertainty is **0**. It also shows us that the uncertainty is zero when both sides are a tail and **p(head) = 0**. The entropy is highest when the coin is fair and **p(head) = 0.5**.

So we can see how it works, let's do the calculation for a coin with both sides as heads so **p(head) = 1**.

$$\begin{aligned} \text{entropy} &= \sum -p \cdot \ln(p) \\ &= -1 \cdot \ln(1) - 0 \cdot \ln(0) \\ &= 0 \end{aligned}$$

The outcomes we calculate over are a head and a tail. The probability of a head is **1**, and the probability of a tail is **0**. The answer drops out easily since **ln(1)** is **0**. That expression **0·ln(0)** is **0**, even if **ln(0)** is undefined. This matches the graph which shows zero entropy when **p(head) = 1**.

Try the calculation yourself for a fair coin so **p(head) = 0.5** and **p(tail) = 0.5**. What is this maximum entropy?

Great, so entropy is a way of expressing uncertainty of outcome.

Let's now talk about **cross entropy**, which is a measure of the uncertainty of outcomes due to a mismatch between the actual likelihood of the outcomes and what we think that likelihood is.

That sounds very abstract so let's go back to our coin example. If we think our coin is fair, but in reality it is not, then we'll be surprised by the outcomes. There will be uncertainty about those outcomes. This is what cross entropy captures. If we think our coin is fair, and in reality it is indeed fair, then we won't be surprised by the outcomes. The cross entropy will be low.

We can think of cross entropy as a comparison between two probability distributions. The more they match, the lower the cross entropy. An exact match gives zero cross entropy.

How does this relate to neural networks? Well the target output is a probability distribution, and so is the actual output. If they're different, the cross entropy will be high. And if they're similar, the cross entropy will be low. Which is exactly what we want a loss function to do.

That's the intuition. The following expression defines cross entropy mathematically. The sum is over all possible classifications, where **x** is the observed probability of a classification, and **y** is the actual probability of that classification.

$$\text{cross entropy} = \sum -y \cdot \ln(x)$$

Let's calculate it for our earlier neural network example when the output **x** is **0.9** but should be **1.0**. We sum over all possible classifications, which here are **1.0** and its opposite **0.0**.

$$\begin{aligned} \text{cross entropy} &= \sum -y \cdot \ln(x) \\ &= -1 \cdot \ln(0.9) - (1-1) \cdot \ln(1-0.9) \\ &= 0.105 \end{aligned}$$

Let's do the same for the other example when the output is **0.1** but should be **1.0**.

$$\text{cross entropy} = \sum -y \cdot \ln(x)$$
$$= -1 \cdot \ln(0.1) - (1-1) \cdot \ln(1-0.1)$$
$$= 2.303$$

We can summarise these results in a table. I've added an extra row where the output **x** is **0.9** but should be **0.0**.

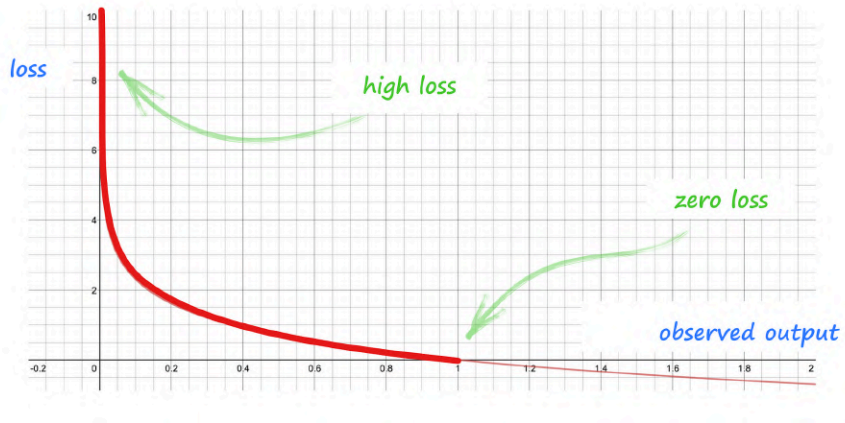
output x	target y	cross entropy
0.9	1.0	0.105
0.1	1.0	2.303
0.9	0.0	2.303

The first two rows show us that cross entropy is larger for the very incorrect output, and smaller for the almost correct output. The third row shows us that confident but incorrect outputs also have a high cross entropy. This is why we can use cross entropy as a loss function.

But why would we use this much more complicated method as a loss function, when the MSE loss is much simpler to calculate and easier to understand.

Strictly speaking, we can use any loss function that punishes wrong outputs. Some might prefer to use cross entropy because it can be mathematically derived from classification tasks rather than regression tasks. But the key reason is that it punishes wrong outputs much more strongly. Remember that unlike the MSE loss, the cross entropy values can grow much larger than **1.0** because of the logarithm. That steep loss function creates strong gradients for feeding back to a neural network.

The following graph shows the cross entropy as a loss for different observed outputs where the correct output should be **1.0**.



It is really clear to see that very wrong outputs are met with very large losses, which also have strong gradients.

Binary cross entropy is just the cross entropy where we know there are only two classifications. This is the case for a discriminator where the classifications are real **1.0** and fake **0.0**.

Let's now finally answer our original question, what is the ideal binary cross entropy loss when the discriminator and generator are balanced?

The discriminator is equally bad at classifying real and generated data, so its outputs are always **0.5**. Some of these should have been a **1.0**, and the rest should have been a **0.0**.

For **x = 0.5** and **y = 1.0** we calculate the cross entropy like this.

$$\begin{aligned} \text{cross entropy} &= \sum -y \cdot \ln(x) \\ &= -(1.0) \cdot \ln(0.5) - (0.0) \cdot \ln(1-0.5) \\ &= 0.693 \end{aligned}$$

And for **x = 0.5** and **y = 0.0** the calculation gives us the same cross entropy value.

$$\begin{aligned} \text{cross entropy} &= \sum -y \cdot \ln(x) \\ &= -(0.0) \cdot \ln(0.5) - (1.0) \cdot \ln(1-0.5) \\ &= 0.693 \end{aligned}$$

So the ideal loss for a GAN when using **BCELoss()** should be **0.693**.

Appendix B: GANs Learn Likelihood

What does a GAN actually learn? This is a very good question which doesn't have an immediately obvious answer.

Here we'll develop an intuition for what a GAN learns in a way that avoids lots of mathematical jargon. Let's start with what a GAN doesn't learn.

GANs Don't Memorise Training Data

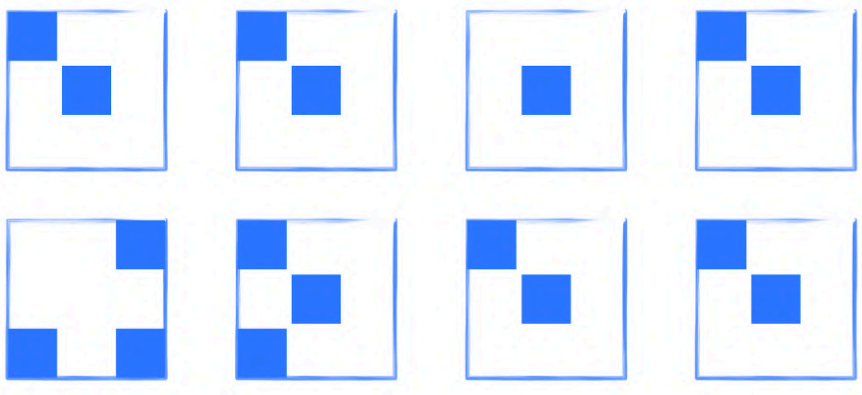
A GAN does not learn to memorise examples from the training data. A GAN doesn't even learn to memorise small parts of the training data examples. For training data consisting of faces, this means the generator doesn't memorise elements like eyes, ears, lips, or noses.

The generator doesn't see the training data directly. All it can learn from is the back-propagated error feedback from a discriminator which itself can only make a binary conclusion that an image is real or fake.

GANs instead learn the **likelihood** of elements appearing in the training data.

Simplified Example

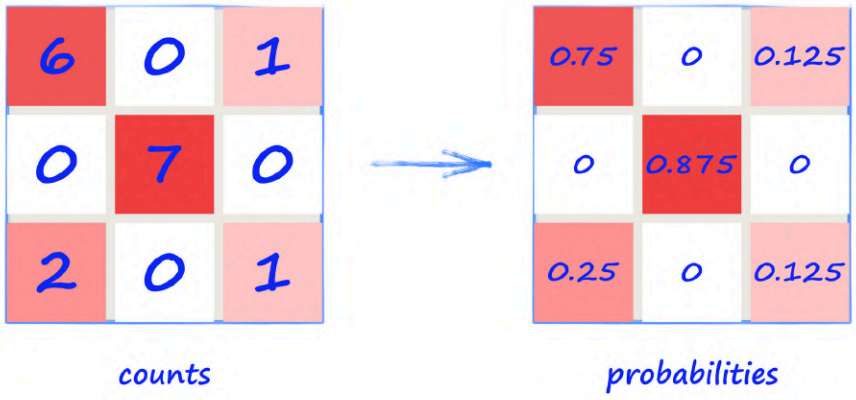
The following picture shows a very tiny dataset of just **8** images. The images are also tiny, **3** pixels by **3** pixels in size. Furthermore, the pixels can only be one of two values, which we'll show here as blue and white.



If we, as humans, were asked to draw images that looked like they could have come from this dataset, we would intuitively draw them with a blue pixel in the middle and top left. We might draw some that had the bottom left pixel coloured blue too.

Behind that intuition is an understanding of how **likely** a pixel is blue. We can see that most images have a blue middle pixel. In fact, all of them except one do. Many also have a blue top left pixel. A few have the bottom left pixel set to blue. Some pixels, like the top middle one, are never blue.

Let's move from intuition to actual calculation. We can count how often each pixel is blue. The first grid in the following picture shows these counts. The top left pixel is coloured blue in **6** of the **8** training images. The bottom left is blue in only **2** of them. The top middle pixel has a count of **0** because it is never blue in any of the training images.



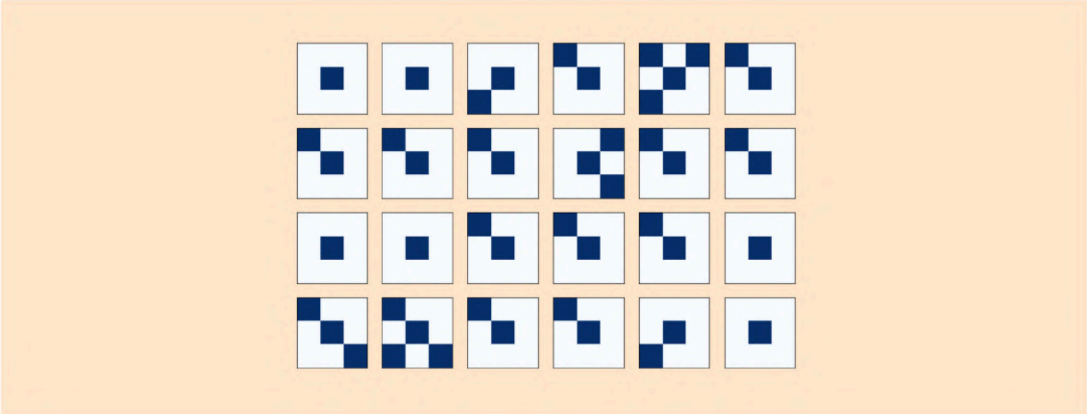
It's easy to convert these counts into measures of **likelihood**, or **probability**, by dividing each count by **8**, the maximum it could ever be. These probabilities are shown on the second grid in the picture above.

That grid is like a **probability distribution**. It shows how the **likelihood** of a pixel being blue is distributed over the **3** by **3** image.

Generating Images from A Probability Distribution

If we were a generator, we would look at those probabilities before deciding to colour a pixel blue or not. For the top left pixel, the chance of being blue is fairly high at **0.75**. For the top middle pixel the chance is **0** so we never colour it blue.

The following shows **24** different images created using this method.



We can see these images do look like they could have come from the training dataset.

The important point about creating images from a probability distribution is that we're not copying images, or bits of images, from the training data. We're making images with elements that match the **likelihoods** seen in the training data.

You can explore the code that converts the counts to probabilities and then generates images from them:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/Appendix_B_generate.ipynb

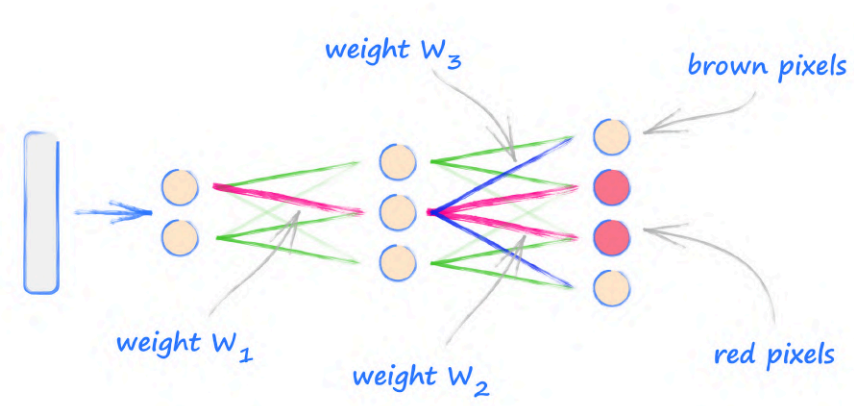
Learning Collections of Pixels For Image Features

In the above simplified example, we only considered the likelihood of individual pixels. This wouldn't actually work well for more realistic images like the MNIST digits or the CelebA faces.

Let's think about a smile in a photo of a face. If some pixels are coloured red for the lips, we need nearby pixels to also be red. We can't have those nearby pixels representing a different smile, with purple lips for example. The results would look messy and unrealistic.

What this tells us is that generator neural networks learn the likelihood of pixels being a particular colour in a way that links to neighbouring pixels too. If a generator produces a red pixel as part of a face's lips, the learned network weights will also make the neighbouring pixels likely to be red too.

The following picture illustrates this idea with a simplified generator network.



We can see how a strong weight **w1** activates the middle node in the middle layer. The signal from that node is continued by a strong weight **w2** to create red pixels. That same node, activated by weight **w1**, also allows a strong weight **w3** to draw skin coloured pixels that surround the red pixels. In this way the weights have jointly learned to draw a collection of red pixels for lips, and also non-red pixels for the face around the lips.

Many Modes & Mode Collapse

If we're training with the MNIST dataset, we want our generator to be able to draw all ten digits. That means learning the probability distributions for all ten digits.

The challenge for a generator is to simultaneously learn multiple probability distributions, one for each kind of image. It can do this by learning the right neural network weights so that different ranges of the random seed activate different paths through the network to match each probability distribution.

Intuitively we can see this is a lot to get just right, and points to why GANs are hard to train successfully.

When we've seen **mode collapse**, the generator has only learned the probability distribution of one class of image.

Appendix C: Convolution Examples

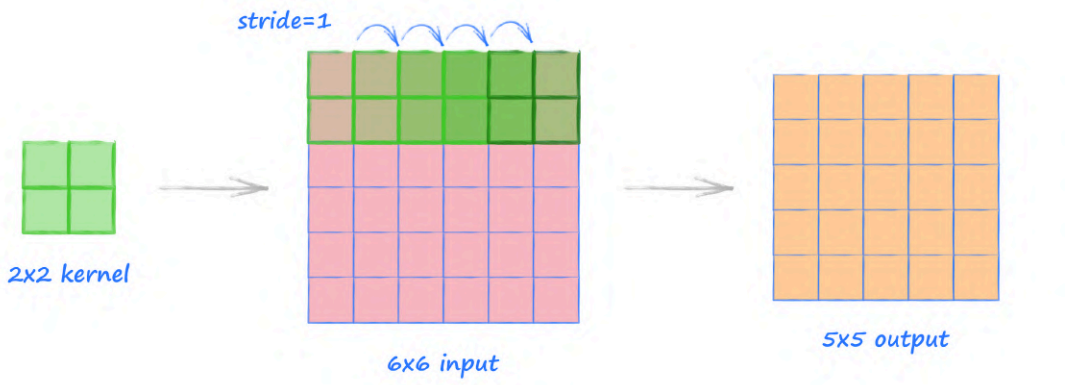
Convolution is common in neural networks which work with images, either as classifiers or as generators. When designing such convolutional neural networks, the shape of data emerging from each convolution layer needs to be worked out.

In this appendix we'll see how this can be done step-by-step with configurations of **convolution** that we're likely to see working with images.

In particular, **transposed convolutions** are seen as difficult to grasp. Here we'll show that they're not difficult at all by working though some examples which all follow a very simple recipe.

Example 1: Convolution With Stride 1, No Padding

In this first simple example we apply a **2** by **2** kernel to an input of size **6** by **6**, with **stride 1**.



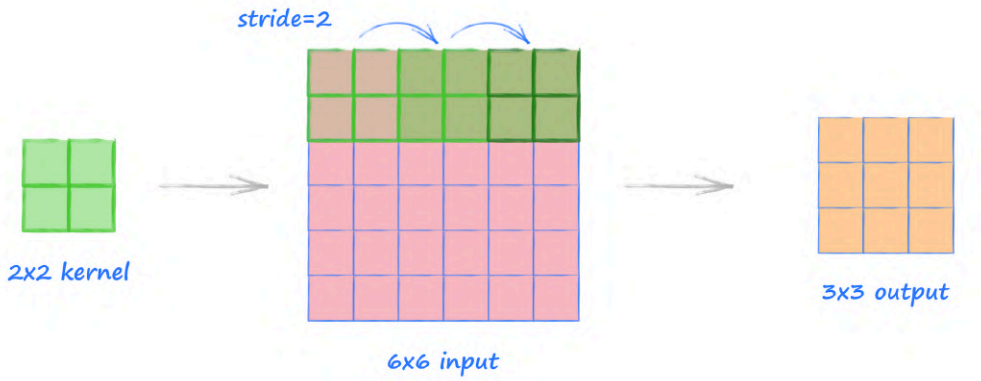
The picture shows how the kernel moves along the image in steps of size **1**. The areas covered by the kernel do overlap but this is not a problem. Across the top of the image, the kernel can take **5** positions, which is why the output is **5** wide. Down the image, the kernel can also take **5** positions, which is why the output is a **5** by **5** square. Easy!

The PyTorch function for this convolution is:

```
nn.Conv2d(in_channels, out_channels, kernel_size=2, stride=1)
```

Example 2: Convolution With Stride 2, No Padding

This second example is the same as the previous one, but we now have a **stride** of **2**.



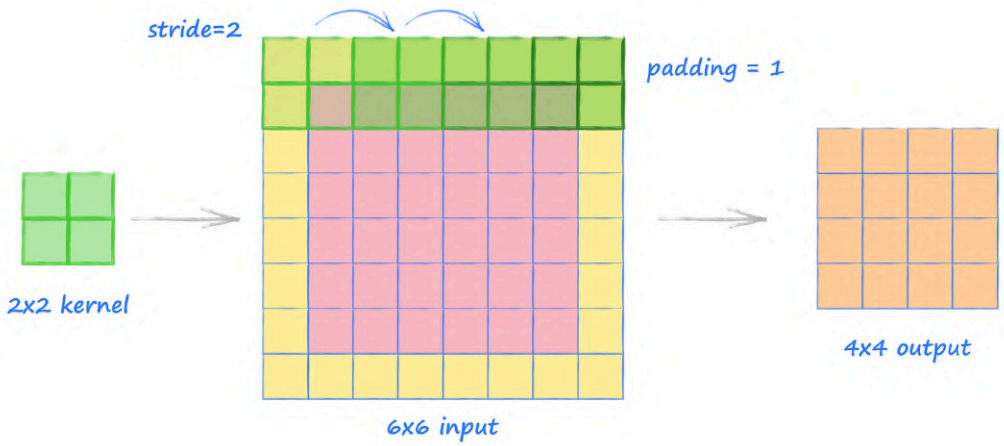
We can see the kernel moves along the image in steps of size **2**. This time the areas covered by the kernel don't overlap. In fact, because the kernel size is the same as the stride, the image is covered without overlaps or gaps. The kernel can take **3** positions across and down the image, so the output is **3** by **3**.

The PyTorch function for this convolution is:

```
nn.Conv2d(in_channels, out_channels, kernel_size=2, stride=2)
```

Example 3: Convolution With Stride 2, With Padding

This third example is the same as the previous one, but this time we use a **padding** of **1**.



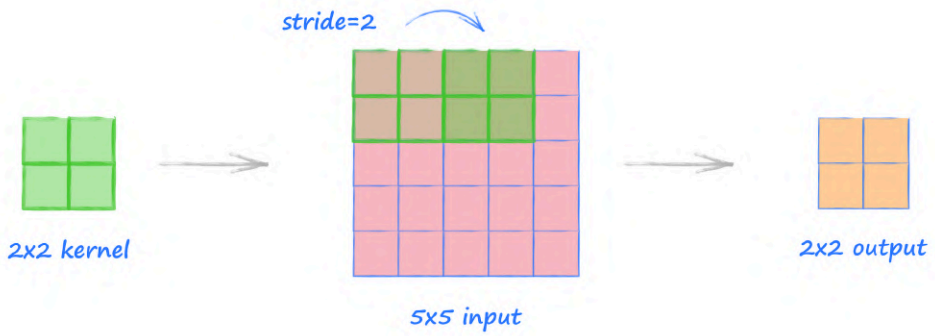
By setting padding to **1**, we extend all the image edges by **1** pixel, with values set to **0**. That means the image width has grown by **2**. We apply the kernel to this extended image. The picture shows the kernel can take **4** positions across the image. This is why the output is **4** by **4**.

The PyTorch function for this convolution is:

```
nn.Conv2d(in_channels, out_channels, kernel_size=2, stride=2,
padding=2)
```

Example 4: Convolution With Coverage Gaps

This example illustrates the case where the chosen kernel size and stride mean it doesn't reach the end of the image.



Here, the **2** by **2** kernel moves with a step size of **2** over the **5** by **5** image. The last column of the image is not covered by the kernel.

The easiest thing to do is to just ignore the uncovered column, and this is in fact the approach taken by many implementations, including PyTorch. That's why the output is **2** by **2**.

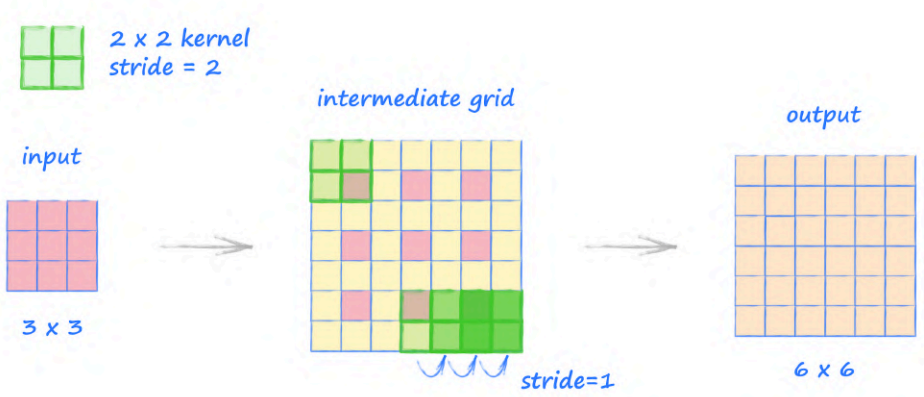
For medium to large images, the loss of information from the very edge of the image is rarely a problem as the meaningful content is usually in the middle of the image. Even if it wasn't, the fraction of information lost is very small.

If we really wanted to avoid any information being lost, we'd adjust some of the options. We could add padding to ensure no part of the input image was missed, or we could adjust the kernel and stride sizes so they match the image size.

Example 5: Transpose Convolution With Stride 2, No Padding

The **transpose convolution** is commonly used to expand a tensor to a larger tensor. This is the opposite of a normal convolution which is used to reduce a tensor to a smaller tensor.

In this example we use a **2** by **2** kernel again, set to stride **2**, applied to a **3** by **3** input.



The process for transposed convolution has a few extra steps but is not complicated.

First we create an intermediate grid which has the original input's cells spaced apart with a step size set to the stride. In the picture above, we can see the pink cells spaced apart with a step size of **2**. The new in-between cells have value **0**.

Next we extend the edges of the intermediate image with additional cells with value **0**. We add the maximum amount of these so that a kernel in the top left covers one of the original cells. This is shown in the picture at the top left of the intermediate grid. If we added another ring of cells, the kernel would no longer cover the original pink cell.

Finally, the kernel is moved across this intermediate grid in step sizes of **1**. This step size is always **1**. The stride option is used to set how far apart the original cells are in the intermediate grid. Unlike normal convolution, here the stride is not used to decide how the kernel moves.

The kernel moving across this **7** by **7** intermediate grid gives us an output of **6** by **6**.

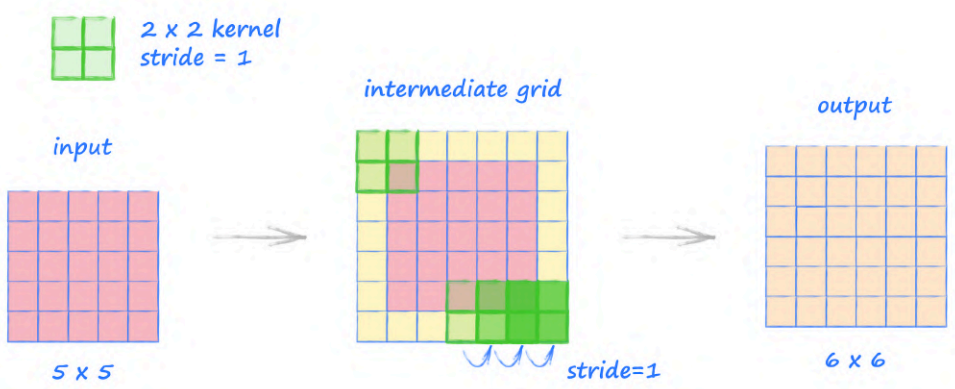
Notice how this transformation of a **3** by **3** input to a **6** by **6** output is the opposite of **Example 2** which transformed an input of size **6** by **6** to an output of size **3** by **3**, using the same kernel size and stride options.

The PyTorch function for this transpose convolution is:

```
nn.ConvTranspose2d(in_channels, out_channels, kernel_size=2, stride=2)
```

Example 6: Transpose Convolution With Stride 1, No Padding

In the previous example we used a stride of **2** because it is easier to see how it is used in the process. In this example we use a stride of **1**.



The process is exactly the same. Because the stride is **1**, the original cells are spaced apart without a gap in the intermediate grid. We then grow the intermediate grid with the maximum number of additional outer rings so that a kernel in the top left can still cover one of the original cells. We then move the kernel with step size **1** over this intermediate **7** by **7** grid to give an output of size **6** by **6**.

You'll notice this is the opposite transformation to **Example 1**.

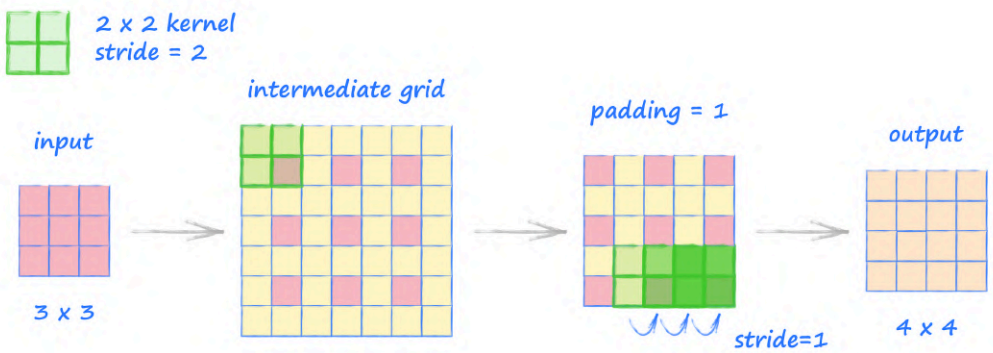
The PyTorch function for this transpose convolution is:

```
nn.ConvTranspose2d(in_channels, out_channels, kernel_size=2, stride=1)
```

Example 7: Transpose Convolution With Stride 2, With Padding

In this transpose convolution example we introduce **padding**. Unlike the normal convolution where padding is used to expand the image, here it is used to reduce it.

We have a **2** by **2** kernel with stride set to **2**, and an input of size **3** by **3**, and we have set **padding** to **1**.



We create the intermediate grid just as we did in **Example 5**. The original cells are spaced 2 apart, and the grid is expanded so that the kernel can cover one of the original values.

The padding is set to **1**, so we remove **1** ring from around the grid. This leaves the grid at size **5** by **5**. Applying the kernel to this grid gives us an output of size **4** by **4**.

The PyTorch function for this transpose convolution is:

```
nn.ConvTranspose2d(in_channels, out_channels, kernel_size=2, stride=2,
padding=1)
```

Calculating Output Sizes

Assuming we're working with square shaped input, with equal width and height, the formula for calculating the output size for a **convolution** is:

$$\text{output size} = \left\lfloor \frac{(\text{input size}) + 2*\text{padding} - (\text{kernel size} - 1) - 1}{\text{stride}} \right\rfloor + 1$$

The L-shaped brackets take the mathematical **floor** of the value inside them. That means the largest integer below or equal to the given value. For example, the floor of **2.3** is **2**.

If we use this formula for **Example 3**, we have **input size = 6**, **padding = 1**, **kernel size = 2**. The calculation inside the floor brackets is **(6 + 2 - 1 -1) /2 + 1**, which is **4**. The floor of **4** remains **4**, which is the size of the output.

Again, assuming square shaped tensors, the formula for **transposed convolution** is:

$$\text{output size} = (\text{input size} - 1)*\text{stride} - 2*\text{padding} + (\text{kernel size} - 1) + 1$$

Let's try this with **Example 7**, where the **input size = 3**, **stride = 2**, **padding = 1**, **kernel size = 2**. The calculation is then simply **2*2 - 2 + 1 + 1 = 4**, so the output is of size **4**.

On the PyTorch references pages you can read about more general formulae, which can work with rectangular tensors and also additional configuration options we've not needed here.

- **nn.Conv2d** https://pytorch.org/docs/stable/nn.html#conv2d
- **nn.ConvTranspose2d** https://pytorch.org/docs/stable/nn.html#convtranspose2d

Appendix D: Unstable Learning

Is Gradient Descent Suitable for Training GANs?

When training neural networks we use **gradient descent** to find a path down a loss function to find the combination of learnable parameters that minimise the error. This is a very well researched area and techniques today are very sophisticated, the Adam optimiser being a good example.

The dynamics of a GAN are different to a simple neural network. The generator and discriminator networks are trying to achieve opposing objectives. There are parallels between a GAN and adversarial games where one player is trying to maximise an objective while the other is trying to minimise it, each undoing the benefit of the opponent's previous move.

Is the gradient descent method suitable for such adversarial games? This might seem like an unnecessary question, but the answer is rather interesting.

Simple Adversarial Example

The following is a very simple objective function:

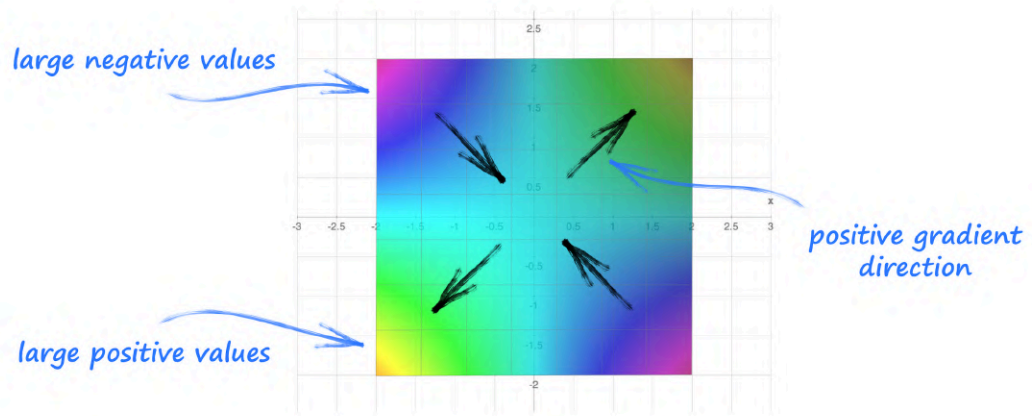
$$f = x \cdot y$$

One player has control over the values of **x** and is trying to maximise the objective **f**. A second player has control over **y** and is trying to minimise the objective **f**.

Let's visualise this function to get a feel for it. The following picture shows a surface plot of **f = x·y** from three different angles.

We can see that the surface of **f = x·y** is a **saddle**. That means, along one direction the values rise then fall, but in another direction, the values fall then rise.

The following picture shows the same function from above, using colours to indicate the values of **f**. Also marked are the directions of increasing gradient.



If we used our intuition to find a good solution to this adversarial game, we would probably say the best answer is the middle of that saddle at **(x,y) = (0,0)**. At this point, if one player sets **x = 0**, the second player can't affect the value of **f** no matter what value of **y** is chosen. The same applies if **y = 0**, no value of **x** can change the value of **f**. The actual value of **f** at this point is also the best compromise. Elsewhere there are as many higher values of **f** as there are lower, so this value should keep both players equally happy - or unhappy!

You can explore the surface interactively yourself using the **math3d.org** website:

- https://www.math3d.org/wz85eIlP
- https://www.math3d.org/x6xNjkaR

Let's now move away from intuition and work out the answer by simulating both players using gradient descent, each trying to find a good solution for themselves.

You'll remember from *Make Your Own Neural Network* that parameters are adjusted by a small amount that depends on the gradient of the objective function.

$$x \rightarrow x + lr * \delta f/\delta x$$

$$y \rightarrow y - lr * \delta f/\delta y$$

The reason we have different signs in these **update rules** is that **y** is trying to minimise **f** by moving down the gradient, but **x** is trying to maximise **f** by moving up the gradient. That **lr** is the usual learning rate.

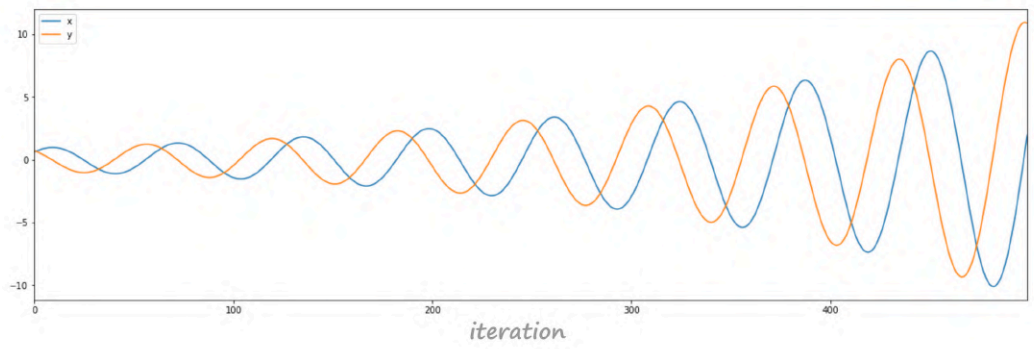
Because we know **f = x·y** we can write those update rules with the gradients worked out.

$$x \rightarrow x + lr * y$$

$$y \rightarrow y - lr * x$$

We can write some code to pick starting values for **x** and **y**, and then repeatedly apply these update rules to get successive **x** and **y** values.

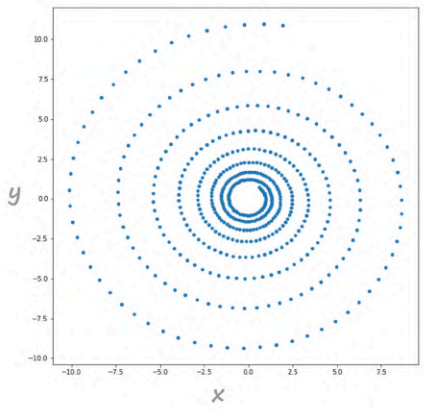
The following shows how **x** and **y** evolve as training progresses.



We can see that the values of **x** and **y** don't converge, but oscillate with ever greater amplitude. Trying different starting values leads to the same behaviour. Reducing the learning rate merely delays the inevitable **divergence**.

This is bad. It shows that gradient descent can't find a good solution to this simple adversarial game, and even worse, the method leads to disastrous divergence.

The following picture shows **x** and **y** plotted together. We can see the values orbit around the ideal point **(0,0)** but run away from it.



It can be shown mathematically (see below) that the best case scenario is that **(x,y)** orbits in a fixed circle around the **(0,0)** without getting closer to it, but this is only when the update step is infinitesimally small. As soon we have a finite step size, as we do when we approximate that continuous process in discrete steps, the orbit diverges.

You can explore the code which plays this adversarial game using gradient descent here:

- https://github.com/makeyourownneuralnetwork/gan/blob/master/Appendix_D_convergence.ipynb

Gradient Descent Isn't Ideal For Adversarial Games

We've shown that gradient descent fails to find a solution to an adversarial game with a very simple objective function. In fact, it doesn't just fail to find a solution, it catastrophically diverges. In contrast, gradient descent used in the normal way to minimise a function is guaranteed to find a minimum, even if it isn't the global minimum.

Does this mean GAN training will fail in general?

Realistic GANs with meaningful data will have much more complex loss surfaces, and that can reduce the chances of runaway divergence. That's why GAN training throughout this book has worked fairly well. But this analysis does indicate why training GANs is hard, and can become chaotic. Orbiting around a good solution might also explain why many simple GANs seem to progress onto different modes of single-mode collapse with extended training rather than improving the quality of images themselves.

Fundamentally, gradient descent is the wrong approach for GANs, even if it works well enough in many cases. Finding optimisation techniques designed for the adversarial dynamics in GANs is currently an open research question, with some researchers already publishing encouraging results.

Why A Circular Orbit?

Above we stated that **(x,y)** orbits as a circle when two players each use gradient descent to optimise **f = x·y** in opposite directions. Here we'll do the maths to show why it is a circle.

Let's look at the update rules again.

$$x \rightarrow x + lr * y$$

$$y \rightarrow y - lr * x$$

If we want to know how **x** and **y** evolve over time **t**, we can write:

$$dx/dt = lr * y$$

$$dy/dt = - lr * x$$

If we take the second derivatives with respect to **t**, we get the following.

$$d^2x/dt^2 \quad = + lr * dy/dt \quad = - lr^2 * x$$

$$d^2y/dt^2 \quad = - lr * dx/dt \quad = - lr^2 * y$$

You may remember from school algebra that expressions of the form $\mathbf{d^2y/dt^2 = - a^2x}$ have a solution the form **y = sin(at)** or **y = cos(at)**. To satisfy the first derivatives above, we need **x** and **y** to be the following combination.

$$x = \sin (lr * t)$$

$$y = \cos (lr * t)$$

These describe **(x,y)** moving around a unit circle with angular speed **lr**.

Appendix E: Acknowledgments

MNIST Dataset

The official source of the MNIST dataset is at Yann leCun's website:

- http://yann.lecun.com/exdb/mnist/index.html

The data is available under a Creative Commons Attribution-Share Alike 3.0 license:

- https://creativecommons.org/licenses/by-sa/3.0/

CelebA Dataset

The source of the CelebA dataset is at the Multimedia Laboratory, The Chinese University of Hong Kong:

- http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html

The dataset is for non-commercial research and educational purposes only. Permission has been obtained for this book to provide instructions for working with the dataset. Images from the dataset won't be reproduced here, and only GAN generated images will be shown.

NVIDIA And Google

NVIDIA® and CUDA™ are registered trademarks of NVIDIA®.

CoLaboratory™, often referred to as Colab, is a registered trademark of Google. Google and the Google logo are registered trademarks of Google LLC.

Open Source

I'd like to thank the open source community, without which we would never have such amazing software to use, knowledge to share, and friends to support and inspire us.